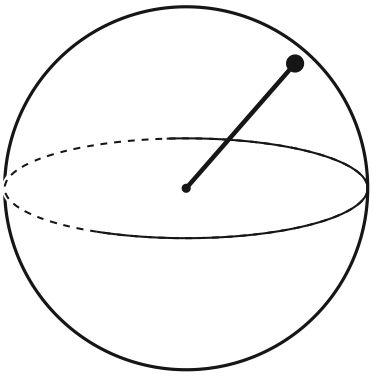
Quantum Computing, Gently

An introduction for people who have never studied physics or computer science

Kaiser



Free edition · Version 1.0 · January 2026

Quantum Computing, Gently
by Kaiser
Written in January 2026
Version 1.0, January 2026

© 2026 Kaiser.

This book is free. It is released under the Creative Commons Attribution-ShareAlike 4.0 International licence (CC BY-SA 4.0). You may copy it, share it, print it, translate it, adapt it and use it commercially, provided you credit the author and release any adapted version under the same licence. Licence text: https://creativecommons.org/licenses/by-sa/4.0/

All text and figures in this book are original work. Suggestions for further reading, including the standard graduate textbooks, are collected at the end.

Found a mistake, or know a better way to explain something? Corrections and improvements are welcome.

Contents

Preface

Who this book is for
What this book tries to do
How the book is organised
A word of encouragement
A free book

Part I: Foundations

Why build a quantum computer?

1.1 The end of an easy road
1.2 Feynman's question
1.3 From curiosity to headline
1.4 Some things a quantum computer is not
1.5 Where the word comes from
1.6 The road ahead

How an ordinary computer thinks

2.1 Everything is yes or no
2.2 Computing means changing bits
2.3 A circuit that adds
2.4 Algorithms: recipes for computation
2.5 Easy and hard problems
2.6 The factoring problem
2.7 Throwing information away

A small mathematical toolkit

3.1 Numbers with a sign
3.2 Squares and square roots
3.3 Arrows
3.4 Adding and stretching arrows
3.5 Angles and turning

3.6 Probability
3.7 Tables that transform arrows
3.8 Complex numbers, briefly

Light, waves and a puzzle

4.1 Waves that add and waves that cancel
4.2 Young's experiment
4.3 Light comes in lumps
4.4 A cleaner version: the beam splitter
4.5 Two beam splitters
4.6 The rule of amplitudes
4.7 What does it mean?

The qubit

5.1 Anything with two states
5.2 Kets
5.3 Superposition
5.4 What superposition is not
5.5 Complex amplitudes and phase
5.6 How much does a qubit hold?

Looking at a qubit

6.1 The measurement rule
6.2 Many runs, not one
6.3 Three polarising filters
6.4 Measuring in another basis
6.5 Measurement is not a passive glance
6.6 What counts as a measurement?

Part II: Computing with qubits

A qubit as a point on a globe

7.1 Two numbers are enough
7.2 Reading the sphere
7.3 Height is probability
7.4 Opposites on the sphere
7.5 Gates as rotations
7.6 Inside the sphere

Changing a qubit: quantum gates

8.1 What a gate is allowed to do
8.2 The X gate

8.3 The Z gate
8.4 The Hadamard gate
8.5 Phase gates and general rotations
8.6 Reading circuit diagrams
8.7 Measuring in any basis

Many qubits

9.1 Two qubits
9.2 Putting qubits side by side
9.3 The explosion
9.4 All numbers at once?
9.5 Measuring some of the qubits
9.6 How much does a register hold?

Entanglement

10.1 A state that cannot be split
10.2 Perfect correlation
10.3 Correlation in every direction
10.4 No faster-than-light messages
10.5 The four Bell states
10.6 Entanglement among many
10.7 Making entanglement

Two-qubit gates and quantum circuits

11.1 The controlled-NOT
11.2 Making a Bell pair
11.3 More two-qubit gates
11.4 The Toffoli gate and classical logic
11.5 Cleaning up after yourself
11.6 Universality
11.7 Size and depth

What the quantum world forbids

12.1 No copying
12.2 Why no-cloning matters
12.3 Telling states apart
12.4 Bell's game
12.5 Winning more often with entanglement
12.6 The experiments

Part III: Quantum algorithms

Interference as a way of computing

13.1 A question about a black box
13.2 Phase kickback
13.3 Deutsch's algorithm
13.4 The Deutsch–Jozsa algorithm
13.5 Finding a hidden string
13.6 The pattern

Teleportation and superdense coding

14.1 The problem
14.2 The protocol
14.3 Why it works
14.4 What teleportation does not do
14.5 Superdense coding
14.6 A symmetry

Finding rhythm: the quantum Fourier transform

15.1 Hearing the notes in a chord
15.2 Arrows going round a clock
15.3 How the transform detects a period
15.4 The quantum version
15.5 An exponentially fast circuit
15.6 The fine print
15.7 Phase estimation

Shor's algorithm: breaking numbers apart

16.1 Why factoring matters
16.2 Clock arithmetic
16.3 From factoring to rhythm
16.4 Quantum period finding
16.5 A small example
16.6 How close are we?

Grover's algorithm: a needle in a haystack

17.1 The search problem
17.2 The idea: grow the right amplitude
17.3 A rotation in a plane
17.4 The circuit
17.5 How good is it?
17.6 Amplitude amplification

Simulating nature

18.1 Why chemistry is hard
18.2 How quantum states change in time
18.3 Small steps
18.4 From molecules to qubits
18.5 Finding the energy
18.6 Where things stand

Part IV: Information, noise and machines

Quantum cryptography

19.1 The one-time pad
19.2 Two bases, again
19.3 The protocol
19.4 Catching the eavesdropper
19.5 From a noisy key to a secret key
19.6 Entanglement-based QKD
19.7 Quantum cryptography in practice

Noise: the enemy of every qubit

20.1 Every leak is a measurement
20.2 Two ways to forget
20.3 Errors from our own hands
20.4 How good are qubits today?
20.5 The noisy era
20.6 Why not just build better qubits?

Protecting quantum information

21.1 A classical warm-up
21.2 Encoding without copying
21.3 Asking the right question
21.4 Continuous errors become discrete
21.5 Phase flips too
21.6 The threshold theorem
21.7 The surface code

Building a quantum computer

22.1 What a qubit needs
22.2 Superconducting circuits
22.3 Trapped ions
22.4 Neutral atoms
22.5 Photons
22.6 Spins in silicon and other approaches

22.7 The rest of the machine
22.8 Which will win?

Measuring information

23.1 Surprise
23.2 Entropy
23.3 Quantum entropy
23.4 Entanglement makes parts uncertain
23.5 Holevo's limit revisited
23.6 Information is physical

What quantum computers can and cannot do

24.1 Sorting problems by difficulty
24.2 The quantum class
24.3 Evidence for a quantum advantage
24.4 The extended Church–Turing thesis
24.5 A rough map of applications

Where we stand and where we might go

25.1 Forty years in one page
25.2 The road to useful machines
25.3 Beyond computing
25.4 How to read the news
25.5 Where to go from here
25.6 A last word

Glossary

Hints and answers

Further reading

References

Preface

Newspaper articles about quantum computers tend to follow a pattern. They announce that these machines will soon break every password on the internet, design new medicines and "try every possible answer at once". Then, just as the reader leans in, they add a sentence that closes the door: *the details are, of course, extremely complicated.*

The details are not as forbidding as that sentence suggests. What usually goes wrong when someone tries to explain them is that half a dozen small steps get skipped, steps that seem obvious to the explainer only because they took them years earlier, slowly, with a lot of help. Start from bits, jump to superposition, mention the word "amplitude", and most listeners are gone. Nobody failed to understand; the staircase simply had missing stairs. This book is an attempt to put those stairs back.

Who this book is for

It is written for readers who start from nothing. You do not need to know any physics. You do not need to have studied computer science, and you do not need any mathematics beyond what most people meet by the age of fourteen: adding and multiplying, fractions, percentages, and perhaps a vague memory of Pythagoras and his triangle. If you can work out a 15% tip on a restaurant bill, you have enough to begin.

It is also written for people who have tried before and bounced off. Most introductions to quantum computing are aimed at students in their third or fourth year of a physics or mathematics degree. They are excellent books, and if you enjoy this one you should read them afterwards (some suggestions are given at the end). But they assume a fluency with linear algebra and probability that most people simply do not have, and they move at a pace that makes sense for a graduate course and very little sense for anyone else.

What this book tries to do

The aim is understanding, not memorisation. By the end you should be able to explain, in your own words, what a qubit is, why measurement is such a strange business, what entanglement does and does not allow, how a quantum computer could factor a large number, why quantum computers are so hard to build, and why error correction is the heart of the whole enterprise. You should also be able to read a news article about a "quantum breakthrough" and form your own opinion about whether it deserves the headline.

There is mathematics in this book, but it arrives gently and always with a reason. Chapter 3 is a small toolkit containing everything we need: numbers, arrows, rotations and probabilities. When a new idea appears later, it is introduced with pictures and worked examples before any symbol is written down. Wherever a calculation is shown, the text around it says in plain English what the calculation means, so that a reader who prefers to skim the symbols can still follow the argument.

The mathematical parts are not optional decoration, though. Quantum computing is, at heart, a mathematical idea that happens to be realised by physics. A version of the subject with all of the mathematics removed would be like a cookbook with all of the quantities removed: pleasant to read, but you could never actually cook anything. What remains here is exactly as much as is needed to really see how things work, and no more.

How the book is organised

The book has four parts.

Part I builds the foundations. We look at what an ordinary computer does, gather our mathematical tools, and then step into the quantum world through one famous experiment with light. By the end of Part I you will know what a qubit is and what happens when you look at one.

Part II is about computing with qubits. We learn to picture a single qubit as a point on a sphere, to change it with quantum gates, to combine several qubits, and to understand entanglement. We also meet the things the quantum world forbids, which turn out to be just as important as the things it allows.

Part III is about quantum algorithms, the recipes that make quantum computers interesting. It covers interference as a computational tool, teleportation, the quantum Fourier transform, Shor's factoring algorithm, Grover's search algorithm and the simulation of nature.

Part IV deals with the messier side: quantum cryptography, noise, error correction, the hardware that real laboratories are building, and some honest remarks about where the field stands and where it might go.

Each chapter ends with a short summary and a handful of exercises. The exercises are not tests. Most of them can be done in a few minutes with pencil and paper, and they are the quickest way to find out whether an idea has really settled in. Hints and answers to many of them are given at the back.

A word of encouragement

Some of the ideas in this book are genuinely strange. That is not a failure of explanation; it is a feature of nature. Physicists in the 1920s argued for years about what the new quantum theory meant, and some of those arguments are still going on. If at some point you find yourself thinking *that cannot possibly be true*, you are in excellent company. The usual cure is to work through one concrete example slowly, and then another. Familiarity is not the same as understanding, but it helps a great deal.

A free book

This book is free, in both senses. You may read it, print it, share it, translate it, use it in a classroom or rewrite parts of it, under the terms of the licence printed at the front. Knowledge of how these machines work should not be locked behind jargon and expensive textbooks. If you find an error, or if you find a better way to explain something, please improve it and pass it on.

Part I: Foundations

What ordinary computers do, a small mathematical toolkit, and a first look at the strange rules that govern very small things.

CHAPTER 1

Why build a quantum computer?

Imagine a library with a million shelves, each holding a million books. Somewhere in one of those books is a slip of paper with a phone number you need. There is no catalogue and no index, and the slip could be anywhere. What do you do?

There is really only one option. You start at the first shelf and open the books one by one. If you are lucky the slip turns up early; if you are unlucky it is in the very last book. On average you will have to open half of them, which is five hundred billion books.

An ordinary computer facing a problem of this kind does essentially the same thing, only very quickly. It is a tireless librarian who can open billions of books per second. For a great many tasks that is more than enough. But some problems are so large that even the fastest librarian imaginable would need longer than the age of the universe to work through them.

A quantum computer is not a faster librarian. It is a different kind of machine altogether, one that exploits rules of nature that never show up in everyday life. For certain carefully chosen problems those rules allow a completely different route to the answer, a route with dramatically fewer steps. The rest of this book is about what those rules are, which problems benefit, and why.

1.1 The end of an easy road

The computer on your desk, or in your pocket, contains billions of tiny switches called *transistors*. Each one can be switched on or off by an electrical signal, and modern chips flip them billions of times per second. Everything a computer does, from displaying this page to recognising your face in a photograph, is built out of these switching operations.

For about half a century the switches kept getting smaller, and computers kept getting faster and cheaper as a result. In 1965 Gordon Moore, who would later co-found Intel, noticed that the number of transistors that could be packed onto a chip was doubling at a steady rate, roughly every year at first and later every two years. This observation became known as *Moore's law*, and for decades it held with remarkable regularity. A smartphone today has more computing power than the machines that guided the Apollo spacecraft to the Moon, by a factor of many thousands.

Shrinking cannot go on forever. A modern transistor has features only a few dozen atoms across, and at that scale electrons begin to behave in ways that engineers would rather they did not. They leak through barriers that should stop them. They refuse to sit neatly in one place. These effects come from *quantum mechanics*, the physics of the very small, and for chip designers they have long been a nuisance to be fought.

The story of quantum computing begins with a reversal of that attitude. What if, instead of fighting quantum effects, we put them to work?

1.2 Feynman's question

In the early 1980s several physicists began asking a deceptively simple question. The most famous of them was Richard Feynman, a Nobel laureate known as much for his bongo drums and safe-cracking as for his physics. The question was this: *why is it so hard to simulate quantum physics on a computer?*

Chemists, for instance, would dearly like to predict how a new drug molecule will bind to a protein, or how a new material will conduct electricity. In principle these questions can be answered by calculating what the electrons in the molecules are doing. Electrons obey the rules of quantum mechanics, and it turns out that the amount of computation needed to follow them grows explosively with the number of particles. A handful of electrons is manageable. A few dozen is hard. A few hundred is beyond every computer that exists or is ever likely to exist, if we stick to the straightforward approach.

Feynman turned the problem around. Nature evidently has no difficulty "computing" what a molecule does; the molecule simply does it, in a fraction of a second. So perhaps we could build a computer that itself runs on quantum rules.

Such a machine would not need to imitate the quantum world laboriously. It would *be* a piece of the quantum world, arranged so that its behaviour answers the question we care about.

There is an old engineering trick that captures the spirit of this idea. Before powerful computers were available, the way water flowed around the piers of a planned bridge was studied by building a scale model of the river and letting real water run through it. Nobody tried to calculate the motion of every water molecule. The water did the calculation simply by flowing. A quantum computer is, very loosely, a model of this kind for the quantum world, except that it is programmable: the same machine can be arranged to imitate many different systems.

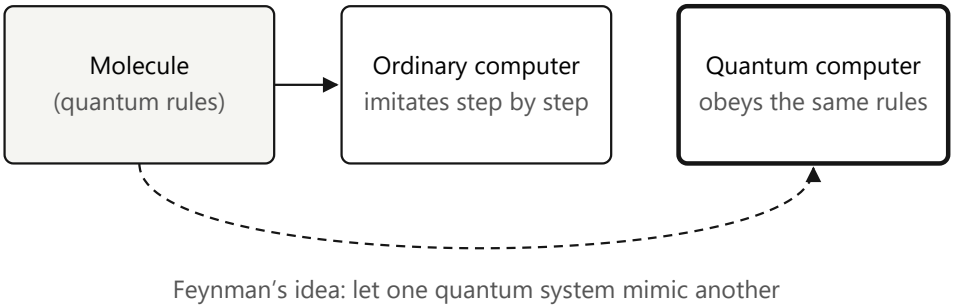


Figure 1.1. Two ways to predict what a molecule will do. An ordinary computer has to imitate quantum behaviour with ordinary arithmetic, which becomes hopelessly expensive for large molecules. A quantum computer follows the same physical rules as the molecule itself.

1.3 From curiosity to headline

For about a decade the quantum computer remained a curiosity discussed by a small group of theorists. Two discoveries in the mid-1990s changed that.

In 1994 Peter Shor, a mathematician at Bell Labs, showed that a quantum computer could solve a particular arithmetic problem astonishingly quickly: breaking a large whole number into its prime factors. That sounds like a school exercise, but it sits underneath a large part of the cryptography that protects internet traffic, including online banking and secure web pages. Those systems are

safe precisely because factoring a number hundreds of digits long would take ordinary computers an absurdly long time. Shor's result meant that a sufficiently large quantum computer could undo that protection.

Almost overnight the subject acquired a very different audience: intelligence agencies, banks, governments and large technology companies. Two years later, in 1996, Lov Grover found a second celebrated quantum algorithm, a way of searching an unstructured collection of possibilities faster than any ordinary computer can. The library at the start of this chapter is exactly that kind of problem.

At about the same time, Shor and Andrew Steane independently discovered that quantum information could be protected against errors, something many physicists had believed was impossible. Without that discovery quantum computers would have remained a beautiful idea that could never be built. With it, building one became an engineering problem: an extraordinarily hard one, but no longer a hopeless one.

Today there are working quantum computers in laboratories around the world, some of which can be used over the internet. They are still small and still make many errors. How large the gap is between these machines and truly useful ones is one of the questions we will look at honestly in the last part of the book.

1.4 Some things a quantum computer is not

Before going further it is worth clearing away a few misunderstandings, since you will meet them in news stories and online videos again and again.

The most common is the claim that a quantum computer "tries every possible answer at the same time". There is a grain of truth in it, which we will find in Chapter 9. But if a quantum computer could really compute every answer in parallel and then simply hand us the right one, it would be a miracle machine for *every* problem. It is not. The real mechanism is subtler and far more interesting. It depends on a phenomenon called *interference*, in which possibilities can reinforce or cancel one another the way water waves do, and the art of quantum programming lies in arranging for wrong answers to cancel and right answers to reinforce. Most of Part III is about how that can be done.

A quantum computer will not replace your laptop. For writing letters, watching films, running spreadsheets or playing games, ordinary computers are ideal and will stay that way. A quantum computer is a specialised instrument, closer in spirit to a particle accelerator or an MRI scanner than to a pocket calculator, and in practice it will probably always be used alongside ordinary computers, which prepare its inputs and interpret its outputs.

Nor is a quantum computer simply "faster". On most tasks it would be slower than an ordinary machine, because each of its operations is delicate and comparatively slow. Its advantage lies in taking a different route that needs vastly fewer steps, for particular problems.

A quantum computer cannot compute anything that an ordinary computer could not compute in principle. Given unlimited time and memory, an ordinary computer can simulate everything a quantum computer does. The difference lies entirely in the resources needed. That difference, however, can be the difference between a few hours and a few billion years.

Finally, the word "quantum" has nothing to do with healing crystals, "quantum energy" bracelets or any of the other products that borrow it for marketing. Quantum mechanics is one of the most precisely tested theories in all of science. Lasers, the transistors in every computer chip, LED lights and hospital MRI machines all depend on it.

1.5 Where the word comes from

"Quantum" is Latin for "how much". Around 1900 the German physicist Max Planck was trying to explain the colour of the light given off by hot objects (the dull red of an electric stove, the yellow-white of a light bulb filament). He found that the experimental data could only be explained if light carried energy in small fixed packets, not in arbitrary amounts. He called such a packet a *quantum*.

A rough everyday comparison is money. You can pay a bill of €3.47 but not one of €3.4718, because the smallest coin is one cent. Energy at the scale of atoms behaves somewhat like that: it comes in indivisible units. The units are so tiny that we never notice them in everyday life, just as nobody notices the individual cents in the national budget.

Planck himself regarded his packets as a mathematical trick and hoped they would eventually go away. They did not. Over the next three decades, through the work of Einstein, Bohr, Heisenberg, Schrödinger, Dirac and many others, a completely new theory of nature was built on top of them. This theory, *quantum mechanics*, describes how atoms, electrons and particles of light behave, and as we shall see their behaviour is, to put it mildly, unusual.

1.6 The road ahead

Our route through the subject is as follows. Chapter 2 looks at how an ordinary computer works, because we cannot appreciate what is new about a quantum computer without knowing what is old. Chapter 3 gathers the small amount of mathematics we need. Chapter 4 introduces the quantum world through one experiment with light that contains, in miniature, almost every strange feature we will meet later. Chapters 5 and 6 then define the qubit and describe what happens when it is measured. From there we build upwards, one floor at a time.

Chapter summary. Ordinary computers work by performing simple steps very quickly, and some problems are so large that no amount of speed is enough. Feynman proposed building computers that themselves obey quantum rules, first to simulate nature and later, after the work of Shor and Grover, to solve certain mathematical problems. Quantum computers are specialised instruments, not faster laptops, and their power does not come from "trying all answers at once" but from interference between possibilities.

Exercises

1.1 The library in this chapter has 10^{12} books. Suppose a librarian can check one billion (10^9) books per second. How long does it take, on average, to find the slip of paper? Now suppose the library has 10^{30} books. How long does it take then, and how does that compare with the age of the universe (about 4×10^{17} seconds)?

1.2 Think of three everyday problems that have the "library" character, in which there is no obvious shortcut and the only straightforward method is to try possibilities one after another. For each, estimate roughly how many possibilities there are.

1.3 A news headline claims that “a new quantum computer is a million times faster than a laptop”. Using what you have read in Section 1.4, write two questions you would want answered before believing it.

CHAPTER 2

How an ordinary computer thinks

To see what is new about a quantum computer, we first need a clear picture of what an ordinary computer does. We will not be concerned with processors, memory chips or operating systems. Only the underlying idea matters here, and it is surprisingly simple. Throughout the book we will call ordinary computers *classical* computers, which is the standard term; it means nothing more than "not quantum".

2.1 Everything is yes or no

At bottom, a classical computer distinguishes between just two states. A current flows or it does not. A tiny region of a hard disk is magnetised one way or the other. A pulse of light in a fibre-optic cable is present or absent. It is customary to label the two states 0 and 1, and a single piece of information of this kind is called a *bit*, short for *binary digit*.

A bit is like a light switch. It is either up or down; a well-made switch does not hover in between. A computer, from this point of view, is a vast panel of switches, billions of them, together with machinery for flipping them according to rules.

That sounds like a poor foundation for music, photographs and novels. The trick is to use many bits together. One bit can distinguish two things. Two bits can distinguish four:

00 01 10 11

Three bits can distinguish eight:

000 001 010 011 100 101 110 111

Each extra bit doubles the number of possibilities, because every existing pattern can be extended either with a 0 or with a 1. So n bits can be arranged in 2^n different ways.

Number of bits	Number of patterns
1	2
2	4
3	8
8	256
10	1,024
20	about one million
30	about one billion
64	about 18 billion billion

This doubling will play a leading role later in the book, so it is worth pausing over the table. Thirty bits is a tiny amount of memory (a short text message uses far more) and yet it can hold any one of a billion different patterns.

A group of eight bits is called a *byte*. With 256 patterns available, a byte is enough to give every letter, digit and punctuation mark on a keyboard its own code. In the common ASCII code, for instance, the capital letter A is 01000001. A photograph is stored as a long list of numbers saying how much red, green and blue light each tiny dot of the image contains. A song is stored as a long list of numbers describing the position of a loudspeaker membrane thousands of times per second. Everything a computer stores is, in the end, a long string of 0s and 1s.

Box 2.1: Counting in binary

Bit strings can be read as numbers, using base two instead of the familiar base ten. In base ten each position is worth ten times the one to its right: in 352, the 3 is worth 300, the 5 is worth 50 and the 2 is worth 2. In base two each position is worth twice the one to its right. Reading from the right, the positions are worth 1, 2, 4, 8, 16, and so on.

So the string 1011 means 1×8 + 0×4 + 1×2 + 1×1, which is eleven, and 110 means 1×4 + 1×2 + 0×1, which is six. With n bits we can write every whole number from 0 up to $2^n - 1$. When we later number the possible states of several qubits, we will use exactly this scheme.

2.2 Computing means changing bits

Storing bits is not enough; a computer has to compute. For a classical computer, computing means producing new bits from old ones according to fixed rules. The rules are built into tiny circuits called *logic gates*.

The simplest gate is NOT. It has one input and one output and simply flips the bit: 0 becomes 1 and 1 becomes 0.

Input	NOT
0	1
1	0

The AND gate has two inputs and one output. Its output is 1 only if both inputs are 1. You can think of a door with two locks, which opens only when both keys have been turned. The OR gate outputs 1 if at least one input is 1, like a door that opens with either of two keys. Finally the XOR gate, or “exclusive or”, outputs 1 when the two inputs are different. We will meet XOR again in quantum form; it is one of the most useful gates there is.

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Tables like these are called *truth tables*, and they describe a gate completely. Here is the remarkable fact: every computation a classical computer can perform can be broken down into a (possibly enormous) network of AND, OR and NOT gates. Adding numbers, sorting names, sharpening photographs, playing chess: all of it reduces to these three. A set of gates with this property is called *universal*. In fact even a single gate type suffices: the NAND gate ("not AND") on its own is universal, which is one reason chip manufacturers like it so much.

2.3 A circuit that adds

To see how this works in practice, let us build a circuit that adds two single bits. The possible sums are

$$0 + 0 = 0, \qquad 0 + 1 = 1, \qquad 1 + 0 = 1, \qquad 1 + 1 = 2.$$

In binary, 2 is written 10, so the answer needs two bits: a *sum* bit on the right and a *carry* bit on the left, just like the "carry the one" of long addition at school. Look at the sum bit for the four cases: 0, 1, 1, 0. That is exactly the XOR column of the truth table above. The carry bit is 0, 0, 0, 1, which is exactly AND. So a circuit that adds two bits is nothing more than an XOR gate and an AND gate side by side, both fed with the same inputs.

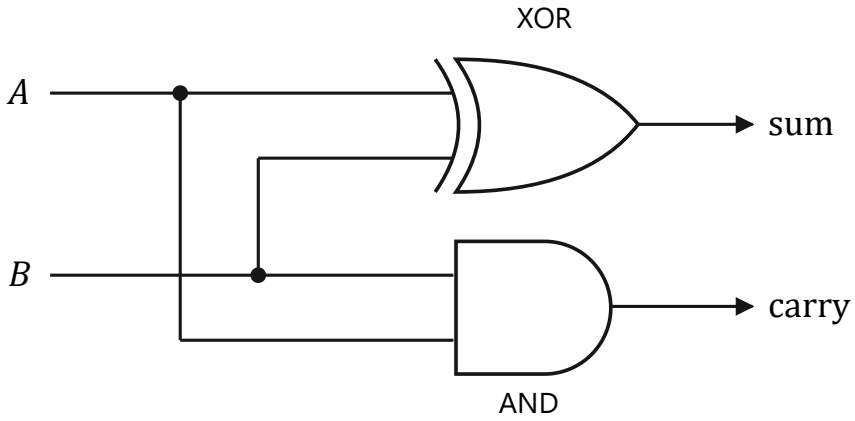


Figure 2.1. A half adder. The two input bits A and B are each copied to an XOR gate and an AND gate. The XOR output is the sum bit and the AND output is the carry bit. The curved shapes are the standard symbols engineers use for these gates.

This little circuit is called a *half adder*. Chain several of them together, with a few extra gates to handle incoming carries, and you can add numbers of any length. From adders you can build multipliers, from multipliers more elaborate arithmetic, and so on. A modern processor is, in essence, an unimaginably large and cleverly arranged network of simple gates like these.

Notice one detail in Figure 2.1 that is so ordinary it is easy to miss: the wires *split*. Bit A is sent both to the XOR gate and to the AND gate. Copying a bit in this way costs nothing in a classical computer. In Chapter 12 we will discover that the quantum world forbids it.

2.4 Algorithms: recipes for computation

An *algorithm* is a precise, step-by-step procedure for solving a problem. A cooking recipe is an algorithm, if a rather loose one. The long-multiplication method taught in primary school is an algorithm. A computer program is an algorithm written in a language a machine can follow.

For a given problem there are often many algorithms, and some are much faster than others. Suppose you want to find the name “Miller” in a printed phone book with 100,000 entries. One method is to start on the first page and read every name until you reach Miller, which in the worst case takes 100,000 steps. A better method is to open the book in the middle. If the name there comes after Miller in

the alphabet, Miller must be in the first half, and the second half can be ignored. Open the first half in its middle and repeat. Each step halves what is left, and after about seventeen steps (because 2^{17} is just over 100,000) you are looking at the right entry.

Seventeen steps instead of a hundred thousand: that is the difference a good algorithm can make. But notice why the second method works. It relies on the book being sorted. If the names were printed in random order, no clever halving would be possible and we would be back in the library of Chapter 1, checking entries one at a time. Remember this situation; in Chapter 17 we will find that a quantum computer has a genuinely new trick for it.

2.5 Easy and hard problems

Computer scientists are usually less interested in how long an algorithm takes on one particular input than in how the time *grows* as the inputs get larger. Let n stand for the size of the problem: the number of names in a list, the number of digits in a number, the number of cities on a map. Roughly speaking, algorithms then fall into two families.

In the first family, doubling the size of the problem multiplies the work by some fixed amount: doubling it, quadrupling it, perhaps multiplying it by eight. Long multiplication is like this; numbers with twice as many digits need about four times as much work. The number of steps grows like a power of n, such as n^2 or n^3, and such algorithms are called *polynomial-time*. In practice they are considered efficient, or "easy".

In the second family, increasing the size by just one multiplies the work by some fixed amount. The number of steps grows like 2^n, and the algorithm is called *exponential-time*. Such algorithms are considered inefficient, or "hard".

Box 2.2: Rice on a chessboard

An old story tells of a king who offered to reward the inventor of chess. The inventor asked only for rice: one grain on the first square of the board, two on the second, four on the third, eight on the fourth, each square holding twice as much as the one before. The king laughed at such a modest request and ordered it granted. But the 64th square alone would need 2^{63} grains, more than nine billion billion, which is more rice than has been harvested in all of human history.

Exponential growth always behaves like this. For the first few squares it is harmless, even unimpressive. Then it overwhelms everything.

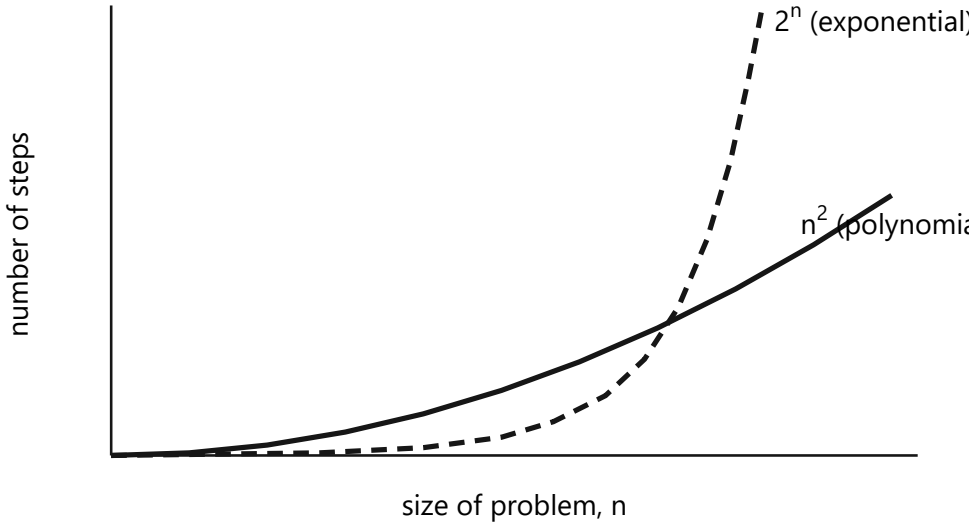


Figure 2.2. Polynomial growth (solid) and exponential growth (dashed). For small problems the two curves are close together, and the exponential one may even look better. Beyond a certain size the exponential curve leaves the page.

The distinction matters because faster hardware does not rescue an exponential algorithm. Suppose a computer a thousand times faster becomes available. With a polynomial algorithm growing like n^2, we can now handle problems about thirty times larger in the same time. With an exponential algorithm growing like 2^n, we can handle problems that are larger by only about ten, because 2^{10} is roughly a thousand. A thousandfold improvement in hardware buys almost nothing.

2.6 The factoring problem

A famous example of a problem for which every known classical algorithm is slow is *factoring*. You are given a whole number and asked to find the prime numbers that multiply together to give it.

A *prime number* is a whole number greater than 1 that can be divided evenly only by 1 and by itself: 2, 3, 5, 7, 11, 13, 17 and so on. Every other whole number can be written as a product of primes in exactly one way, for example $15 = 3 \times 5$ and $84 = 2 \times 2 \times 3 \times 7$.

What makes factoring interesting is that it is a one-way street. Multiplying is easy. Working out $1{,}709 \times 2{,}003$ takes a few minutes with pencil and paper and no time at all for a computer, and the same is true for numbers with hundreds of digits. Going backwards is hard. If I give you only the product, 3,423,127, and ask which two primes produced it, you must hunt around for a long time. For numbers several hundred digits long, the best known classical methods would run for longer than the universe has existed.

The best classical factoring algorithms are not quite as bad as exponential, but they are not polynomial either; they sit uncomfortably in between. This one-way street is what protects a large share of today's encrypted communication. In Chapter 16 we will see how Shor's algorithm turns it into a two-way street, at least for anyone who owns a large enough quantum computer.

2.7 Throwing information away

We end the chapter with an observation that seems minor now but will matter a great deal later.

Look again at the AND gate. If its output is 0, can you tell what its inputs were? You cannot: they might have been 00, 01 or 10. The AND gate *destroys information*. Its computation cannot be run backwards. The NOT gate, by contrast, can always be undone; if the output is 1, the input was 0, and vice versa. Gates that can be run backwards are called *reversible*.

In 1961 the physicist Rolf Landauer pointed out that destroying information has a physical cost: erasing one bit necessarily releases a tiny amount of heat. More important for us is a fact we will meet in Chapter 8. In the quantum world, left to itself, information is never destroyed. Every quantum gate is reversible. This might seem like a crippling restriction, but it is not. In the 1970s Charles Bennett and

others showed that any computation can be carried out using only reversible gates, provided we keep some extra bits around to remember where we came from. We will see how in Chapter 11.

Chapter summary. A classical computer represents information as bits, each 0 or 1, and n bits can take 2^n different values. It computes by applying logic gates such as NOT, AND, OR and XOR, and a small set of gates is enough to build any computation. What matters about an algorithm is how its running time grows with the problem size; polynomial growth is regarded as efficient and exponential growth as hopeless for large inputs. Factoring large numbers appears to be hard for classical computers, which is why it is used in cryptography. Some gates destroy information and some are reversible, and quantum gates are always reversible.

Exercises

2.1 Write the numbers 5, 9 and 13 in binary using four bits each. What number does 1111 represent?

2.2 How many different patterns can be stored in 40 bits? Give a rough answer in powers of ten, using the fact that 2^{10} is about 10^3.

2.3 Show that NOT can be built from a NAND gate by connecting both of its inputs to the same wire. (NAND outputs 0 only when both inputs are 1.)

2.4 Is the XOR gate reversible? If you know only its output, can you recover both inputs? What if the gate also passes input A through unchanged, so that it has two outputs, A and A XOR B?

2.5 A certain algorithm takes 2^n microseconds on a problem of size n. How long does it take for n = 20, n = 40 and n = 80? (One year is about 3×10^{13} microseconds.)

CHAPTER 3

A small mathematical toolkit

This chapter collects all of the mathematics we will need in the rest of the book. There is less of it than you might fear. Quantum computing rests on a handful of ideas: negative numbers, square roots, arrows that can be added and rotated, probabilities, and small tables of numbers that describe how arrows are transformed. If some of this is familiar, skim it. If it is all new, take it slowly and try the exercises at the end, because every later chapter leans on this one.

3.1 Numbers with a sign

Most everyday quantities are positive. A distance, a weight or a price is never less than zero. But some quantities have a direction as well as a size, and for those the sign matters. A bank balance can be +200 or −200, and the difference between them is rather important. A temperature can be above or below zero. A step can be taken forwards or backwards.

The feature of signed numbers that will matter most to us is that they can *cancel*. If you walk three steps forwards and then three steps backwards, you are back where you started: $+3 + (-3) = 0$. Two quantities, each perfectly real, add up to nothing.

In everyday probability this never happens. Chances are never negative, and adding two chances together can only make something more likely, never less. In quantum mechanics, as we shall see in Chapter 4, the basic quantities are not probabilities but signed numbers (and, more generally, complex numbers, which we meet in Section 3.8). That single fact is the source of nearly everything surprising in this book. Possibilities in quantum mechanics can cancel each other out.

Multiplying signed numbers follows the familiar rules: a positive times a negative is negative, and a negative times a negative is positive. So $(-2) \times 3 = -6$ and $(-2) \times (-3) = +6$.

3.2 Squares and square roots

The *square* of a number is the number multiplied by itself. The square of 3 is $3 \times 3 = 9$, written $3^2 = 9$. Because a negative times a negative is positive, the square of −3 is also 9. Squares are never negative. This will be important, because in quantum mechanics probabilities are obtained by squaring, and squaring is what turns signed numbers back into ordinary, non-negative chances.

The *square root* of a number asks the question in reverse: which positive number, multiplied by itself, gives this one? The square root of 9 is 3, written $\sqrt{9} = 3$. The square root of 2 is not a tidy number; it is about 1.414. One particular number will appear on almost every page of this book from Chapter 5 onwards:

$$1/\sqrt{2} \approx 0.7071$$

Its square is exactly one half: $(1/\sqrt{2}) \times (1/\sqrt{2}) = 1/2$. Keep this fact in mind. Whenever you see $1/\sqrt{2}$ in a quantum calculation, a fifty-fifty chance is usually lurking nearby.

3.3 Arrows

An arrow in a flat plane has a length and a direction. Physicists and mathematicians call such an arrow a *vector*. Wind is a good example: a weather report that says “20 km/h from the north-west” gives both a strength and a direction.

To describe an arrow with numbers, draw two perpendicular axes, one horizontal and one vertical, and place the tail of the arrow where they cross. Then record how far the tip lies to the right (or left, if negative) and how far up (or down, if negative). These two numbers are the arrow’s *components*. An arrow that goes 3 to the right and 4 up has components (3, 4).

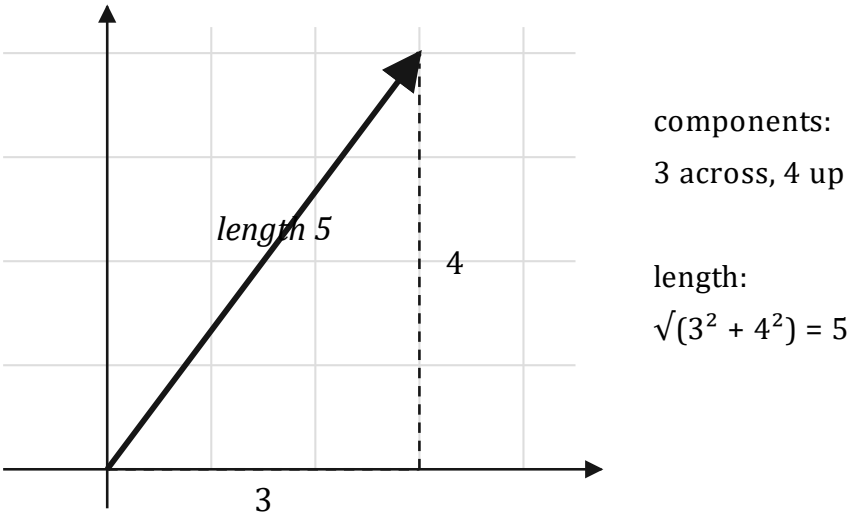


Figure 3.1. An arrow with components (3, 4). The components and the arrow form a right-angled triangle, so the arrow's length follows from Pythagoras' theorem.

The length of an arrow can be found from its components using Pythagoras' theorem, which you may remember from school: in a right-angled triangle, the square of the longest side equals the sum of the squares of the other two. For the arrow (3, 4) the length is $\sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$. In general, an arrow with components (a, b) has length $\sqrt{(a^2 + b^2)}$.

Arrows of length exactly 1 are called *unit vectors*, and they are the ones we will use most. The arrow (1, 0) points straight to the right and has length 1. The arrow (0, 1) points straight up and has length 1. A slightly more interesting example is $(1/\sqrt{2}, 1/\sqrt{2})$, which points diagonally up and to the right at 45°. Its length is

$$\sqrt{(\,(1/\sqrt{2})^2 + (1/\sqrt{2})^2\,)} = \sqrt{(\,1/2 + 1/2\,)} = \sqrt{1} = 1.$$

So it too is a unit vector. Notice that the squares of its components, 1/2 and 1/2, add up to 1. For any unit vector (a, b) we have $a^2 + b^2 = 1$. When we come to qubits, this innocent fact will turn out to say that probabilities add up to one.

3.4 Adding and stretching arrows

Two arrows can be added by placing them head to tail: start at the origin, follow the first arrow, then follow the second from wherever the first one ended. The sum is the arrow from the starting point to the final position. In components this is as simple as it could be. You add the across-parts and the up-parts separately:

$$(a, b) + (c, d) = (a + c, b + d).$$

For example (3, 1) + (1, 2) = (4, 3). And here is the point to remember: (1, 1) + (1, −1) = (2, 0). The upward part of the first arrow and the downward part of the second cancel each other exactly, leaving an arrow that points purely to the right.

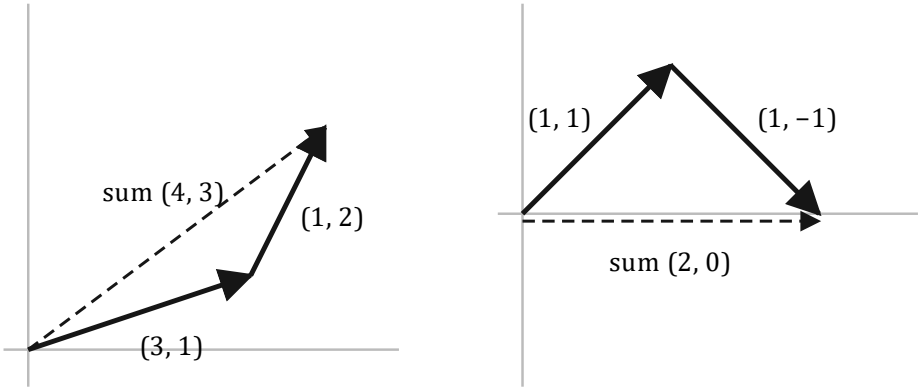


Figure 3.2. Adding arrows head to tail. On the left, (3, 1) + (1, 2) = (4, 3). On the right, (1, 1) + (1, −1) = (2, 0): the vertical parts cancel completely. Cancellation of this kind is the mechanism behind quantum interference.

An arrow can also be stretched or shrunk by multiplying it by an ordinary number, which multiplies both components: 2 × (3, 1) = (6, 2). Multiplying by a negative number flips the arrow round to point the opposite way: −1 × (3, 1) = (−3, −1). Multiplying by $1/\sqrt{2}$ shrinks it by about 29%. Combining these two operations, we can write any arrow in the plane as a mixture of the two basic arrows (1, 0) and (0, 1):

$$(a, b) = a \times (1, 0) + b \times (0, 1).$$

That may look like a statement of the obvious, and in the plane it is. But the same idea, writing a state as a weighted combination of basic states, is exactly how a qubit will be described in Chapter 5. There the two basic arrows will be called $|0\rangle$ and $|1\rangle$ instead of (1, 0) and (0, 1), and the weights will be called *amplitudes*.

3.5 Angles and turning

A unit vector is completely determined by its direction, which we can describe by an angle. Measure angles anticlockwise from the horizontal axis. The arrow (1, 0) has angle 0°, the arrow (0, 1) has angle 90°, and the diagonal arrow $(1/\sqrt{2}, 1/\sqrt{2})$ has angle 45°. As the angle increases, the tip of the unit arrow travels anticlockwise around a circle of radius 1, called the *unit circle*.

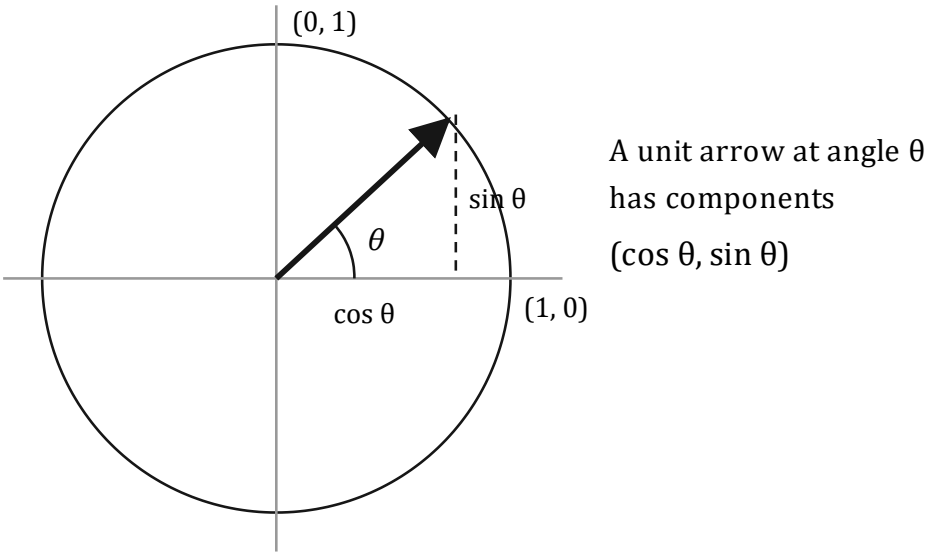


Figure 3.3. The unit circle. Every point on it is the tip of an arrow of length 1. The across-component of the arrow at angle θ is called the cosine of θ and the up-component is called the sine of θ.

The components of the unit arrow at angle θ (the Greek letter theta, which mathematicians use for angles) have names. The across-component is called the *cosine* of θ, written cos θ, and the up-component the *sine*, written sin θ. Calculators and spreadsheets compute them for you, and we will only ever need a few values:

angle θ	0°	45°	90°	135°	180°
cos θ	1	$1/\sqrt{2}$	0	$-1/\sqrt{2}$	−1
sin θ	0	$1/\sqrt{2}$	1	$1/\sqrt{2}$	0

Because the arrow has length 1, Pythagoras tells us that $(\cos\theta)^2 + (\sin\theta)^2 = 1$ for every angle. You do not need to remember any trigonometry beyond this table and this one identity.

Rotating an arrow means turning it about its tail without changing its length. A rotation by 90° anticlockwise takes (1, 0) to (0, 1) and takes (0, 1) to (−1, 0). Rotations are the single most important kind of operation in this book, because, as we will see in Chapter 8, quantum gates are essentially rotations. They change the direction of the state but never its length.

3.6 Probability

A *probability* is a number between 0 and 1 that measures how likely something is. Probability 0 means “never”, probability 1 means “certainly”, and probability 1/2 means “half the time”. A fair coin lands heads with probability 1/2. A fair die shows a six with probability 1/6. We will often write probabilities as percentages: 1/2 is 50% and 1/6 is about 16.7%.

The most useful way to think about a probability is as a long-run frequency. If you toss a fair coin a thousand times you will not get exactly 500 heads, but you will get something close, perhaps 487 or 512. The more times you repeat the experiment, the closer the fraction of heads comes to 1/2. This is important for quantum computing, because a single run of a quantum computation often gives a random answer, and we learn the underlying probabilities by repeating the run many times.

Two rules are all we need. First, if a situation has several possible outcomes, exactly one of which must happen, their probabilities add up to 1. A die must show one of its six faces, so the six probabilities 1/6 add up to 1. Second, if two independent things both happen, their probabilities multiply. The chance of two heads in two tosses of a coin is $1/2 \times 1/2 = 1/4$.

3.7 Tables that transform arrows

We now need a compact way to describe an operation that turns one arrow into another, such as a rotation. The standard tool is a *matrix*, a small rectangular table of numbers. For arrows in the plane, a matrix has two rows and two columns. Write the arrow as a column of two numbers instead of a row, and the rule for applying the matrix is this:

$$\begin{pmatrix} p & q \\ r & s \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} pa + qb \\ ra + sb \end{pmatrix}$$

In words: the new top number is the first row of the matrix "combined" with the arrow (first times first, plus second times second), and the new bottom number is the second row combined with the arrow in the same way. That is all there is to it. It may help to think of a matrix as a little machine with a recipe: the top row says how much of the old across-component and old up-component go into the new across-component, and the bottom row does the same for the new up-component.

Let us try two examples. The matrix with rows (0, 1) and (1, 0) swaps the two components:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a \end{pmatrix}$$

It turns (1, 0) into (0, 1) and (0, 1) into (1, 0). Geometrically it is a reflection in the diagonal line. In Chapter 8 this exact matrix will reappear as the quantum version of the NOT gate.

The second example has rows $(1/\sqrt{2}, 1/\sqrt{2})$ and $(1/\sqrt{2}, -1/\sqrt{2})$:

$$\begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$$

Applying it to (1, 0) gives $(1/\sqrt{2}, 1/\sqrt{2})$, the diagonal arrow at 45°. Applying it to (0, 1) gives $(1/\sqrt{2}, -1/\sqrt{2})$, the diagonal arrow pointing down at −45°. Now apply it to $(1/\sqrt{2}, 1/\sqrt{2})$:

$$\begin{aligned} \text{top:} \quad & 1/\sqrt{2} \times 1/\sqrt{2} + 1/\sqrt{2} \times 1/\sqrt{2} = 1/2 + 1/2 = 1, \\ \text{bottom:} \quad & 1/\sqrt{2} \times 1/\sqrt{2} - 1/\sqrt{2} \times 1/\sqrt{2} = 1/2 - 1/2 = 0. \end{aligned}$$

So the diagonal arrow goes back to (1, 0). Applying the matrix twice returns every arrow to where it started. Look at the bottom line of that calculation: two contributions of equal size and opposite sign cancelled to give zero. This matrix is called the *Hadamard matrix*, after the French mathematician Jacques Hadamard, and it is probably the most important single object in quantum computing. We will meet it properly in Chapter 8.

Box 3.1: Doing one thing after another

If we apply matrix A to an arrow and then apply matrix B to the result, the combined effect is again described by a single matrix, called the product BA. (The order of writing is right to left, because A acts first.) In general the order matters: putting on your socks and then your shoes is not the same as putting on your shoes and then your socks. Two rotations in the plane happen to commute, but most quantum operations do not, and the order in which gates are applied in a quantum circuit is crucial.

3.8 Complex numbers, briefly

There is one more kind of number used throughout quantum mechanics. We will keep our use of it to a minimum, but it is worth knowing what it is, so that the word holds no terror when it appears.

Ordinary numbers can be pictured as points on a line, with negative numbers to the left of zero and positive numbers to the right. Multiplying by −1 flips a number to the other side of zero, which is a half-turn of 180°. A natural question, first taken seriously by Italian mathematicians in the sixteenth century, is whether there is a number that performs a *quarter*-turn, so that doing it twice gives a half-turn. Such a number would satisfy i × i = −1. No number on the ordinary line can do that, so mathematicians extended the line into a plane. The new number i points straight "up", at right angles to the ordinary numbers, and is called the *imaginary unit*. The name is an accident of history; i is no more imaginary than −1 is.

A *complex number* is then simply a point in this plane, written a + bi, with an ordinary part a and an imaginary part b. You can think of it as an arrow with a length and a direction, exactly like the arrows of Section 3.3. Multiplying two

complex numbers multiplies their lengths and adds their angles. In particular, multiplying by a complex number of length 1 simply rotates an arrow without changing its length.

Why does quantum mechanics need them? Because the amplitudes that describe quantum states can point in any direction in this plane, not merely left or right. Two amplitudes of the same length can then cancel completely (if they point in opposite directions), reinforce each other (if they point the same way), or do something in between. The direction of an amplitude is called its *phase*. For most of this book we will get by with amplitudes that are ordinary positive or negative numbers, and only occasionally will we need a phase that points elsewhere. When we do, the arrow picture is all you need.

Chapter summary. Signed numbers can cancel, which ordinary probabilities cannot. Squares are never negative, and $1/\sqrt{2}$ squared is one half. Arrows (vectors) can be described by components, added head to tail, stretched and rotated; a unit vector (a, b) satisfies $a^2 + b^2 = 1$. Probabilities lie between 0 and 1 and add up to 1 over all possible outcomes. A 2×2 matrix is a recipe for transforming an arrow, and the Hadamard matrix, which turns (1, 0) into a diagonal arrow and back again, will be central later. Complex numbers are arrows in a plane, and their direction is called the phase.

Exercises

3.1 Find the length of the arrows (6, 8), (1, 1) and (0.6, −0.8). Which of them are unit vectors?

3.2 Compute (2, 5) + (−2, 1) and $(1/\sqrt{2}, 1/\sqrt{2}) + (1/\sqrt{2}, -1/\sqrt{2})$. In the second sum, which component cancels?

3.3 Apply the swap matrix of Section 3.7 to the arrow (0.6, 0.8). Then apply the Hadamard matrix to (0, 1), and apply it once more to the result. What do you get?

3.4 A coin is biased so that it lands heads with probability 0.64. What is the probability of tails? If you toss it twice, what is the probability of two heads?

3.5 Using the table in Section 3.5, check that $(\cos\theta)^2 + (\sin\theta)^2 = 1$ for $\theta = 135°$.

3.6 (For the curious.) Using the rule $i \times i = -1$, compute $(1 + i) \times (1 - i)$. What is the length of the arrow $1 + i$?

CHAPTER 4

Light, waves and a puzzle

The quickest route into the quantum world is not through atoms or electrons but through light. Light is something we all have experience of, and with a few mirrors and a sensitive detector it can be made to display, in a single tabletop experiment, nearly every feature that makes quantum computers possible. This chapter tells the story of that experiment. By the end of it we will have arrived at the central rule of quantum mechanics, and the rest of the book will be an exploration of its consequences.

4.1 Waves that add and waves that cancel

Drop a stone into a still pond and ripples spread out in rings. Drop two stones a short distance apart and the two sets of ripples run into each other. Where a crest from one stone meets a crest from the other, the water rises twice as high. Where a crest meets a trough, the two cancel and the water stays flat. Looking down on the pond you would see a pattern of calm lines radiating outwards between regions of choppy water. This adding and cancelling of waves is called *interference*. When waves reinforce each other we speak of *constructive* interference, and when they cancel, of *destructive* interference.

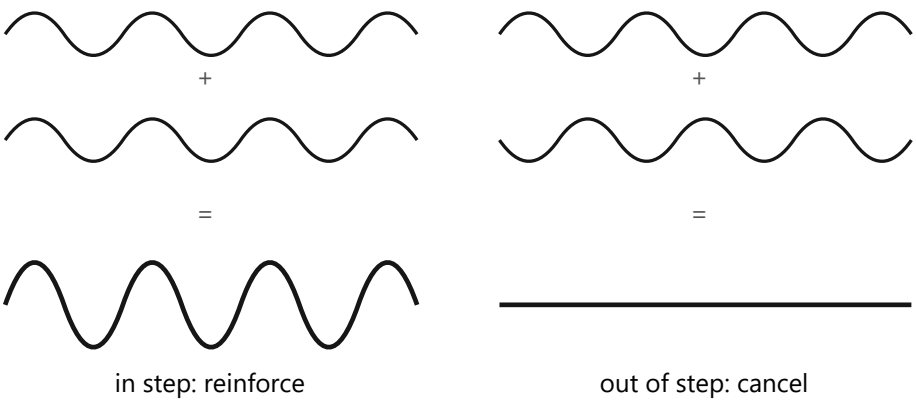


Figure 4.1. Interference. On the left, two waves whose crests line up add to give a wave twice as tall. On the right, the crests of one wave line up with the troughs of the other, and the two cancel completely.

You may own a device that uses destructive interference every day. Noise-cancelling headphones contain a small microphone that listens to the sound around you and a loudspeaker that plays the same sound back, but inverted, with every crest turned into a trough. The two sound waves cancel inside your ear, and the drone of the aeroplane engine fades away. Two sounds add up to silence.

Interference is the fingerprint of a wave. Ordinary objects such as tennis balls do not interfere. If you throw two balls at the same spot, you never find that they cancel and the spot is left untouched. Things that come in lumps can only add.

4.2 Young's experiment

At the start of the nineteenth century there was a lively argument about the nature of light. Isaac Newton had argued that light consisted of tiny particles, which he called corpuscles. Others, notably the Dutch scientist Christiaan Huygens, believed it was a wave. In 1801 the English physician and polymath Thomas Young settled the matter, or so it seemed, with a strikingly simple experiment.

Young let light from a single source fall on a card containing two narrow, closely spaced slits. On a screen behind the card the light from the two slits overlapped. If light were made of particles, one would expect two bright bands on the screen, one behind each slit. What Young saw instead was a row of bright and dark stripes, called *fringes*. At some places on the screen the light from the two slits arrived in

step and reinforced; at others it arrived out of step and cancelled, leaving darkness. Covering either one of the slits made the stripes disappear, leaving a single smooth patch of light.

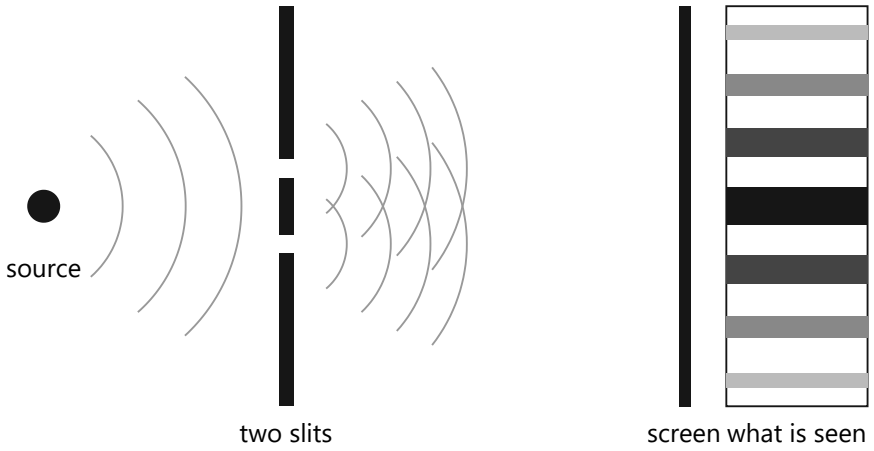


Figure 4.2. Young's double-slit experiment. Light spreading out from the two slits overlaps on the screen. Where the waves arrive in step the screen is bright, and where they arrive out of step it is dark, giving a pattern of stripes (shown on the right, with dark bands representing bright light).

The stripes are exactly what the wave theory predicts and exactly what a particle theory cannot explain. Over the following decades the wave picture of light was triumphantly confirmed. James Clerk Maxwell showed in the 1860s that light is a wave of electric and magnetic fields, and his equations remain the basis of radio, radar, mobile phones and much else. By the end of the nineteenth century, the question seemed closed.

4.3 Light comes in lumps

It did not stay closed. In 1905 Albert Einstein, taking Planck's energy packets more seriously than Planck himself had, proposed that light really does arrive in discrete lumps. He used the idea to explain the *photoelectric effect*, in which light knocks electrons out of a metal surface; the details of that effect made no sense if light were a smooth wave. These lumps of light are now called *photons*, and it was for this work, not for relativity, that Einstein received his Nobel Prize.

Today photons can be detected one at a time. A modern single-photon detector produces a sharp electrical "click" each time a photon arrives, and it never registers half a photon or a third of one. Light sources can also be made so dim that they emit, on average, only one photon at a time, with long gaps between photons.

So what happens if we repeat Young's experiment with such a source, sending photons through the slits one by one and recording where each lands on a sensitive screen?

Each photon arrives at a single point. It makes one small, sharp dot, just as a particle should. The position of each dot appears to be random. But as the dots accumulate, hundreds and then thousands of them, a pattern emerges. The dots avoid certain places and crowd into others, and gradually Young's stripes appear, assembled dot by dot.

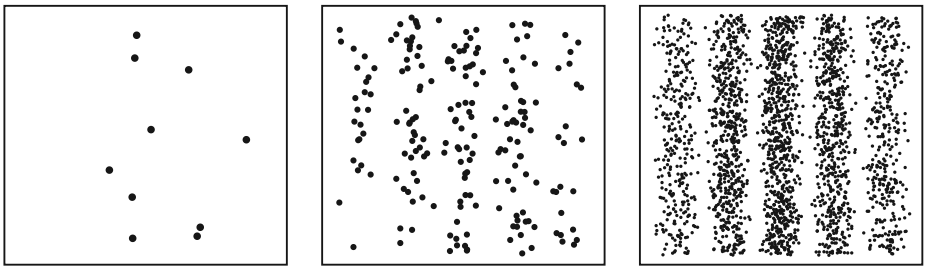


Figure 4.3. The double-slit experiment performed one photon at a time. Each photon lands at a single random point, but the points are not uniformly random: as more photons arrive, the stripes of Figure 4.2 build up. (Simulated data.)

This experiment, first carried out with very faint light in 1909 by Geoffrey Taylor and later repeated with great precision using single photons, electrons and even large molecules, is one of the most famous in all of physics. Richard Feynman called it a phenomenon that contains "the only mystery" of quantum mechanics.

Think about what it means. A single photon goes through the apparatus on its own. There is no second photon for it to interfere with. Yet the places where it may land are governed by an interference pattern that depends on *both* slits being open. If we close one slit, the stripes vanish, and photons start landing in places that were previously dark. Somehow each individual photon is affected by both routes through the apparatus.

The natural response is to look and see which slit each photon goes through. This can be done, for instance by placing a detector at each slit. And when it is done, the photon is always found at one slit or the other, never at both. But the moment we gather this "which-way" information, the stripes disappear. The photons now land in a smooth patch, exactly as if they were tennis balls passing through one slit or the other. The interference is there only when there is no way, even in principle, of telling which route the photon took.

4.4 A cleaner version: the beam splitter

The double-slit experiment has infinitely many possible landing points, which makes it awkward to analyse. Physicists prefer a version with only two routes and only two possible outcomes. It is built from a simple optical component called a *beam splitter*.

A beam splitter is a piece of glass coated with a very thin layer of metal, so thin that it reflects half of the light falling on it and lets the other half through. Shine a laser beam onto it and two beams of equal brightness come out: one passes straight through, and the other is reflected at a right angle.

Now send in a single photon. A photon cannot be split in half; our detectors only ever see whole photons. Place a detector in each of the two outgoing paths, and in each run of the experiment exactly one detector clicks. Which one is completely random, with probability 1/2 each. As far as anyone has been able to tell, this randomness is not due to some hidden detail we failed to control. It appears to be genuinely, fundamentally random. In fact devices like this are sold commercially as random number generators.

So far nothing strange has happened. A photon arriving at a beam splitter behaves like a coin being tossed.

4.5 Two beam splitters

Now for the experiment that matters. We arrange two beam splitters and two ordinary mirrors in a square, as in Figure 4.4. A photon enters at the lower left and meets the first beam splitter. It may pass straight through, taking the lower route, or be reflected upwards, taking the upper route. Each route uses a mirror to steer the photon towards the second beam splitter at the upper right, where the two routes meet again. Beyond the second beam splitter are two detectors, which we

will call A (to the right) and B (at the top). This arrangement is called a *Mach–Zehnder interferometer* after Ludwig Mach and Ludwig Zehnder, who devised it in the 1890s.

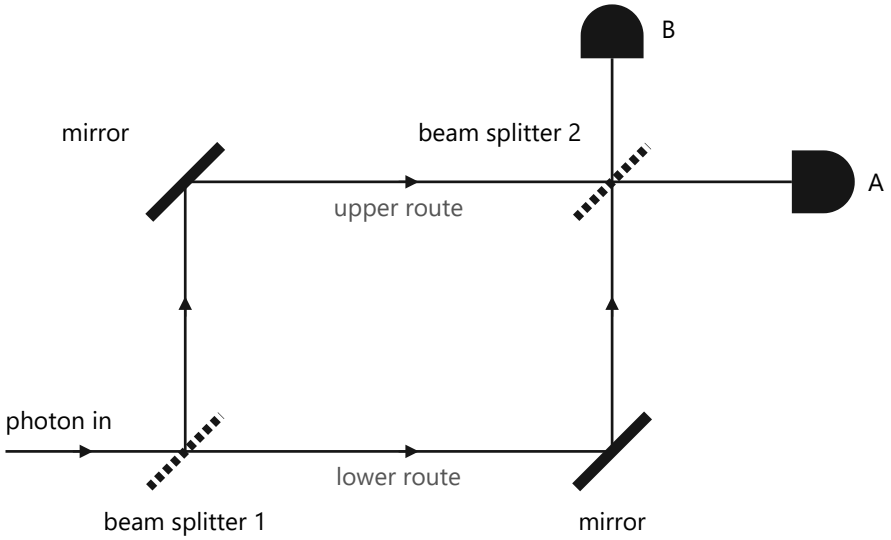


Figure 4.4. A Mach–Zehnder interferometer. A photon meets beam splitter 1 (dashed) and can take the lower or the upper route. Mirrors (solid) steer both routes to beam splitter 2, after which the photon is caught by detector A or detector B.

What should we expect? Let us reason as if the photon were a coin. At the first beam splitter it chooses a route at random, fifty-fifty. Whichever route it takes, it then arrives at the second beam splitter, where it is again sent one way or the other at random, fifty-fifty. Adding up the possibilities, detector A should click half of the time and detector B half of the time.

route	chance of route	then to A	then to B
lower	1/2	1/2 × 1/2 = 1/4	1/2 × 1/2 = 1/4
upper	1/2	1/2 × 1/2 = 1/4	1/2 × 1/2 = 1/4
total		1/2	1/2

That is the prediction of ordinary, coin-tossing probability. When the experiment is done, it is wrong. If the two routes are carefully adjusted to have exactly the same length, *detector A clicks every single time*. Detector B never clicks at all. Photon

after photon, all of them arrive at A.

Now block the upper route, by placing a piece of card in it. Half of the photons now hit the card and are absorbed, which is no surprise. But of the photons that get through, half go to A and half go to B. Detector B, which never clicked when both routes were open, now starts clicking. *Opening an extra route for the photon has made it impossible for the photon to reach B.*

There is no way to understand this with ordinary probabilities. Opening an additional route can never make an outcome less likely when chances simply add up. But it is exactly what we would expect of waves. With both routes open, the waves travelling along the two routes arrive at the second beam splitter and interfere: constructively in the direction of A, destructively in the direction of B. Block one route and there is nothing to interfere with.

And yet the photon is detected whole, at one detector, every time. If we put detectors in the routes themselves to see which one it took, we always find it in exactly one of them, and then, just as with the slits, the interference vanishes and A and B each click half of the time.

4.6 The rule of amplitudes

Physicists spent the first quarter of the twentieth century wrestling with puzzles of this kind. The rule they eventually arrived at is simple to state, even if it remains strange to contemplate. Here it is, in the form we will use throughout this book.

For each way an event can happen, quantum mechanics assigns a number called an *amplitude*. Amplitudes are like probabilities in that they measure how much each possibility counts, but unlike probabilities they can be negative (and, more generally, complex). Then:

1. When an event can happen in several ways that cannot be told apart, even in principle, we *add the amplitudes* for the different ways.
2. The probability of the event is the *square* of the total amplitude.

That is the whole rule. Let us apply it to the interferometer. First we need the amplitudes for what happens at a beam splitter. A photon can arrive travelling rightwards or travelling upwards, and in either case it can pass straight through or be reflected, so there are four cases. In three of them the amplitude is $1/\sqrt{2}$. In the fourth, a photon travelling upwards that passes straight through, the amplitude is $-1/\sqrt{2}$. Mirrors simply redirect the photon and leave its amplitude unchanged.

Where does the minus sign come from, and why that case in particular? It reflects the physics of the thin metal coating, and exactly where it sits is a matter of convention: physicists often describe beam splitters with complex amplitudes instead, and any consistent choice leads to the same predictions. What matters is that a minus sign (or more generally a phase) appears somewhere. Without it, nothing could ever cancel.

Now follow the photon through Figure 4.4. On the lower route it passes straight through beam splitter 1 while travelling rightwards, is turned upwards by the mirror, and so arrives at beam splitter 2 travelling upwards. There it is either reflected to the right, into detector A, or passes straight through upwards, into detector B. On the upper route it is reflected upwards at beam splitter 1, turned to the right by the mirror, and arrives at beam splitter 2 travelling rightwards. There it either passes straight through into A or is reflected upwards into B. The table collects the amplitudes for each step.

route	at splitter 1	at splitter 2, to A	at splitter 2, to B
lower (through, then up)	$1/\sqrt{2}$	reflected: $1/\sqrt{2}$	through: $-1/\sqrt{2}$
upper (reflected, then right)	$1/\sqrt{2}$	through: $1/\sqrt{2}$	reflected: $1/\sqrt{2}$

The amplitude for a whole route is the product of the amplitudes along it. So the lower route contributes $1/\sqrt{2} \times 1/\sqrt{2} = 1/2$ to A and $1/\sqrt{2} \times (-1/\sqrt{2}) = -1/2$ to B. The upper route contributes $1/2$ to A and $1/2$ to B. Because nothing in the experiment records which route was taken, we add the amplitudes for the two routes:

$$\text{amplitude for A} = 1/2 + 1/2 = 1, \qquad \text{amplitude for B} = -1/2 + 1/2 = 0.$$

Squaring gives the probabilities: $1^2 = 1$ for A and $0^2 = 0$ for B. Detector A always clicks and detector B never does, exactly as observed. The photon's two routes cancel each other in the direction of B, just as the arrows (1, 1) and (1, −1) cancelled vertically in Figure 3.2.

Now block the upper route. Only the lower route remains, so there is nothing to add. The amplitude for A is $1/2$, giving probability $1/4$, and the amplitude for B is $-1/2$, giving probability $1/4$ as well. The remaining probability of $1/2$ belongs to

the photon being absorbed by the card. Again this matches the experiment perfectly, including the strange fact that blocking a route makes B possible.

And if we put detectors in the routes to find out which one the photon took? Then the two routes *can* be told apart, the first part of the rule no longer applies, and we must square each route's amplitude separately and add the resulting probabilities. That gives $1/4 + 1/4 = 1/2$ for each detector, the coin-tossing answer. Information about the route destroys the interference.

Box 4.1: Why squares?

Why should probabilities be the squares of amplitudes, rather than the amplitudes themselves? One way to see that it has to be something like this is to notice that amplitudes can be negative, while probabilities cannot. Squaring is the simplest operation that turns any number into a non-negative one. A deeper reason is that squaring fits naturally with the arrow picture of Chapter 3: if the amplitudes of all outcomes are the components of a unit vector, then Pythagoras guarantees that their squares add up to exactly 1, which is what probabilities must do. This rule was first stated by Max Born in 1926 and is called the *Born rule*.

4.7 What does it mean?

What *is* the photon doing between the beam splitters? Does it take both routes? Neither? Is it a wave that turns into a particle when it is detected? Physicists have argued about these questions for a century, and several schools of thought (usually called *interpretations* of quantum mechanics) give different answers. Fortunately, for our purposes it does not matter. All interpretations agree on the rule of amplitudes and on every prediction it makes, and the rule is all a quantum computer needs.

The lesson to carry forward is this. In the quantum world, the possibilities for what might happen are not merely options waiting to be chosen at random. They carry amplitudes, and amplitudes from different possibilities can cancel. A quantum computer is a device for orchestrating this cancellation on a grand scale, so that the possibilities leading to wrong answers cancel each other and the possibilities leading to the right answer reinforce. Everything else is detail, albeit a great deal of fascinating detail.

Chapter summary. Waves interfere: they can reinforce or cancel. Young's double-slit experiment shows that light interferes, yet light is detected in indivisible lumps called photons, and even single photons build up an interference pattern. In a Mach–Zehnder interferometer a photon that could take two indistinguishable routes always reaches the same detector, and blocking one route makes the other detector click. The explanation is the rule of amplitudes: each way an event can happen has an amplitude, which can be negative; amplitudes of indistinguishable ways are added; and the probability is the square of the total. Gaining information about the route destroys the interference.

Exercises

4.1 Using the amplitude table in Section 4.6, suppose the lower route is blocked instead of the upper one. What are the probabilities that A clicks, that B clicks, and that the photon is absorbed?

4.2 Suppose we used a different convention, in which the minus sign belongs to a photon travelling *rightwards* that passes straight through a beam splitter, and all other cases have amplitude $+1/\sqrt{2}$. Redo the calculation of Section 4.6. Do the predictions for the detectors change? (This illustrates why the placement of the minus sign is only a convention.)

4.3 An experimenter adds an extra piece of glass to the upper route which changes the sign of that route's amplitude (multiplies it by -1). Which detector clicks now? Real experimenters use exactly this effect to switch photons between detectors.

4.4 In your own words, explain why ordinary probabilities can never produce the result that opening an extra route makes an outcome impossible.

CHAPTER 5

The qubit

In Chapter 4 a photon travelling through an interferometer had two possible routes, and the whole behaviour of the experiment was captured by two numbers: the amplitude for the lower route and the amplitude for the upper route. That is a very small amount of description for a very surprising amount of behaviour. In this chapter we take that observation and turn it into the basic unit of quantum information, the *quantum bit* or *qubit*.

5.1 Anything with two states

A classical bit is any physical system with two clearly distinguishable states, one of which we call 0 and the other 1. The physical details do not matter: a bit can be a voltage, a magnetised spot or a hole punched in a card. In the same way, a qubit is any quantum system with two distinguishable states, which we label 0 and 1. Here are a few of the systems that are used as qubits in laboratories today.

A photon can be a qubit in several ways. The two routes of the interferometer give one; we could call the lower route 0 and the upper route 1. A more common choice is the *polarisation* of the photon, the direction in which its electric field vibrates, which can be horizontal (0) or vertical (1). Polarised sunglasses work by blocking light of one polarisation.

An electron behaves in some ways like a tiny magnet, and when it is placed in a magnetic field its north pole can point either along the field or against it. Physicists call this property *spin*, and the two states are called spin up and spin down. They make a natural qubit.

An atom or ion can sit in its lowest-energy state, the *ground state*, or in one of its higher-energy *excited* states. If we pick the ground state and one particular excited state and arrange things so that the atom never wanders into any other

state, those two levels form a qubit. Several of the most advanced quantum computers use ions trapped by electric fields in exactly this way.

Finally, a tiny loop of superconducting metal, cooled to within a hair of absolute zero, can carry electrical oscillations that only come in discrete energy levels, like the levels of an atom. The two lowest levels form a qubit. These *superconducting qubits* are used in the machines built by several large technology companies.

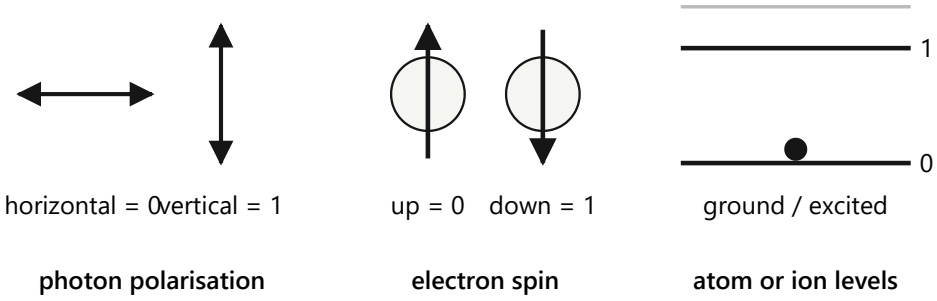


Figure 5.1. Three physical systems that can serve as qubits. Each has two distinguishable states labelled 0 and 1. For the atom, higher levels (grey) exist but are kept out of play.

As with classical bits, the physical details matter enormously to the engineers who build the machines, and we will come back to them in Chapter 22. For understanding how a quantum computer computes, though, they do not matter at all. Every one of these systems obeys the same mathematical description, and that description is what this chapter is about.

5.2 Kets

We need a notation for the states of a qubit. The standard one was invented by the English physicist Paul Dirac in 1939, and it is used everywhere in quantum computing. The state 0 is written

$$|0\rangle$$

and the state 1 is written $|1\rangle$. The odd-looking bracket is called a *ket*. (Dirac was fond of puns: a full bracket $\langle\ \rangle$ is split into a "bra" $\langle\ |$ and a "ket" $|\ \rangle$.) There is nothing mysterious about kets. They are simply labels for states, in the same way

that a name on a door is a label for a room. Anything can go inside a ket: $|0\rangle$, $|1\rangle$, $|\text{up}\rangle$, $|\text{lower route}\rangle$, or a Greek letter such as $|\psi\rangle$ (psi) when we want to talk about a general state without saying which one.

Behind the notation lies the arrow picture of Chapter 3. The state $|0\rangle$ is the unit arrow pointing along the horizontal axis, (1, 0), and the state $|1\rangle$ is the unit arrow pointing along the vertical axis, (0, 1). Physicists call $|0\rangle$ and $|1\rangle$ the *computational basis states*, or just the *basis states*.

5.3 Superposition

If a qubit could only be in $|0\rangle$ or $|1\rangle$, it would be nothing more than a bit with a fancy name. What makes it different is that it can also be in states that are combinations of the two. Recall from Chapter 3 that any arrow in the plane can be written as a weighted sum of the two basic arrows. In ket notation, the general state of a qubit is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{5.1}$$

where α (alpha) and β (beta) are numbers called the *amplitudes* of the state. They play exactly the role that the route amplitudes played in the interferometer. Such a combination is called a *superposition* of $|0\rangle$ and $|1\rangle$.

The amplitudes cannot be just any numbers. Because the state is a unit arrow, its length must be 1, and so

$$\alpha^2 + \beta^2 = 1. \tag{5.2}$$

This condition is called *normalisation*, and in Chapter 6 we will see that it is precisely the statement that probabilities add up to one.

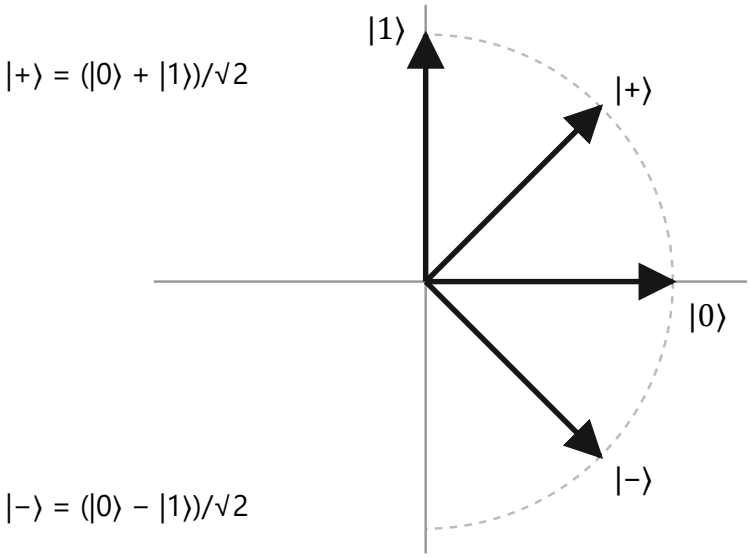


Figure 5.2. Qubit states drawn as unit arrows. The basis states |0⟩ and |1⟩ point along the axes. The states |+⟩ and |−⟩ point diagonally, and are equal superpositions of |0⟩ and |1⟩ that differ only in the sign of the |1⟩ amplitude.

Two superpositions are so important that they have their own names. The first is

$$|+\rangle = (1/\sqrt{2})|0\rangle + (1/\sqrt{2})|1\rangle,$$

the diagonal arrow at 45°. The second is

$$|-\rangle = (1/\sqrt{2})|0\rangle - (1/\sqrt{2})|1\rangle,$$

the diagonal arrow at −45°. Both have amplitudes of equal size on |0⟩ and |1⟩; they differ only in a sign. Check that each satisfies the normalisation condition: $(1/\sqrt{2})^2$ + $(\pm 1/\sqrt{2})^2$ = 1/2 + 1/2 = 1. The two states are at right angles to each other, just as |0⟩ and |1⟩ are, and we will see in Chapter 6 that this makes them perfectly distinguishable from each other, in the right kind of measurement.

You may notice that the photon in the interferometer, just after the first beam splitter, was in exactly the state |+⟩, if we call the lower route |0⟩ and the upper route |1⟩: it had amplitude $1/\sqrt{2}$ for each route. The second beam splitter then converted that state into the definite state “detector A”. In Chapter 8 we will recognise both beam splitters as the same quantum gate.

5.4 What superposition is not

The word “superposition” invites misunderstanding, and it is worth going slowly here.

A common description says that a qubit in the state $|+\rangle$ “is 0 and 1 at the same time”. This is not exactly wrong, but it is not a very helpful picture either, because it suggests that the qubit secretly contains two values side by side, like a box holding two coins. Nothing of the kind can be found by any experiment. Whenever we look at the qubit, as we will see in the next chapter, we find either 0 or 1, never both.

The opposite misunderstanding is more tempting and more dangerous. It says that a qubit in the state $|+\rangle$ is really either 0 or 1, we just do not know which, so that the superposition merely describes our ignorance. Imagine a coin that has been tossed and covered with a cup. It definitely shows heads or tails; we simply have not looked. Is $|+\rangle$ like that?

The interferometer says no. A photon that has passed the first beam splitter has amplitudes for both routes, and if it were secretly on one route or the other we would get the coin-tossing answer at the detectors: half the photons at A and half at B. Instead, all of them arrive at A. The state $|+\rangle$ is something that a hidden coin under a cup cannot imitate.

There is a cleaner way to see the difference. Consider the two states $|+\rangle$ and $|-\rangle$. If you could only ever look at whether a qubit is 0 or 1, these two states would appear identical: each gives 0 half of the time and 1 half of the time, just like a fair coin under a cup. But they are not the same state, and they are not the same as the coin. As we will see in Chapter 8, a single operation (the same one the beam splitter performs) turns $|+\rangle$ into $|0\rangle$ with certainty and $|-\rangle$ into $|1\rangle$ with certainty, whereas it leaves a hidden coin as random as before. The relative sign between the amplitudes, which is invisible if you look only at 0 or 1, is real and can be brought to light by interference.

So superposition is neither “both at once” in a naive sense nor “one or the other, unknown”. It is a third kind of thing, with no exact counterpart in everyday life, and the best description of it is the mathematical one: an arrow pointing in some direction in the plane spanned by $|0\rangle$ and $|1\rangle$.

Box 5.1: Schrödinger's cat

In 1935 Erwin Schrödinger, one of the founders of quantum mechanics, described a thought experiment designed to show how strange superposition would be if it applied to everyday objects. A cat is sealed in a box together with a radioactive atom, a detector and a vial of poison. If the atom decays, the detector triggers the release of the poison. After an hour the atom is in a superposition of "decayed" and "not decayed", and so, Schrödinger argued, the cat should be in a superposition of dead and alive.

Schrödinger intended this as a criticism, to show that something was missing from the theory as it was then understood. The modern understanding, which we will meet in Chapter 20, is that large objects constantly interact with their surroundings, and this interaction very quickly destroys the delicate phase relations that superposition depends on. This process, called *decoherence*, is why we never see cats in superposition, and it is also the greatest enemy of the quantum computer.

5.5 Complex amplitudes and phase

So far our amplitudes have been ordinary positive or negative numbers. In full generality, quantum mechanics allows them to be complex numbers, the arrows-in-a-plane of Section 3.8. Every complex number has a size and a direction, and we write $|\alpha|$ for the size of α. The normalisation condition then reads

$$|\alpha|^2 + |\beta|^2 = 1.$$

One example of a qubit state with a complex amplitude is

$$(1/\sqrt{2})|0\rangle + (\mathrm{i}/\sqrt{2})|1\rangle,$$

where the amplitude of $|1\rangle$ points "upwards" in the complex plane instead of to the right (positive) or to the left (negative). This state is neither $|+\rangle$ nor $|-\rangle$ but something in between, and it will appear on the sphere picture of Chapter 7.

There is one subtlety. If we multiply both amplitudes by the same complex number of size 1, for instance by -1, the state changes on paper but nothing observable changes. The state $-|0\rangle$ behaves in every possible experiment exactly like $|0\rangle$. Such an overall factor is called a *global phase*, and it can always be ignored.

Only the *relative* phase between the amplitudes, such as the sign that distinguishes $|+\rangle$ from $|-\rangle$, has any physical effect. It is the relative phase that controls interference.

5.6 How much does a qubit hold?

A classical bit holds exactly one of two values. A qubit, by contrast, is described by two amplitudes, which can be any numbers satisfying the normalisation condition. There are infinitely many such pairs. Does that mean a qubit can store an infinite amount of information, perhaps an entire encyclopaedia encoded in the hundredth decimal place of α?

It does not, and the reason is measurement. As we are about to see, looking at a qubit gives only a single bit, 0 or 1, and destroys the superposition in the process. The amplitudes govern the probabilities of the answers but cannot be read off directly. A theorem proved by Alexander Holevo in 1973 makes this precise: no matter how cleverly you encode classical information into qubits and how cleverly you measure them, you can never get out more than one bit per qubit.

This is a recurring theme. The amount of description needed to specify a quantum state is enormous, and grows explosively with the number of qubits, as we will see in Chapter 9. The amount of information that can be *extracted* from the state is small. Quantum algorithms live in the gap between the two. They let the huge hidden description do the work and then arrange, through interference, for the small amount of information we can extract to be the information we want.

Chapter summary. A qubit is any quantum system with two distinguishable states, written $|0\rangle$ and $|1\rangle$. Its general state is a superposition $\alpha|0\rangle + \beta|1\rangle$, a unit arrow whose amplitudes satisfy $|\alpha|^2 + |\beta|^2 = 1$. Superposition is neither "both values at once" in a simple sense nor mere ignorance of a hidden value; the relative sign or phase between amplitudes is real and shows up through interference. The states $|+\rangle$ and $|-\rangle$ are equal superpositions differing only in sign. Global phases have no physical effect. Although a qubit is described by continuous amplitudes, only one bit of information can be extracted from it.

Exercises

5.1 Which of the following are valid qubit states? (a) $0.6|0\rangle + 0.8|1\rangle$; (b) $0.5|0\rangle + 0.5|1\rangle$; (c) $-|1\rangle$; (d) $(\sqrt{3}/2)|0\rangle - (1/2)|1\rangle$.

5.2 Write the state $(3/5)|0\rangle - (4/5)|1\rangle$ as an arrow (a, b) and sketch it on a diagram like Figure 5.2.

5.3 For what value of β, positive, is $(1/3)|0\rangle + \beta|1\rangle$ a valid state?

5.4 Show that $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$. (Substitute the definitions of $|+\rangle$ and $|-\rangle$ and collect terms.) What is $|1\rangle$ in terms of $|+\rangle$ and $|-\rangle$?

5.5 Explain in two or three sentences why the interferometer experiment rules out the idea that a photon after the first beam splitter is "really on one route, we just don't know which".

CHAPTER 6

Looking at a qubit

A qubit can be in any of infinitely many superpositions. But a quantum computer is useless unless we can get answers out of it, and to get an answer we have to look. This chapter is about what happens when we do. It is the strangest part of quantum mechanics, and also, in some ways, the simplest.

6.1 The measurement rule

Suppose a qubit is in the state $\alpha|0\rangle + \beta|1\rangle$, and we measure it to see whether it is 0 or 1. The rule, which we already met in Chapter 4 as the rule of amplitudes, is this.

The outcome is 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. Whichever outcome occurs, the qubit is left in the corresponding basis state: $|0\rangle$ if we saw 0, $|1\rangle$ if we saw 1.

That is all. The normalisation condition $|\alpha|^2 + |\beta|^2 = 1$ from Chapter 5 now has a clear meaning: the two probabilities add up to one, as they must, because the measurement always gives one answer or the other.

For example, a qubit in the state $0.6|0\rangle + 0.8|1\rangle$ gives 0 with probability $0.6^2 = 0.36$ and 1 with probability $0.8^2 = 0.64$. A qubit in the state $|+\rangle$ gives 0 or 1 with probability 1/2 each. So does a qubit in the state $|-\rangle$, because squaring removes the minus sign. A qubit in the state $|0\rangle$ gives 0 with certainty.

The second half of the rule says that measurement changes the state. Before the measurement the qubit was in a superposition; afterwards it is in a definite basis state. This sudden change is often called the *collapse* of the state. It has an immediate consequence: if we measure the same qubit a second time, we get the same answer as the first time, with certainty. The superposition is gone and cannot be recovered, and the information that was stored in the amplitudes α and β, other than the single bit we observed, is lost forever.

Box 6.1: Is the randomness real?

Many people's first reaction to the measurement rule is to suspect that the randomness must come from something we have not yet discovered, some hidden detail of the qubit that secretly decides the outcome in advance, as the precise way a coin leaves your thumb decides heads or tails. Einstein held a view of this kind for most of his life. In 1964 the Northern Irish physicist John Bell showed that this idea, in its most natural form, makes predictions that differ from those of quantum mechanics, and that the difference can be tested. The experiments have been done many times since, with ever greater care, most decisively in 2015, and quantum mechanics wins every time. We will look at Bell's argument in Chapter 12. For now we take the randomness at face value: as far as anyone can tell, it is a basic feature of nature.

6.2 Many runs, not one

Since a single measurement gives only one random bit, how could anyone ever find out what the amplitudes of a state are? The answer is to prepare many qubits in the same state and measure each of them. If we prepare a thousand qubits in the state $0.6|0\rangle + 0.8|1\rangle$ and measure them all, we will see roughly 360 zeros and 640 ones. The exact numbers will vary a little from one batch to the next, just as a thousand coin tosses do not give exactly 500 heads, but with enough repetitions the fractions settle down close to the true probabilities.

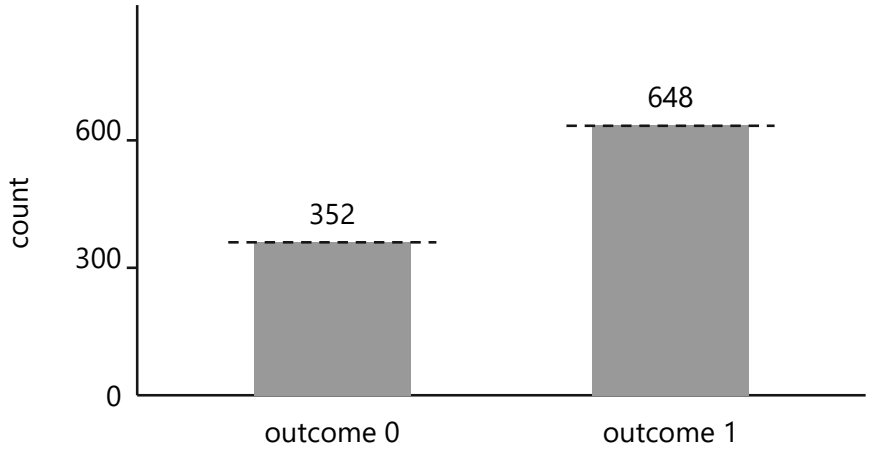


Figure 6.1. Results of measuring 1,000 qubits, each prepared in the state $0.6|0\rangle + 0.8|1\rangle$ (simulated). The dashed lines show the expected counts, 360 and 640. Repeating the experiment many times is the only way to learn the probabilities, and hence the sizes of the amplitudes.

This has a practical side for quantum computers. A typical quantum computation is run not once but hundreds or thousands of times, and the answer is read off from the statistics of the results. For the best quantum algorithms the right answer comes out with high probability, so a few runs are enough; for others, many more are needed. In either case the user of a quantum computer receives not a single answer but a list of outcomes, sometimes called *shots*, from which the answer must be extracted.

Even with many runs, measuring 0 or 1 only tells us $|\alpha|^2$ and $|\beta|^2$. It cannot distinguish $|+\rangle$ from $|-\rangle$, because both give 0 and 1 equally often. To learn about the relative sign we need to measure in a different way.

6.3 Three polarising filters

Here is a simple experiment that you can do yourself with three pairs of cheap polarising sunglasses, or with the polarising filters sold for photography.

A polarising filter lets through light whose electric field vibrates in one particular direction and blocks light vibrating at right angles to it. Ordinary light from a lamp or the sun is a jumble of all directions, and a filter lets through about

half of it. Whatever comes through is polarised along the filter's direction.

Take two filters, one set horizontal and one set vertical, and hold them one behind the other. No light gets through at all. This makes sense: the first filter passes only horizontally polarised light, and the second blocks exactly that.

Now slide a third filter, set diagonally at 45°, *between* the other two. Light reappears. Adding an extra filter, which can only ever block light and never create it, has made the stack more transparent. Only about one eighth of the original light gets through, but before the middle filter was added, none did.

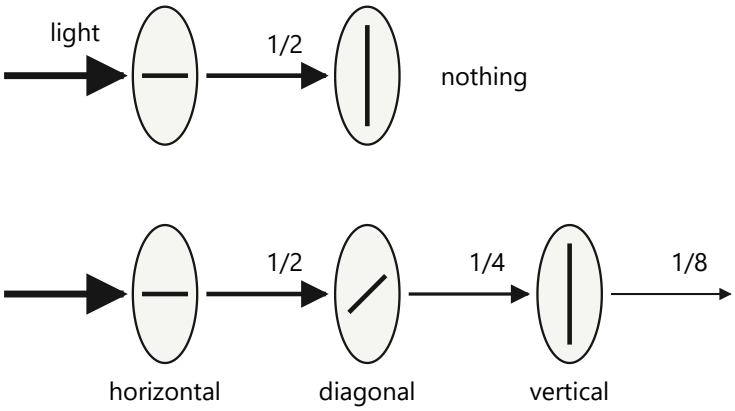


Figure 6.2. The three-filter experiment. Top: crossed horizontal and vertical filters block all light. Bottom: inserting a diagonal filter between them lets one eighth of the original light through. The fractions give the share of the original light at each stage.

With light as a classical wave, the effect has an explanation in terms of how the wave's vibration is projected onto each filter's direction. But the experiment works just as well with single photons, and there it is a perfect illustration of quantum measurement.

Take horizontal polarisation as $|0\rangle$ and vertical polarisation as $|1\rangle$. A photon that has passed the first filter is in the state $|0\rangle$. A vertical filter measures whether the photon is $|0\rangle$ or $|1\rangle$ and lets it pass only if the answer is $|1\rangle$, which for a horizontal photon never happens. A diagonal filter, on the other hand, asks a different question. It asks whether the photon is diagonally polarised, $|+\rangle$, or polarised along the other diagonal, $|-\rangle$, and lets it pass only for $|+\rangle$. Since $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$ (this was Exercise 5.4), a horizontal photon passes the diagonal filter with probability $(1/\sqrt{2})^2 = 1/2$. If it passes, it collapses to the state $|+\rangle$. But $|+\rangle =$

$(|0\rangle + |1\rangle)/\sqrt{2}$, so at the vertical filter it passes with probability $1/2$ again. Overall, $1/2 \times 1/2 = 1/4$ of the photons that came through the first filter make it all the way. The middle filter, by measuring a different question, erased the photon's horizontal polarisation and gave it a fresh chance at the last filter.

6.4 Measuring in another basis

The diagonal filter is an example of a measurement in a different *basis*. So far we have measured qubits in the basis $|0\rangle$, $|1\rangle$, asking "0 or 1?". But any pair of perpendicular unit arrows can serve as the two possible answers to a measurement. The pair $|+\rangle$, $|-\rangle$ is the most common alternative and is called the *Hadamard basis* or *diagonal basis*.

The rule for a measurement in any basis is the same as before, once the state is written in terms of that basis. To measure in the $|+\rangle$, $|-\rangle$ basis, write the state as

$$|\psi\rangle = \gamma|+\rangle + \delta|-\rangle.$$

Then the outcome is "+" with probability $|\gamma|^2$ and "−" with probability $|\delta|^2$, and afterwards the qubit is in $|+\rangle$ or $|-\rangle$ accordingly.

Geometrically, there is a beautiful way to see this. The probability of each outcome is the square of the *shadow* that the state's arrow casts onto the corresponding basis arrow. If the state is nearly parallel to $|+\rangle$, its shadow on $|+\rangle$ is long and its shadow on $|-\rangle$ is short, so "+" is the likely outcome. If the state lies exactly halfway between the two, the shadows are equal and the outcome is a coin toss.

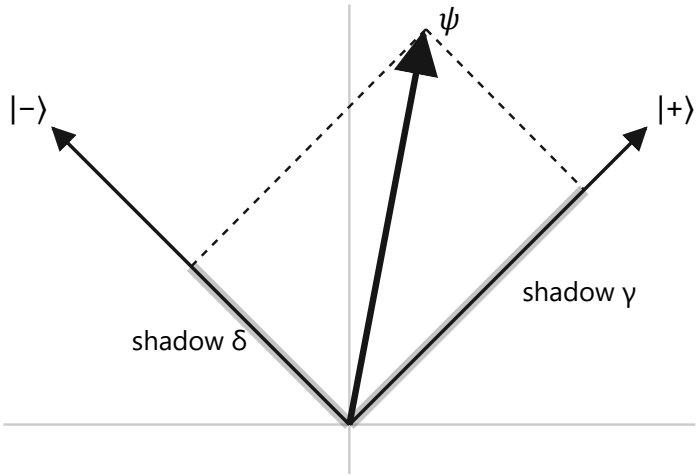


Figure 6.3. Measuring the state ψ in the |+⟩, |−⟩ basis. The shadows (thick grey) that ψ casts onto the two basis arrows are the amplitudes γ and δ, and their squares are the probabilities of the two outcomes. Here "+" is the more likely result.

Measuring |+⟩ in the diagonal basis gives "+" with certainty, and measuring |−⟩ gives "−" with certainty. So the two states that looked identical in the 0/1 basis are perfectly distinguishable in the diagonal basis. Conversely, |0⟩ and |1⟩, which are perfectly distinguishable in the 0/1 basis, give completely random results in the diagonal basis.

This is not an accident of our choice of states. It is a basic feature of quantum mechanics, closely related to Heisenberg's famous *uncertainty principle*. A qubit cannot have a definite answer to both the question "0 or 1?" and the question "+ or −?" at the same time. The more definite one answer is, the more random the other becomes. Quantum cryptography, which we will meet in Chapter 19, is built directly on this fact.

In practice, quantum computers usually measure only in the 0/1 basis. To measure in another basis, they first apply an operation that rotates the desired basis onto |0⟩ and |1⟩, and then measure as usual. We will see how in Chapter 8.

6.5 Measurement is not a passive glance

In everyday life, looking at something does not change it. Reading a thermometer does not change the temperature, at least not noticeably. Our instinct is that measurement simply reveals a value that was already there. In the quantum world this instinct fails. A measurement generally changes the state it measures, and the outcome is not a pre-existing value read off but something that comes into being in the act of measuring.

This has a consequence of great practical importance. Given a single qubit in an unknown state, there is no way to find out what that state is. Measure it in the 0/1 basis and you get one bit and destroy the rest. Measure it in the diagonal basis and you get a different bit and destroy the rest. You cannot measure in both, because the first measurement changes the state before the second can be made. And as we will see in Chapter 12, you cannot get around this by first making copies of the qubit.

For quantum computing this is both an obstacle and a resource. It is an obstacle because the rich information in a quantum state is mostly inaccessible: whatever answer a quantum algorithm computes must be arranged to appear as the outcome of a measurement, ideally with high probability. It is a resource because the same fact makes eavesdropping on quantum communication detectable.

6.6 What counts as a measurement?

You may be wondering what exactly qualifies as a "measurement". Does a measurement need a physicist? A conscious observer? A large piece of laboratory equipment?

None of these. The modern view is that a measurement happens whenever information about the qubit's state leaks into its surroundings in a way that, in principle, could be read out. In the interferometer of Chapter 4, a detector in one route counts as a measurement even if nobody ever looks at the detector's record; the interference disappears regardless. A stray air molecule bumping into an atom can count as a measurement. So can a single photon of heat radiation that carries away information about which state a superconducting circuit was in.

This is why quantum computers are so hard to build. A qubit must be well enough isolated from the rest of the world that its state does not leak out by accident, while still being accessible enough that we can manipulate it and, at the

end, measure it on purpose. The unintended leaking of information is called *decoherence*, and it is the subject of Chapter 20.

Chapter summary. Measuring a qubit in the state $\alpha|0\rangle + \beta|1\rangle$ gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, and leaves the qubit in the corresponding basis state. A single measurement yields one random bit; the probabilities can only be estimated by repeating the experiment many times. Measurements can be made in other bases, such as $|+\rangle$, $|-\rangle$, with probabilities given by squared shadows onto the basis arrows. A state cannot have definite answers in both the 0/1 and the +/− basis. Measurement disturbs the state, so the state of a single unknown qubit cannot be determined. Any leak of information into the surroundings acts as a measurement.

Exercises

6.1 A qubit is in the state $(1/2)|0\rangle + (\sqrt{3}/2)|1\rangle$. What are the probabilities of measuring 0 and 1? If 1 is observed, what is the state afterwards, and what will a second measurement give?

6.2 Write $|1\rangle$ in the $|+\rangle$, $|-\rangle$ basis and find the probabilities of "+" and "−" when $|1\rangle$ is measured in that basis.

6.3 In the three-filter experiment, suppose the middle filter is set at angle θ to the horizontal. A horizontal photon passes it with probability $(\cos\theta)^2$ and then passes the vertical filter with probability $(\sin\theta)^2$. Using the table of Section 3.5, check that the fraction getting through both is largest at $\theta = 45°$.

6.4 You are given a qubit and told that it is either $|0\rangle$ or $|+\rangle$, with no further information. Explain why no measurement can tell you which with certainty. (Hint: what happens if you measure in the 0/1 basis and see 0?)

6.5 You prepare 100 qubits in an unknown state and measure them all in the 0/1 basis, finding 50 zeros and 50 ones. List three different states that are consistent with this result. How could you tell them apart if you had another 100 copies?

Part II: Computing with qubits

How to picture a qubit, how to change it, how qubits combine, and what the quantum world forbids.

CHAPTER 7

A qubit as a point on a globe

The arrow picture of Chapter 5 served us well while the amplitudes were ordinary positive and negative numbers. Once complex amplitudes are allowed, the arrow lives in a space that is impossible to draw on paper, because each of the two amplitudes now needs a little plane of its own. Fortunately there is a way around this. Every possible state of a single qubit can be represented as a point on the surface of an ordinary sphere. This picture, called the *Bloch sphere* after the Swiss-American physicist Felix Bloch, is the most useful mental image in the whole subject, and in this chapter we learn to read it.

7.1 Two numbers are enough

A qubit state $\alpha|0\rangle + \beta|1\rangle$ has two complex amplitudes. Each complex number needs two ordinary numbers to describe it (a length and a direction), so it looks as if four numbers are needed in all. But two of them are not free. The normalisation condition $|\alpha|^2 + |\beta|^2 = 1$ ties the two lengths together, so that once we know how much of the state is $|0\rangle$ we also know how much is $|1\rangle$. And the global phase, as we saw in Section 5.5, has no physical effect, so only the *difference* between the directions of α and β matters.

That leaves exactly two numbers that describe everything physically meaningful about the state of a qubit:

1. how the weight is shared between $|0\rangle$ and $|1\rangle$, and
2. the relative phase between the two amplitudes.

Two numbers are exactly what is needed to specify a place on a globe: a latitude and a longitude. The idea of the Bloch sphere is to use the first number as a latitude and the second as a longitude.

7.2 Reading the sphere

On the Bloch sphere the state $|0\rangle$ sits at the north pole and the state $|1\rangle$ at the south pole. Every other point corresponds to a superposition. Points in the northern hemisphere have more weight on $|0\rangle$, points in the southern hemisphere more weight on $|1\rangle$, and points on the equator are equal superpositions, like $|+\rangle$ and $|-\rangle$, in which 0 and 1 are equally likely.

Moving east or west around a circle of latitude changes the relative phase but not the weights. The four most important points on the equator are

$$|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}, \qquad |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2},$$
$$|+\mathrm{i}\rangle = (|0\rangle + \mathrm{i}|1\rangle)/\sqrt{2}, \qquad |-\mathrm{i}\rangle = (|0\rangle - \mathrm{i}|1\rangle)/\sqrt{2}.$$

They sit a quarter-turn apart, like four towns on the equator at longitudes 0°, 180°, 90° and 270°. Physicists traditionally label the three axes of the sphere x, y and z: the z axis runs from pole to pole, the x axis passes through $|+\rangle$ and $|-\rangle$, and the y axis through $|+\mathrm{i}\rangle$ and $|-\mathrm{i}\rangle$.

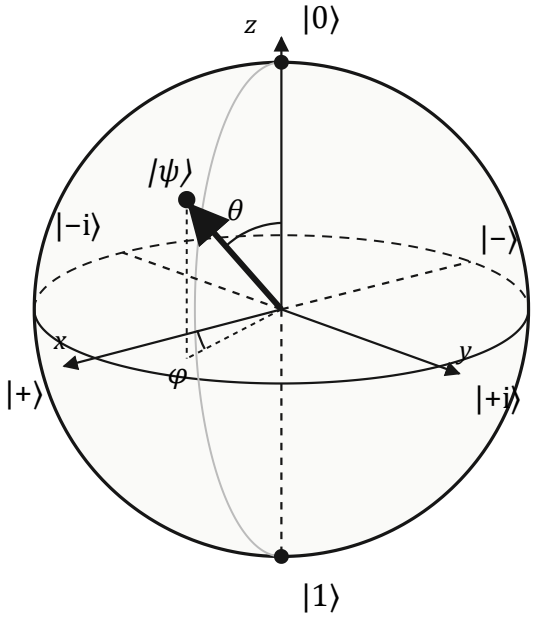


Figure 7.1. The Bloch sphere. The north and south poles are |0⟩ and |1⟩. Equal superpositions lie on the equator, with |+⟩ and |−⟩ on the x axis and |+i⟩ and |−i⟩ on the y axis. A general state |ψ⟩ is a point on the surface, described by a polar angle θ measured down from the north pole and an azimuthal angle φ measured around the equator from the x axis.

A general state is specified by two angles. The *polar angle* θ is measured down from the north pole, from 0° at |0⟩ to 180° at |1⟩. It plays the role of latitude and controls the weights. The *azimuthal angle* φ (phi) is measured around the equator, starting from |+⟩. It plays the role of longitude and controls the relative phase.

7.3 Height is probability

The most useful fact about the Bloch sphere is how simply it encodes the result of a measurement. Imagine the sphere has radius 1, so that the north pole is at height +1 and the south pole at height −1. If the state of the qubit is at height z, then a measurement in the 0/1 basis gives

$$\text{probability of } 0 = (1+z)/2, \qquad \text{probability of } 1 = (1-z)/2.$$

At the north pole, z = 1, and 0 is certain. At the south pole, z = −1, and 1 is certain. Anywhere on the equator, z = 0, and the two outcomes are equally likely. A state three-quarters of the way up from the equator to the north pole, at height 0.5, gives 0 with probability 0.75. The longitude has no effect at all on the result of a 0/1 measurement, which is why |+⟩, |−⟩, |+i⟩ and |−i⟩ all look alike when measured in that basis.

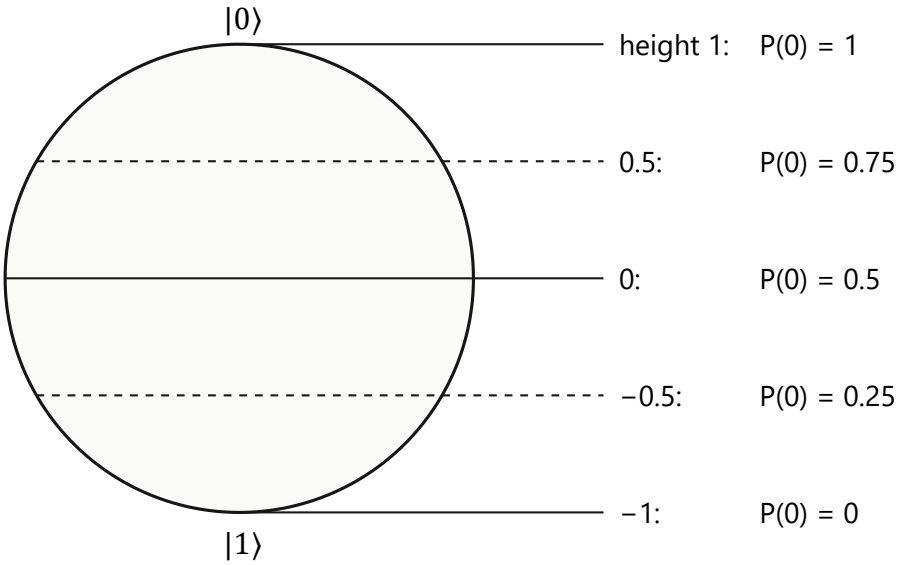


Figure 7.2. The probability of measuring 0 depends only on the height of the state on the Bloch sphere. Every point on a given circle of latitude gives the same probabilities in a 0/1 measurement, whatever its longitude.

The same rule works for measurements in any basis, with "height" measured along a different axis. To measure in the |+⟩, |−⟩ basis, measure how far the state lies along the x axis: a state at position x along that axis gives "+" with probability (1 + x)/2. Measuring a qubit in a given basis is, on the Bloch sphere, asking how far the state leans towards one end of a particular axis.

7.4 Opposites on the sphere

There is one feature of the Bloch sphere that confuses nearly everyone at first. In the arrow picture of Chapter 5, the states |0⟩ and |1⟩ were at right angles to each other, 90° apart. On the Bloch sphere they are at opposite poles, 180° apart.

Likewise $|+\rangle$ and $|-\rangle$ are at right angles as arrows but opposite each other on the sphere. And $|0\rangle$ and $|+\rangle$, which were 45° apart as arrows, are 90° apart on the sphere.

So all angles are doubled when we go from the arrow picture to the sphere. This is not a mistake; it is the price of squeezing complex amplitudes onto a surface we can draw. The rule to remember is that *perfectly distinguishable states are diametrically opposite on the sphere*. Any two antipodal points form a basis, and any axis through the centre defines a possible measurement.

Box 7.1: The formula behind the picture

For readers who want to connect the sphere with the amplitudes, a state with polar angle θ and azimuthal angle φ is

$$|\psi\rangle = \cos(\theta/2)\,|0\rangle + e^{i\varphi}\sin(\theta/2)\,|1\rangle.$$

The halved angles $\theta/2$ are the doubling of Section 7.4 seen from the other side. The factor $e^{i\varphi}$ is a complex number of length 1 pointing at angle φ in the complex plane; it is simply a compact way to write the relative phase. You can check a few cases. With $\theta = 0$ the state is $|0\rangle$. With $\theta = 180°$ it is $e^{i\varphi}|1\rangle$, which is $|1\rangle$ up to a global phase. With $\theta = 90°$ and $\varphi = 0$ it is $(|0\rangle + |1\rangle)/\sqrt{2} = |+\rangle$, and with $\varphi = 180°$ the factor is -1 and the state is $|-\rangle$. The height of the point is $z = \cos\theta$, and since $(\cos(\theta/2))^2 = (1 + \cos\theta)/2$, this formula and the rule of Section 7.3 agree.

7.5 Gates as rotations

The Bloch sphere earns its keep when we start to change the state of a qubit. As we will see in the next chapter, every operation that can be applied to a single qubit, short of measuring it, corresponds to a *rotation* of the sphere. The point representing the state is carried round to a new position, along with every other point on the surface, rigidly, as if someone had picked up the globe and turned it about some axis.

This gives us a wonderfully concrete way to think about single-qubit gates. A gate that flips $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$ must swap the poles, so it might be a half-turn about the x axis. A gate that changes only the relative phase leaves the poles alone and spins the equator, so it is a rotation about the z axis. A gate that takes $|0\rangle$

to |+⟩ moves the north pole down to the equator. Questions about what a sequence of gates does can often be answered by imagining a globe and turning it in your hands.

Rotation also explains why quantum gates are reversible, a property we mentioned at the end of Chapter 2. Any rotation can be undone by rotating back the same amount about the same axis. Measurement, by contrast, is not a rotation. It snaps the state to one of two poles, and there is no way to undo it.

7.6 Inside the sphere

The points on the surface of the Bloch sphere represent all the states a single qubit can be in when we know exactly how it was prepared. Such states are called *pure states*. Physicists also use points *inside* the sphere, to describe qubits whose state is uncertain in the ordinary, classical sense: for instance, a qubit that is |0⟩ with probability 1/2 and |1⟩ with probability 1/2 because someone tossed a coin to decide how to prepare it. Such a qubit is said to be in a *mixed state*, and it is represented by the centre of the sphere.

The distinction between the centre of the sphere and the point |+⟩ on its equator is exactly the distinction between the coin under the cup and genuine superposition that we made in Section 5.4. Both give 0 or 1 at random in a 0/1 measurement. But a measurement along the x axis gives "+" with certainty for |+⟩ and a coin toss for the centre. In Chapter 20 we will see that noise tends to drag the states of real qubits from the surface towards the centre, destroying their quantumness, and the length of the arrow from the centre is a convenient measure of how much damage has been done.

The Bloch sphere has one important limitation: it works only for a single qubit. There is no similarly simple picture for two or more qubits, and for good reason. As we will see in Chapter 9, the states of several qubits have a richness that no picture in ordinary three-dimensional space can capture.

Chapter summary. Every pure state of a qubit corresponds to a point on the surface of the Bloch sphere. The poles are $|0\rangle$ and $|1\rangle$, equal superpositions lie on the equator, latitude encodes the weights and longitude the relative phase. The probability of measuring 0 is $(1 + z)/2$, where z is the height of the point. Perfectly distinguishable states are diametrically opposite, so angles on the sphere are twice the angles between state arrows. Single-qubit gates are rotations of the sphere, which is why they are reversible. Points inside the sphere represent mixed states, which carry ordinary classical uncertainty.

Exercises

7.1 Where on the Bloch sphere is the state $0.6|0\rangle + 0.8|1\rangle$? Find its height z using the rule of Section 7.3 and the probability you computed for it in Section 6.1.

7.2 Which point on the Bloch sphere is diametrically opposite $|+i\rangle$? Write down the corresponding state.

7.3 A state lies on the equator, a quarter of the way from $|+\rangle$ towards $|+i\rangle$. What is the probability of measuring 0? Of measuring "+" in the diagonal basis? (Use the x-axis version of the rule in Section 7.3; the x coordinate of a point on the equator at azimuth φ is $\cos\varphi$.)

7.4 Describe a rotation of the sphere that takes $|0\rangle$ to $|+\rangle$. Is there more than one? What does your rotation do to $|1\rangle$?

7.5 Explain the difference between a qubit at the centre of the Bloch sphere and a qubit at $|+\rangle$, in terms of a measurement you could perform.

CHAPTER 8

Changing a qubit: quantum gates

A classical computer computes by passing bits through logic gates. A quantum computer computes by passing qubits through *quantum gates*. In this chapter we meet the gates that act on a single qubit. There are only a few that matter in practice, they are easy to describe, and with the Bloch sphere in hand they are easy to picture. We will also learn to read the diagrams, called *quantum circuits*, that are used everywhere in the subject to write down quantum algorithms.

8.1 What a gate is allowed to do

A quantum gate takes a qubit in some state and leaves it in another state. Physically, it is usually a carefully timed pulse of laser light, microwaves or magnetic field that nudges the qubit's state in a controlled way. Mathematically, it is a rule that turns one state arrow into another.

Not every rule is allowed. Whatever a gate does, the state that comes out must still be a valid state, with amplitudes whose squares add up to one. In the arrow picture that means a gate must never stretch or shrink an arrow; it can only turn it. And since gates act on superpositions by acting on each part separately (a principle physicists call *linearity*), a gate must turn all arrows in a consistent way, preserving the angles between them as well as their lengths. Operations of this kind are rotations and reflections. In the language of matrices they are called *unitary* matrices, and every quantum gate, on any number of qubits, is described by one.

On the Bloch sphere, every single-qubit gate is simply a rotation of the sphere about some axis. Rotations can always be undone, so every quantum gate is reversible. That is the sense in which, as we said in Chapter 2, quantum mechanics never throws information away.

8.2 The X gate

The simplest gate is the quantum version of NOT. It is called the *X gate* (or sometimes the bit-flip gate), and it swaps $|0\rangle$ and $|1\rangle$:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle.$$

What does it do to a superposition? Because gates act on each part of a superposition separately, we can apply it term by term:

$$X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle = \beta|0\rangle + \alpha|1\rangle.$$

The two amplitudes swap places. This is exactly the swap matrix we met in Section 3.7:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

On the Bloch sphere, X is a half-turn (a rotation by 180°) about the x axis. The north and south poles change places. The points $|+\rangle$ and $|-\rangle$, which lie on the axis of rotation, stay where they are. You can check this from the formula: swapping the amplitudes of $(|0\rangle + |1\rangle)/\sqrt{2}$ gives the same state back, and swapping the amplitudes of $(|0\rangle - |1\rangle)/\sqrt{2}$ gives $(-|0\rangle + |1\rangle)/\sqrt{2}$, which is $-|-\rangle$, the same as $|-\rangle$ up to an unobservable global phase.

In a circuit diagram a single-qubit gate is drawn as a box on the qubit's line:

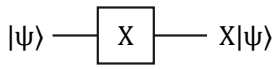


Figure 8.1. The simplest quantum circuit: one qubit, one X gate. Time runs from left to right.

8.3 The Z gate

The *Z gate* leaves $|0\rangle$ alone and multiplies $|1\rangle$ by -1:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

On a superposition it flips the sign of the $|1\rangle$ amplitude:

$$Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle.$$

At first sight Z seems to do nothing at all, since it does not change the probability of measuring 0 or 1. But it turns $|+\rangle$ into $|-\rangle$ and $|-\rangle$ into $|+\rangle$, which is a dramatic change, as the three-filter experiment showed. Z is called the *phase-flip* gate. On the Bloch sphere it is a half-turn about the z axis: the poles stay put, and the equator is spun round by 180°, carrying $|+\rangle$ to $|-\rangle$ and $|+i\rangle$ to $|-i\rangle$.

There is a nice symmetry between X and Z. The X gate flips 0 and 1 and leaves + and − unchanged. The Z gate flips + and − and leaves 0 and 1 unchanged. Each one is a "NOT gate" for a different basis. This symmetry will be important when we come to error correction in Chapter 21, because quantum errors come in both kinds: bit flips and phase flips.

There is also a third gate of this family, the *Y gate*, which is a half-turn about the y axis. It flips both the bit and the phase; up to a global phase it equals X followed by Z. Together, X, Y and Z are called the *Pauli gates*, after the Austrian physicist Wolfgang Pauli, who introduced the matrices in 1927 to describe the spin of the electron.

8.4 The Hadamard gate

The most important single-qubit gate of all is the *Hadamard gate*, written H. Its matrix is the one we met at the end of Section 3.7:

$$H = (1/\sqrt{2}) \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

and its effect on the basis states is

$$H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2} = |+\rangle, \qquad H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle.$$

In words, the Hadamard gate turns the definite states 0 and 1 into equal superpositions. It is the gate we use to create superposition. And because H applied twice does nothing (we checked this with arrows in Chapter 3), it also works the other way round:

$$H|+\rangle = |0\rangle, \qquad H|-\rangle = |1\rangle.$$

This is worth pausing over. A qubit in the state $|+\rangle$ and a qubit in the state $|-\rangle$ both give random results if measured in the 0/1 basis. Apply a Hadamard first, and the first gives 0 with certainty while the second gives 1 with certainty. This is interference at work. Let us see it explicitly by applying H to $|+\rangle$ term by term:

$$\begin{aligned} H|+\rangle &= (1/\sqrt{2})(H|0\rangle + H|1\rangle) \\ &= (1/\sqrt{2})((|0\rangle + |1\rangle)/\sqrt{2} + (|0\rangle - |1\rangle)/\sqrt{2}) \\ &= (1/2)(|0\rangle + |1\rangle + |0\rangle - |1\rangle) = |0\rangle. \end{aligned}$$

The two $|0\rangle$ contributions add up. The two $|1\rangle$ contributions, one positive and one negative, cancel. Compare this with the interferometer of Chapter 4: there too the photon reached detector A with certainty because the contributions to B cancelled. That is no coincidence. With the lower route as $|0\rangle$ and the upper route as $|1\rangle$, each of the beam splitters in the interferometer acts on the photon exactly as a Hadamard gate does. The whole experiment is the circuit of Figure 8.2.

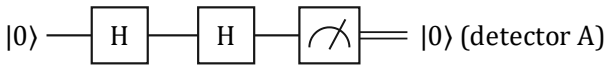


Figure 8.2. The Mach–Zehnder interferometer as a quantum circuit. Each beam splitter is a Hadamard gate. The first creates the superposition $|+\rangle$; the second turns it back into $|0\rangle$, so the measurement always gives 0. The double line after the measurement symbol carries a classical bit, the outcome.

The measurement symbol at the end of the circuit is a small dial, a reminder of an old-fashioned meter. After it the line is drawn double, to show that what comes out is no longer a qubit but an ordinary classical bit.

Blocking one route of the interferometer corresponds to measuring the qubit between the two Hadamard gates. The measurement collapses $|+\rangle$ to $|0\rangle$ or $|1\rangle$ at random, and the second Hadamard then turns either of these into an equal superposition, so the final result is random. In circuit language, the presence or absence of a measurement in the middle is the difference between certainty and a coin toss.

On the Bloch sphere the Hadamard gate is a half-turn about an axis tilted halfway between x and z. This rotation swaps the north pole with $|+\rangle$ and the south pole with $|-\rangle$, and swaps the x axis with the z axis.

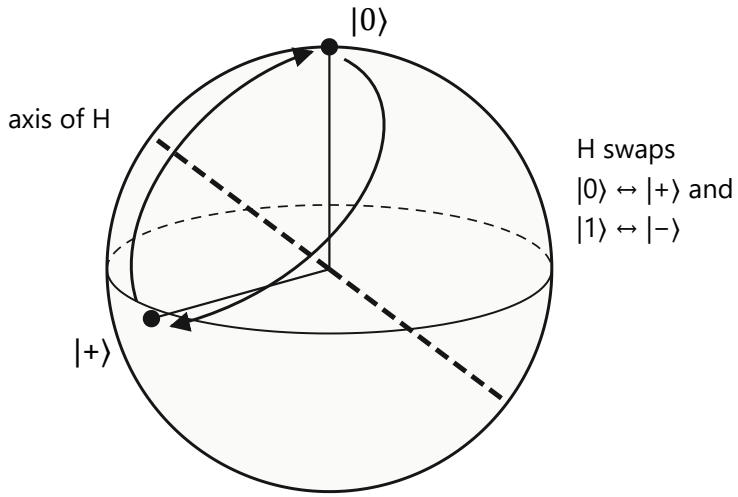


Figure 8.3. The Hadamard gate on the Bloch sphere: a half-turn about the dashed axis, which lies halfway between the z axis (through |0⟩) and the x axis (through |+⟩). The north pole and |+⟩ exchange places.

8.5 Phase gates and general rotations

The Z gate multiplies the |1⟩ amplitude by −1, which is a half-turn about the z axis. There is nothing special about a half-turn; we could equally well turn the equator by a smaller amount. The gate that multiplies the |1⟩ amplitude by i, a quarter-turn about the z axis, is called the *S gate*. It takes |+⟩ to |+i⟩. The gate that turns the equator by an eighth of a turn, 45°, is called the *T gate*. These gates change the relative phase without changing the probabilities of 0 and 1, and they are collectively called *phase gates*. Their matrices are

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

(The number $e^{i\pi/4}$ is just a complex number of length 1 pointing at 45°, as in Box 7.1.)

More generally, we can rotate the Bloch sphere by any angle about any axis. Such gates are written $R_x(\theta)$, $R_y(\theta)$ and $R_z(\theta)$ for rotations by angle θ about the x, y and z axes. A useful fact, which you can probably convince yourself of by turning a ball in your hands, is that any rotation whatsoever can be produced by combining

rotations about just two fixed axes. So any single-qubit gate can be built from a sequence of R_z and R_y rotations. In practice, many quantum computers offer a small menu of native rotations and build everything else out of them.

8.6 Reading circuit diagrams

We have now drawn a few circuit diagrams, and it is worth stating the conventions clearly, because they differ from classical circuit diagrams in one important way.

Each horizontal line represents one qubit. The line is *not* a wire along which something travels. It is a timeline: the qubit stays where it is (an ion in a trap, a superconducting loop on a chip) and the line represents its history from left to right. Gates are drawn as boxes on the line at the moment they are applied. A sequence of gates is read from left to right, which is the order in which they happen.

There is a trap here for anyone reading the mathematics alongside the diagram. In formulas, a gate acts on whatever is written to its right, so the circuit "first H, then Z" is written $ZH|\psi\rangle$, with the first gate nearest the state. The order on paper is reversed. The circuit of Figure 8.4, for instance, is written $HZH|\psi\rangle$.

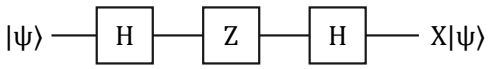


Figure 8.4. Three gates in sequence. A Hadamard, then a Z, then another Hadamard has exactly the same effect as a single X gate. In symbols, HZH = X.

The identity HZH = X can be checked by following the basis states. H turns $|0\rangle$ into $|+\rangle$; Z turns $|+\rangle$ into $|-\rangle$; H turns $|-\rangle$ into $|1\rangle$. So $|0\rangle$ ends up as $|1\rangle$. Similarly $|1\rangle$ becomes $|-\rangle$, then $|+\rangle$, then $|0\rangle$. That is exactly what X does. In Bloch-sphere terms, the Hadamard swaps the x and z axes, so a half-turn about z, sandwiched between two Hadamards, becomes a half-turn about x. Identities like this are used constantly to simplify circuits and to translate between the gates one wants and the gates a particular machine can actually perform.

8.7 Measuring in any basis

At the end of Chapter 6 we mentioned that quantum computers usually measure only in the 0/1 basis, and that other measurements are made by first rotating the qubit. Now we can see how. To measure in the $|+\rangle$, $|-\rangle$ basis, apply a Hadamard and then measure as usual. The Hadamard takes $|+\rangle$ to $|0\rangle$ and $|-\rangle$ to $|1\rangle$, so an outcome of 0 means "+" and an outcome of 1 means "−". To measure along any other axis of the Bloch sphere, first rotate that axis onto the z axis and then measure.

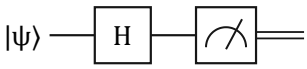


Figure 8.5. A measurement in the $|+\rangle$, $|-\rangle$ basis, built from a Hadamard gate followed by an ordinary 0/1 measurement. Outcome 0 means "+" and outcome 1 means "−".

Box 8.1: Undoing a gate

Because every gate is a rotation, every gate has an inverse that undoes it: the same rotation, turned backwards. For the Pauli gates and the Hadamard, the inverse is the gate itself, since a half-turn followed by another half-turn about the same axis is a full turn. For S the inverse is $S^\dagger$ ("S dagger"), the quarter-turn the other way, which multiplies $|1\rangle$ by $-i$. In matrix language the inverse of a unitary matrix is found by flipping it about its diagonal and replacing every entry by its complex conjugate (changing i to −i), an operation written with the dagger symbol †. This easy way of reversing any gate is used constantly in quantum algorithms, which frequently compute something, use it, and then *uncompute* it by running the same gates backwards in reverse order.

Chapter summary. Quantum gates change the state of qubits without measuring them. They must preserve the length of the state arrow, so they are rotations (unitary matrices) and are always reversible. The Pauli gates X, Y and Z are half-turns of the Bloch sphere; X flips 0 and 1, and Z flips + and −. The Hadamard gate H turns $|0\rangle$ and $|1\rangle$ into $|+\rangle$ and $|-\rangle$ and back again, and two Hadamards in a row reproduce the Mach–Zehnder interferometer. Phase gates such as S and T rotate the equator. In circuit diagrams each line is a qubit's history, read left to right, while formulas are written right to left. Measurements in other bases are made by rotating first and then measuring in the 0/1 basis.

Exercises

8.1 Apply X to the state $0.6|0\rangle + 0.8|1\rangle$. Then apply Z to the result. What are the final amplitudes?

8.2 Use the term-by-term method of Section 8.4 to show that $H|-\rangle = |1\rangle$. Identify which contributions cancel.

8.3 Show that $HXH = Z$ by following $|0\rangle$ and $|1\rangle$ through the three gates.

8.4 What is the effect of applying S twice? Of applying T four times? (Think about how far each turns the equator.)

8.5 A qubit starts in $|0\rangle$. Draw a circuit that ends with the qubit in the state $|-\rangle$. Then draw one that ends in $|+i\rangle$.

8.6 In Figure 8.2, suppose a Z gate is inserted between the two Hadamards. What does the measurement give now? Relate your answer to Exercise 4.3.

CHAPTER 9

Many qubits

One qubit, however interesting, cannot compute very much. Real quantum computers have tens, hundreds or (one day) millions of qubits, and the power of the machine lies in how they behave together. In this chapter we see how the state of several qubits is described, and we meet the explosive growth that makes quantum computers both promising and hard to imitate.

9.1 Two qubits

Two classical bits can be in one of four configurations: 00, 01, 10 or 11. Two qubits have the corresponding four basis states,

$$|00\rangle, \quad |01\rangle, \quad |10\rangle, \quad |11\rangle,$$

where $|01\rangle$, for example, means that the first qubit is 0 and the second is 1. And just as one qubit can be in a superposition of its two basis states, two qubits can be in a superposition of all four:

$$|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle. \tag{9.1}$$

There are now four amplitudes. The rules are exactly the ones we already know. If both qubits are measured, the outcome 00 occurs with probability $|a|^2$, the outcome 01 with probability $|b|^2$, and so on, and so the four probabilities must add up to one:

$$|a|^2 + |b|^2 + |c|^2 + |d|^2 = 1.$$

In the arrow picture, the state of two qubits is a unit arrow in a space with four perpendicular directions instead of two. Nobody can visualise such a space, but the rules for handling it are the same as in the plane: arrows have components, lengths

come from adding squares, and gates are rotations that preserve length. The mathematics does not care how many dimensions there are.

9.2 Putting qubits side by side

Suppose we have two separate qubits, the first in the state $\alpha|0\rangle + \beta|1\rangle$ and the second in the state $\gamma|0\rangle + \delta|1\rangle$. What is the state of the pair? The answer is found by multiplying out, exactly as with ordinary brackets in algebra:

$$(\alpha|0\rangle + \beta|1\rangle)(\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle.$$

Each amplitude of the pair is the product of one amplitude from each qubit. This is the same rule as for independent probabilities in Section 3.6, where the chance of two things both happening is the product of their chances. Physicists call this way of combining systems the *tensor product*, written with the symbol $\otimes$, but in practice it means nothing more than multiplying out the brackets.

For example, if the first qubit is in $|+\rangle$ and the second is in $|0\rangle$, the pair is in the state

$$|+\rangle|0\rangle = ((|0\rangle + |1\rangle)/\sqrt{2})\,|0\rangle = (|00\rangle + |10\rangle)/\sqrt{2}.$$

And if both qubits are in $|+\rangle$, the pair is in

$$|+\rangle|+\rangle = (|00\rangle + |01\rangle + |10\rangle + |11\rangle)/2,$$

an equal superposition of all four basis states, each with amplitude $1/2$ and hence probability $1/4$.

States that can be written as a product of separate single-qubit states in this way are called *product states*. In a product state each qubit has a state of its own, and the pair is nothing more than the sum of its parts. But look again at equation (9.1). A general two-qubit state has four amplitudes, which can be chosen freely (subject only to normalisation), while a product state is built from just two pairs of amplitudes. There are many more two-qubit states than product states. The states that cannot be written as products are called *entangled*, and they are the subject of the next chapter.

9.3 The explosion

Now consider three qubits. They have eight basis states, |000⟩ through |111⟩, and a general state of three qubits needs eight amplitudes. Four qubits need sixteen. In general, n qubits have 2^n basis states, one for each pattern of n bits, and a general state of n qubits is a superposition with 2^n amplitudes.

This is the doubling we met in Chapter 2. There it described how many *different* patterns n bits could hold, only one of which is present at any time. Here it describes how many amplitudes are needed *simultaneously* to describe the state of n qubits. A classical register of 300 bits holds one 300-bit number. A quantum register of 300 qubits is described by 2^{300} amplitudes, roughly 10^{90} of them, which is vastly more than the number of atoms in the observable universe (about 10^{80}).

qubits	amplitudes	memory to store them on a classical computer
1	2	32 bytes
10	1,024	16 kilobytes
20	about 1 million	16 megabytes
30	about 1 billion	16 gigabytes
40	about 1 trillion	16 terabytes
50	about 10^{15}	16 petabytes
60	about 10^{18}	16 exabytes

The table assumes 16 bytes per amplitude, which is typical for a complex number stored to reasonable precision. Around 30 qubits, the state fits in the memory of a good laptop. Around 45 to 50 qubits, it fills the memory of the largest supercomputers on Earth. Every additional qubit doubles the requirement. Somewhere beyond 60 qubits, a straightforward simulation of a general quantum state becomes impossible with any computer that could ever be built.

There are cleverer ways of simulating quantum computers that exploit special structure in the states that arise, and they push these limits somewhat. But for general quantum computations nobody knows a way to avoid the exponential cost,

and there are good reasons to believe none exists. This is the heart of Feynman's observation in Chapter 1: nature keeps track of all those amplitudes effortlessly, while our computers cannot.

9.4 All numbers at once?

Let us see what happens when we apply a Hadamard gate to each of several qubits, all starting in $|0\rangle$. Each Hadamard turns its $|0\rangle$ into $|+\rangle$, and multiplying out the product gives an equal superposition of every basis state. With three qubits,

$$|+\rangle|+\rangle|+\rangle = (|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)/\sqrt{8}.$$

Reading the bit patterns as binary numbers (Box 2.1), this is an equal superposition of the numbers 0 to 7. With n qubits and n Hadamard gates we get an equal superposition of all 2^n numbers from 0 to $2^n - 1$, each with amplitude $1/\sqrt{(2^n)}$. It costs only n gates to prepare a state involving 2^n numbers.

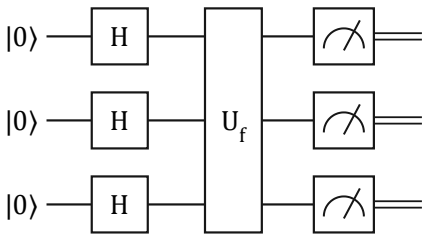


Figure 9.1. The typical opening of a quantum algorithm. Hadamard gates (drawn here in a single column, since they act at the same time on different qubits) create an equal superposition of all eight three-bit numbers. A larger operation, here a box labelled U_f, then acts on the whole superposition before the qubits are measured.

Suppose we then apply some operation that computes a function f for each number. Because quantum gates act on every part of a superposition separately, a single application of the operation acts on all 2^n numbers at once, producing a state in which every number x is paired with its value f(x). This effect is known as *quantum parallelism*, and it is the grain of truth in the popular slogan that quantum computers "try all answers at once".

Here is the catch, and it is a serious one. When we measure the result, we do not see all 2^n values of f. We see one of them, chosen at random, and the rest of the superposition is destroyed. A quantum computer used in this naive way is no better than a classical computer that picks a random input and evaluates f on it. The slogan is misleading precisely because it leaves out the measurement.

The real art of quantum algorithms, which we will explore in Part III, lies in what happens between the parallel evaluation and the measurement. By applying further gates that make the different parts of the superposition interfere, a clever algorithm can arrange for the amplitudes of uninteresting outcomes to cancel and the amplitudes of useful outcomes to reinforce, so that the final measurement reveals some *global property* of the function, such as a hidden pattern or a period, rather than a single value. Only a few kinds of problem have structure that allows this, which is why quantum computers are specialised instruments rather than universal speed-up machines.

9.5 Measuring some of the qubits

It is often useful to measure only some of the qubits in a register and leave the rest alone. The rule follows naturally from what we already know, and an example shows it best.

Take the two-qubit state

$$|\psi\rangle = (1/2)|00\rangle + (1/2)|01\rangle + (1/\sqrt{2})|11\rangle,$$

and measure only the first qubit. The probability of seeing 0 is the total probability of all basis states whose first bit is 0, namely $(1/2)^2 + (1/2)^2 = 1/2$. The probability of seeing 1 is $(1/\sqrt{2})^2 = 1/2$.

If we see 0, the state collapses to the part of the superposition consistent with that outcome, namely $(1/2)|00\rangle + (1/2)|01\rangle$, which we then rescale so that its probabilities add up to one again. The result is $(|00\rangle + |01\rangle)/\sqrt{2}$, which is $|0\rangle|+\rangle$: the first qubit is now definitely 0, and the second is in the state $|+\rangle$. If instead we see 1, only the term $|11\rangle$ survives, and after rescaling the state is $|11\rangle$, so the second qubit is definitely 1.

So the rule is: keep the terms that agree with what was seen, throw away the rest, and rescale. Notice that measuring the first qubit changed what we know about the second. Before the measurement, the second qubit had a three-quarters

chance of being 1 (from the terms |01⟩ and |11⟩). After seeing 0 on the first qubit, it has a one-half chance; after seeing 1, it is certainly 1. This kind of correlation between qubits will become much more striking in the next chapter.

Box 9.1: Labelling conventions

When writing states like |01⟩, we need to agree which qubit is which. In this book the first symbol always refers to the top line of a circuit diagram (qubit 0) and the second to the next line down. Some textbooks and software packages use the opposite order, listing the bottom qubit first, so that the bit string reads like a binary number with the most significant bit at the bottom. Neither is right or wrong, but mixing them up is one of the most common sources of confusion for people starting out with quantum programming. Always check the convention.

9.6 How much does a register hold?

We have reached a striking situation. Describing the state of n qubits requires 2^n numbers, an amount that dwarfs anything a classical computer could store for n of a few hundred. And yet, by Holevo's theorem (Section 5.6), measuring those n qubits yields at most n classical bits. The vast majority of the information needed to describe the state is locked away, inaccessible to any direct measurement.

Is that information "really there"? Physicists disagree about how best to put it. What is certain is that the amplitudes are not idle: every one of them takes part in interference, and the outcome of a quantum computation depends on all of them. Quantum algorithms work by using this enormous hidden space as a kind of scratchpad, letting amplitudes combine and cancel there, and then extracting a small, carefully chosen piece of information at the end.

Chapter summary. Two qubits have four basis states and a general two-qubit state has four amplitudes; n qubits have 2^n. States of separate qubits combine by multiplying out the brackets (the tensor product), but most multi-qubit states are not products of single-qubit states. Simulating n qubits classically requires memory that doubles with every qubit, which becomes impossible beyond about 50 to 60 qubits. Applying a Hadamard to each of n qubits produces an equal superposition of all 2^n numbers, and a function can be evaluated on all of them at once, but measuring gives only one random result. Useful quantum algorithms must use interference to extract global properties. Measuring some qubits keeps only the terms consistent with the result, and can change what we know about the others.

Exercises

9.1 Multiply out $(|0\rangle + |1\rangle)/\sqrt{2}$ times $(|0\rangle - |1\rangle)/\sqrt{2}$. What is the probability that both qubits are measured as 1?

9.2 Write out the state of four qubits after a Hadamard gate is applied to each one starting from $|0000\rangle$. How many terms are there, and what is each amplitude?

9.3 Using the table in Section 9.3, roughly how many qubits would a state need in order for its amplitudes to fill 16 zettabytes (1,000 exabytes)?

9.4 For the state in Section 9.5, suppose we measure the *second* qubit instead. What are the probabilities of 0 and 1, and what is the state afterwards in each case?

9.5 Is the state $(|00\rangle + |01\rangle + |10\rangle - |11\rangle)/2$ a product state? (Hint: try to write it as $(\alpha|0\rangle + \beta|1\rangle)(\gamma|0\rangle + \delta|1\rangle)$. Compare with Exercise 9.1 and with the state of Section 9.2.)

CHAPTER 10

Entanglement

In 1935 Erwin Schrödinger called entanglement "*the* characteristic trait of quantum mechanics", the feature that marks its break with all earlier physics. In the same year Albert Einstein, together with Boris Podolsky and Nathan Rosen, used it to argue that quantum mechanics must be incomplete. Einstein's objection eventually became famous under his own phrase, *spooky action at a distance*. Nearly a century later, entanglement is routinely produced in laboratories, sent between satellites and ground stations, and used as a working resource in quantum computers. This chapter explains what it is, what it does, and what it does not do.

10.1 A state that cannot be split

At the end of Section 9.2 we found that most two-qubit states cannot be written as a product of two single-qubit states. Here is the most famous example:

$$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}. \tag{10.1}$$

(The symbol Φ is the capital Greek letter phi, and the name $|\Phi^+\rangle$ is conventional.) In this state, the two qubits are found either both 0 or both 1, each with probability 1/2. The outcomes 01 and 10 never occur.

Why is this not a product state? Suppose it were, so that $|\Phi^+\rangle = (\alpha|0\rangle + \beta|1\rangle)(\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$. The state has no $|01\rangle$ term, so $\alpha\delta = 0$, which means either α or δ is zero. But if α were zero, the $|00\rangle$ term $\alpha\gamma$ would vanish too, and it does not; and if δ were zero, the $|11\rangle$ term $\beta\delta$ would vanish, and it does not. Either way we reach a contradiction. There is no way to assign a state to each qubit separately that reproduces $|\Phi^+\rangle$.

This is what *entanglement* means. An entangled pair of qubits has a definite joint state, but neither qubit has a state of its own. All of the information is in the relationship between them. Schrödinger made the same point: knowing everything there is to know about the pair does not mean knowing everything about each member.

10.2 Perfect correlation

Imagine that two physicists, traditionally named Alice and Bob, each hold one qubit of the pair $|\Phi^+\rangle$, and that they have travelled to opposite sides of the world. Alice measures her qubit in the 0/1 basis. She sees 0 or 1 with probability 1/2 each; her result is completely random. But by the rule of Section 9.5, if she sees 0 the pair collapses to $|00\rangle$, and Bob's qubit is now definitely 0. If she sees 1, Bob's qubit is definitely 1. Whenever they later compare notes, they find that their results agree, every time.

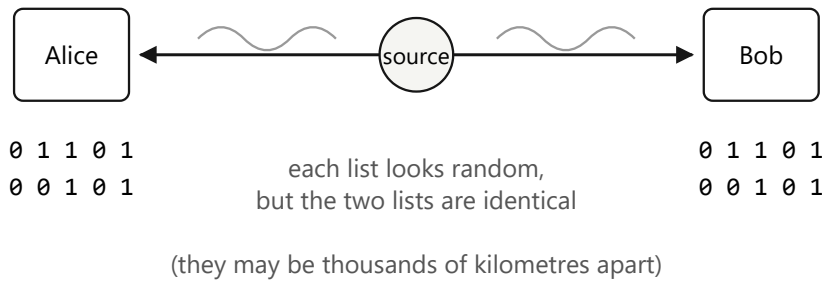


Figure 10.1. A source sends one qubit of each entangled pair $|\Phi^+\rangle$ to Alice and the other to Bob. Each measures in the 0/1 basis. Separately, their results look like fair coin tosses; together, they match perfectly.

On its own, perfect correlation is not mysterious. Suppose someone takes a pair of gloves, puts one into each of two boxes at random, and posts one box to Alice and the other to Bob. When Alice opens her box and finds a left glove, she knows instantly that Bob's box contains a right glove, however far away it is. Nothing travelled between the boxes; the correlation was fixed when the gloves were

packed. The physicist John Bell liked to tell a similar story about his colleague Reinhold Bertlmann, who always wore socks of different colours: if you see that one of his socks is pink, you know at once that the other is not.

Is entanglement just a quantum version of the gloves? That was more or less Einstein's view. He believed that each qubit must carry, from the moment the pair was created, some hidden instructions that decide what it will show when measured, just as each glove "knows" whether it is left or right. Quantum mechanics, which does not mention any such instructions, would then be an incomplete description of reality.

10.3 Correlation in every direction

The glove explanation runs into trouble as soon as Alice and Bob measure in a different basis. Let us rewrite $|\Phi^+\rangle$ using the diagonal basis states. Since $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$ and $|1\rangle = (|+\rangle - |-\rangle)/\sqrt{2}$ (Exercise 5.4), we can substitute and multiply out. After the dust settles (Exercise 10.2 asks you to check this), the result is

$$|\Phi^+\rangle = (|{+}{+}\rangle + |{-}{-}\rangle)/\sqrt{2}.$$

The state looks exactly the same in the diagonal basis as in the 0/1 basis. So if Alice and Bob both measure in the diagonal basis, they again get perfectly correlated results: both "+" or both "−", at random.

Now, gloves have one property (left or right) and can be correlated in that. But the entangled pair is correlated in the 0/1 basis *and* in the diagonal basis, and in fact along any axis of the Bloch sphere, provided both measure along the same axis. Could the hidden instructions in each qubit simply specify an answer for every possible measurement in advance? At first it seems they could; each qubit would carry a little table: "if asked 0-or-1, say 0; if asked plus-or-minus, say −; ..." and the two qubits of a pair would carry matching tables.

It took almost thirty years for anyone to see what is wrong with this idea. In 1964 John Bell found that if Alice and Bob measure along *different* axes, chosen at random, then any scheme of pre-arranged tables predicts a certain limit on how strongly their results can be correlated. Quantum mechanics predicts that entangled pairs exceed that limit. The difference can be tested in the laboratory, and the experiments come down firmly on the side of quantum mechanics. No pre-

arranged instructions, of any kind, can explain the correlations of entangled particles. We will work through Bell's argument, which needs nothing more than counting, in Chapter 12.

10.4 No faster-than-light messages

If the correlation is not fixed in advance, does Alice's measurement instantly *cause* Bob's result, across any distance? This is the "spooky action" that troubled Einstein, and it might seem to allow messages to be sent faster than light, in violation of relativity. It does not, and the reason is worth understanding carefully.

Look at the situation from Bob's side. Before Alice does anything, if Bob measures his qubit in the 0/1 basis, he sees 0 or 1 at random, half and half. Now suppose Alice measures first. With probability 1/2 she sees 0 and Bob's qubit becomes $|0\rangle$; with probability 1/2 she sees 1 and Bob's qubit becomes $|1\rangle$. Bob does not know which. From his point of view, his qubit is $|0\rangle$ or $|1\rangle$ with probability 1/2 each, and his measurement gives 0 or 1 at random, half and half. Exactly as before.

The same is true whatever Alice does: whether she measures in the 0/1 basis, the diagonal basis or not at all, Bob's own statistics are identical. Nothing Alice chooses to do on her side changes anything Bob can observe on his side alone. The correlation only shows up when the two of them bring their results together and compare, and that comparison has to travel by ordinary means, at or below the speed of light. This principle is called *no-signalling*, and it holds for every entangled state and every measurement. Entanglement is strange, but it is not a telephone.

Box 10.1: Entanglement in space

Entanglement does not weaken with distance. In 2017 the Chinese satellite Micius, orbiting at an altitude of about 500 kilometres, distributed entangled photon pairs to two ground stations 1,200 kilometres apart, and the photons showed the predicted correlations. Ground-based experiments have since sent entangled photons through hundreds of kilometres of optical fibre. Such links are the first steps towards a *quantum internet*, in which entanglement shared between distant quantum computers could be used for secure communication and for joining small processors into larger ones.

10.5 The four Bell states

The state $|\Phi^+\rangle$ has three close relatives, and together the four are called the *Bell states*:

$$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}, \quad |\Phi^-\rangle = (|00\rangle - |11\rangle)/\sqrt{2},$$
$$|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}, \quad |\Psi^-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}.$$

(Ψ is the capital Greek letter psi.) In the Φ states the two qubits always agree in a 0/1 measurement; in the Ψ states they always disagree. The sign distinguishes states that look identical in the 0/1 basis but behave differently in the diagonal basis. For example, in $|\Phi^-\rangle$ the diagonal-basis results always *disagree*, whereas in $|\Phi^+\rangle$ they always agree.

The four Bell states are mutually perpendicular arrows in the four-dimensional space of two qubits, so they form a basis, and a *Bell measurement* that distinguishes between them perfectly is possible. They are also easily converted into one another: applying X to one qubit of $|\Phi^+\rangle$ produces $|\Psi^+\rangle$, applying Z produces $|\Phi^-\rangle$, and applying both produces $|\Psi^-\rangle$ (up to a sign). Notice that these conversions are done by touching *only one* of the two qubits. Alice, on her own, can turn the shared pair into any of the four Bell states. This curious fact is the key to two protocols we will meet in Chapter 14, superdense coding and teleportation.

10.6 Entanglement among many

Entanglement is not limited to pairs. Three qubits can be in the state

$$(|000\rangle + |111\rangle)/\sqrt{2},$$

known as a GHZ state after Daniel Greenberger, Michael Horne and Anton Zeilinger, who studied it in the late 1980s. Its three qubits are all 0 or all 1, and as with the Bell states this correlation persists in other bases in a way that no pre-arranged instructions can explain. In fact GHZ states give a version of Bell's argument in which quantum mechanics and the instruction-table picture disagree not merely about averages but about the outcome of individual experiments.

In a large quantum computer the state of the register during a computation is typically entangled across all of the qubits in complicated ways. This is no accident. A quantum computation in which the qubits never become entangled, or only

slightly entangled, can be simulated efficiently on a classical computer, because each qubit (or each small group) can be tracked separately, with no exponential growth. Entanglement is therefore a necessary ingredient for a quantum computer to do anything a classical computer could not do just as well. It is not sufficient, however; there are highly entangled computations that are still easy to simulate classically. Like interference, entanglement is a resource that a good algorithm must use wisely.

10.7 Making entanglement

How do we create an entangled state in the first place? Not with single-qubit gates. A gate acting on one qubit of a product state produces another product state, since it changes that qubit's own state and leaves the other alone. To entangle two qubits, they must interact: we need a gate that acts on both of them together, so that what happens to one depends on the state of the other.

The standard such gate is the *controlled-NOT*, which flips one qubit if and only if the other is 1. As we will see at the start of the next chapter, a single Hadamard followed by a single controlled-NOT turns $|00\rangle$ into $|\Phi^+\rangle$. Building reliable two-qubit gates is among the hardest problems in quantum engineering, and the quality of a quantum computer is often summed up by how well it performs them.

Chapter summary. An entangled state of two qubits, such as $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$, cannot be written as a product of single-qubit states; neither qubit has a state of its own. Measurements on the two qubits are perfectly correlated, in the 0/1 basis and in every other basis if both measure along the same axis. Unlike the correlation of a pair of gloves, these correlations cannot be explained by instructions fixed in advance, as Bell showed. Entanglement cannot be used to send messages faster than light, because each party's own results look random whatever the other does. The four Bell states form a basis and can be converted into each other by acting on one qubit. Entanglement is needed for any quantum speed-up and is created by two-qubit gates such as the controlled-NOT.

Exercises

10.1 Show that the state $(|01\rangle + |10\rangle)/\sqrt{2}$ is not a product state, using the same argument as in Section 10.1.

10.2 Substitute $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$ and $|1\rangle = (|+\rangle - |-\rangle)/\sqrt{2}$ into $|00\rangle + |11\rangle$ and multiply out. Show that the $|+-\rangle$ and $|-+\rangle$ terms cancel and that the result is $|++\rangle + |--\rangle$. (The overall factor of $1/\sqrt{2}$ can be carried along at the end.)

10.3 Alice and Bob share the state $|\Psi^-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$. If both measure in the 0/1 basis, how are their results related? Without calculating, guess how they are related if both measure in the diagonal basis, then check by substitution as in Exercise 10.2.

10.4 Starting from $|\Phi^+\rangle$, apply Z to the first qubit. Which Bell state results? Now apply X to the first qubit of the result. Which Bell state is that?

10.5 Explain in your own words why Alice cannot use a shared $|\Phi^+\rangle$ to send Bob a message by choosing whether or not to measure her qubit.

CHAPTER 11

Two-qubit gates and quantum circuits

Single-qubit gates can rotate each qubit on its own Bloch sphere, but they cannot make qubits talk to each other. For that we need gates that act on two qubits at once. It turns out that one such gate, added to the single-qubit gates we already know, is enough to build any quantum computation whatsoever. This chapter introduces it, shows how it creates entanglement, and explains how a quantum computer can carry out ordinary arithmetic, which most quantum algorithms need as a component.

11.1 The controlled-NOT

The *controlled-NOT* gate, usually called CNOT, acts on two qubits, a *control* and a *target*. If the control is $|0\rangle$, nothing happens. If the control is $|1\rangle$, an X gate is applied to the target, flipping it. On the four basis states:

input	output
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|01\rangle$
$\|10\rangle$	$\|11\rangle$
$\|11\rangle$	$\|10\rangle$

(Here the first qubit is the control.) The target ends up holding the XOR of the two input bits, while the control passes through unchanged. In the language of Exercise 2.4, the CNOT is the reversible version of XOR: it keeps a copy of one input so that the computation can always be undone. Applying CNOT twice returns every state to where it started.

In circuit diagrams the CNOT is drawn as a vertical line joining a solid dot on the control qubit to a circle with a plus sign, ⊕, on the target. The ⊕ symbol is the mathematician's notation for XOR.

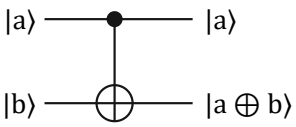


Figure 11.1. The controlled-NOT gate. The control qubit (dot) passes through unchanged, and the target (⊕) is flipped if the control is 1. For basis states the output on the target line is a ⊕ b, the XOR of the two bits.

So far this is just a classical gate. The quantum part is what happens to superpositions. As always, the gate acts on each term separately. So the state $\alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$ becomes $\alpha|00\rangle + \beta|01\rangle + \gamma|11\rangle + \delta|10\rangle$. In terms of the amplitudes, the CNOT swaps the amplitudes of $|10\rangle$ and $|11\rangle$ and leaves the other two alone. Its matrix, which acts on arrows in four dimensions, is

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

with the rows and columns in the order $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$.

11.2 Making a Bell pair

Now we can create entanglement. Start with two qubits in $|00\rangle$. Apply a Hadamard to the first qubit, which turns the state into

$$|+\rangle|0\rangle = (|00\rangle + |10\rangle)/\sqrt{2}.$$

This is still a product state. Now apply a CNOT with the first qubit as control. The term $|00\rangle$ is unchanged, because its control bit is 0. In the term $|10\rangle$ the control is 1, so the target flips, giving $|11\rangle$. The result is

$$(|00\rangle + |11\rangle)/\sqrt{2} = |\Phi^+\rangle,$$

the Bell state of Chapter 10. Two gates have turned two independent qubits into a perfectly entangled pair.

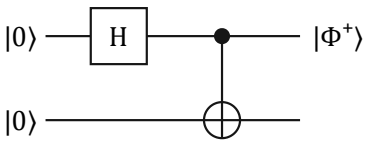


Figure 11.2. Creating the Bell state $|\Phi^+\rangle$. The Hadamard puts the control qubit into superposition; the controlled-NOT copies the control's value onto the target in each branch of the superposition, producing an entangled pair. Starting from $|01\rangle$, $|10\rangle$ and $|11\rangle$ instead gives the other three Bell states.

It is worth seeing why the result is entangled while the CNOT itself is a perfectly ordinary classical gate. In each branch of the superposition, the CNOT simply copies the control bit into the target: 0 into 0 in one branch, 1 into 1 in the other. Each branch on its own is boring. The entanglement arises because the control was in a superposition of both branches at once, and the CNOT tied the target's value to the control's in each branch separately. This pattern, a superposition followed by a classical-looking operation that correlates qubits branch by branch, appears again and again in quantum algorithms.

Running the circuit backwards undoes it. A CNOT followed by a Hadamard on the first qubit turns $|\Phi^+\rangle$ back into $|00\rangle$, and more generally turns each of the four Bell states into a different one of the four basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$. Measuring both qubits then tells us which Bell state we had. This reversed circuit is how a Bell measurement is carried out in practice, and we will use it in Chapter 14.

11.3 More two-qubit gates

The *controlled-Z* gate, or CZ, applies a Z gate to the target if the control is 1. Since Z only does something to $|1\rangle$, the net effect is to multiply the amplitude of $|11\rangle$ by -1 and leave everything else alone. This description is symmetric between the two qubits, so in a CZ there is no real distinction between control and target, and it is drawn with a dot on each qubit. It is related to the CNOT by the identity HZH = X from Chapter 8: a CZ with Hadamards on either side of the target qubit is a CNOT. Some quantum computers perform CZ gates natively and build CNOTs this way.

More generally, any single-qubit gate U can be turned into a *controlled-U* gate that applies U to the target only when the control is |1⟩. Controlled phase rotations, in particular, will be the main ingredient of the quantum Fourier transform in Chapter 15.

The *SWAP* gate exchanges the states of two qubits. It is drawn as two crosses joined by a line. It can be built from three CNOTs, with the direction of the middle one reversed, a trick also known in classical programming as the XOR swap.

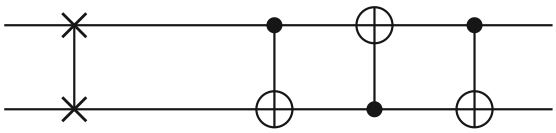


Figure 11.3. A SWAP gate (left) and its construction from three CNOT gates (right). Exercise 11.3 asks you to check that the two are equivalent by following the four basis states through.

SWAPs matter in practice because in most real devices a qubit can only interact directly with a few physical neighbours. If two distant qubits need a two-qubit gate, their states must first be moved next to each other by a chain of SWAPs, each costing three CNOTs. The layout of a chip therefore has a real effect on how long a given algorithm takes to run.

11.4 The Toffoli gate and classical logic

There is one three-qubit gate that deserves a special introduction. The *Toffoli gate*, named after Tommaso Toffoli who studied it in 1980, has two controls and one target. It flips the target if and only if *both* controls are 1. For this reason it is also called the controlled-controlled-NOT, or CCNOT.

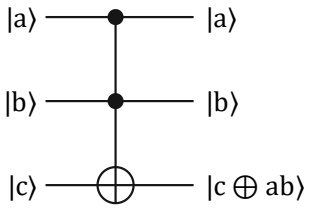


Figure 11.4. The Toffoli gate. The target is flipped only when both controls are 1. With the target starting at 0, the output on the target line is the AND of a and b.

If the target starts in |0⟩, the Toffoli gate leaves it holding a AND b, while keeping copies of a and b on the control lines. So the Toffoli gate is a reversible AND gate. If both controls are set to |1⟩, it acts as a NOT on the target. And from AND and NOT, as we saw in Chapter 2, all of classical computation can be built. Any classical computation can therefore be carried out by a circuit of Toffoli gates, and hence on a quantum computer, with only a modest overhead.

This might seem a curiosity, but it is crucial. Most quantum algorithms need to compute some ordinary function as one of their steps. Shor's algorithm, for instance, needs to compute powers of a number modulo another number, which is plain arithmetic. The Toffoli construction guarantees that whatever a classical computer can compute efficiently, a quantum computer can compute efficiently too, and moreover it can compute it on a superposition of inputs.

11.5 Cleaning up after yourself

There is a subtlety in running classical computations on a quantum computer. A reversible circuit cannot throw anything away, so as it computes it tends to leave behind intermediate results, often called *garbage*, on extra qubits. In a classical computer, garbage is harmless; you can ignore it or erase it. In a quantum computer it is not harmless at all. The garbage qubits are entangled with the rest of the computation, and as we saw with the interferometer, a record of which path was taken, even one that nobody looks at, destroys interference. Garbage left lying around would ruin the delicate cancellations that quantum algorithms depend on.

The solution, found by Charles Bennett in 1973, is beautifully simple. Compute the answer, leaving garbage behind. Copy the answer onto a fresh register using CNOT gates (copying a classical value in this way is allowed, as we will see in the

next chapter). Then run the whole computation backwards, which returns the garbage qubits to their original state and erases them. What remains is the input, unchanged, and the answer in the fresh register.

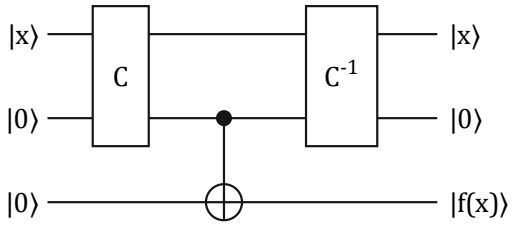


Figure 11.5. Bennett's trick. The circuit C computes f(x) into the second register, possibly leaving garbage there and on extra work qubits (not shown). A CNOT copies the answer into a fresh register. Running C backwards (C^{-1}) then cleans up, leaving the input and the answer and nothing else.

With this trick, any efficiently computable function f can be built into a clean quantum operation of the form

$$|x\rangle|y\rangle \to |x\rangle|y \oplus f(x)\rangle,$$

which keeps the input x and XORs the output f(x) into a target register y. Such an operation is often called an *oracle* for f, a name borrowed from computer science, where it describes a black box that answers questions about f. Most of the algorithms in Part III are described in terms of oracles: they assume that f is available in this form and ask how many times the box needs to be used.

11.6 Universality

We now have a collection of gates: Hadamard, the phase gates S and T, the Pauli gates and arbitrary single-qubit rotations, and the CNOT. A fundamental result says that this collection is enough. *Any* operation on any number of qubits can be built, to any desired accuracy, from single-qubit gates and CNOTs. In fact the small set consisting only of H, T and CNOT is already universal in this sense.

The phrase "to any desired accuracy" matters. Since there are infinitely many possible rotations, a finite set of gates cannot produce all of them exactly. But a remarkable theorem, due to Robert Solovay and Alexei Kitaev, shows that any single-qubit rotation can be approximated by a fairly short sequence of H and T

gates, and that improving the accuracy requires only a modest increase in length. This is important for error correction, where only a limited set of gates can be performed reliably.

Universality does not mean that every operation is *cheap*. Just as most classical functions of n bits need enormous circuits, most quantum operations on n qubits need a number of gates that grows exponentially with n. The operations that quantum algorithms use are special ones that happen to have short circuits. Finding such operations, and showing that they solve useful problems, is the whole difficulty of designing quantum algorithms.

11.7 Size and depth

Two numbers describe the cost of a quantum circuit. Its *width* is the number of qubits it uses. Its *depth* is the number of time steps it takes, where gates acting on different qubits at the same time count as a single step. Depth matters because qubits slowly lose their quantum state over time, a process we will study in Chapter 20, so a computation must finish before the qubits decay. Today's machines can run circuits with a few thousand gates at most before errors swamp the result. A full-scale run of Shor's algorithm on a cryptographically relevant number would need billions of gates. Closing that gap is the subject of Chapter 21.

Chapter summary. The controlled-NOT flips its target qubit when its control is 1; it is a reversible XOR and acts on superpositions term by term. A Hadamard followed by a CNOT turns $|00\rangle$ into the Bell state $|\Phi^+\rangle$, and the reverse circuit performs a Bell measurement. The controlled-Z, controlled-U and SWAP gates are built from the same ingredients. The Toffoli gate is a reversible AND, so any classical computation can be run on a quantum computer, and Bennett's trick of computing, copying and uncomputing removes the garbage that would otherwise destroy interference. Single-qubit gates plus CNOT are universal, though most operations would require exponentially many gates. A circuit's cost is measured by its width and depth.

Exercises

11.1 Apply the circuit of Figure 11.2 to the input $|10\rangle$. Which Bell state do you get? Repeat for $|01\rangle$ and $|11\rangle$.

11.2 Apply a CNOT (first qubit as control) to the product state $|+\rangle|+\rangle$. Is the result entangled? What about $|+\rangle|-\rangle$? (Write out the four terms in each case and look carefully.)

11.3 Check that the three-CNOT circuit of Figure 11.3 swaps the two qubits, by following $|01\rangle$ and $|10\rangle$ through it step by step.

11.4 Using Toffoli gates and a qubit prepared in $|0\rangle$, design a circuit that computes the OR of two bits a and b. (Hint: a OR b = NOT(NOT a AND NOT b).)

11.5 A circuit on 20 qubits consists of 10 layers, each layer containing a Hadamard on every qubit. What are its width, its depth and its total number of gates?

CHAPTER 12

What the quantum world forbids

Physics is as much about what cannot happen as about what can. Energy cannot be created from nothing; nothing travels faster than light; heat does not flow by itself from a cold object to a hot one. Quantum mechanics adds some prohibitions of its own, and they turn out to shape quantum computing just as much as superposition and entanglement do. In this chapter we meet two of them. The first is that an unknown quantum state cannot be copied. The second, Bell's theorem, says that the correlations of entangled particles cannot be explained by any hidden instructions carried along with them. Both can be understood with nothing more than the tools we already have.

12.1 No copying

Copying is so cheap in the classical world that we rarely think about it. Every time you send an email, open a web page or save a file, data is copied, often many times. In the half adder of Figure 2.1, a wire simply splits and each branch carries the same bit.

Could we build a quantum photocopier, a device that takes a qubit in any state $|\psi\rangle$ and a blank qubit in $|0\rangle$, and produces two qubits both in the state $|\psi\rangle$? Let us try. The CNOT gate is an obvious candidate, because it copies basis states perfectly:

$$\text{CNOT}\,|0\rangle|0\rangle = |0\rangle|0\rangle, \qquad \text{CNOT}\,|1\rangle|0\rangle = |1\rangle|1\rangle.$$

Now feed it the state $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. A working copier should produce $|+\rangle|+\rangle = (|00\rangle + |01\rangle + |10\rangle + |11\rangle)/2$. But the CNOT acts on each term separately, and so it produces

$$\text{CNOT}\,|+\rangle|0\rangle = (|00\rangle + |11\rangle)/\sqrt{2},$$

the Bell state. That is not two copies of $|+\rangle$; it is an entangled pair, with completely different properties. The copier fails.

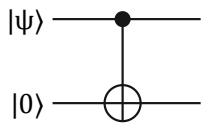


Figure 12.1. A failed quantum copier. For the inputs $|0\rangle$ and $|1\rangle$ the CNOT copies perfectly. For any superposition $\alpha|0\rangle + \beta|1\rangle$ it produces the entangled state $\alpha|00\rangle + \beta|11\rangle$ instead of two independent copies.

Perhaps the CNOT was a poor choice and some cleverer device would work? It would not. The argument above did not really use anything special about the CNOT. Any device that copies $|0\rangle$ and $|1\rangle$ correctly must, because every quantum operation acts on superpositions term by term, turn $\alpha|0\rangle + \beta|1\rangle$ into $\alpha|00\rangle + \beta|11\rangle$. But the correct copy would be

$$(\alpha|0\rangle + \beta|1\rangle)(\alpha|0\rangle + \beta|1\rangle) = \alpha^2|00\rangle + \alpha\beta|01\rangle + \alpha\beta|10\rangle + \beta^2|11\rangle,$$

which has cross terms $|01\rangle$ and $|10\rangle$ that the device cannot produce, unless α or β is zero. So no device can copy both basis states and their superpositions. A slightly more general version of the same argument shows that no device can copy any two states unless they are perpendicular. This is the *no-cloning theorem*, published in 1982 by William Wootters and Wojciech Zurek, and independently by Dennis Dieks.

It is important to be precise about what the theorem forbids. It does not forbid copying *known* states: if you know a qubit is in the state $|+\rangle$, you can simply prepare another one. It does not forbid copying classical information: the CNOT copies basis states perfectly, and Bennett's trick in Chapter 11 relies on this. What is forbidden is a universal machine that copies an *unknown* quantum state.

12.2 Why no-cloning matters

The no-cloning theorem has consequences everywhere in quantum information.

It explains why measurement is such a one-shot affair. If we could copy an unknown qubit, we could make a thousand copies, measure some in the 0/1 basis and some in the diagonal basis, and work out the state exactly. No-cloning closes this loophole: one unknown qubit yields one measurement, and that is all.

It protects the no-signalling principle of Section 10.4. If Bob could clone his half of an entangled pair, he could check whether his copies were all 0 or all 1 (meaning Alice had measured in the 0/1 basis and collapsed his qubit) or a mixture (meaning she had not). Alice could then send messages instantaneously. No-cloning prevents it.

It underlies the security of quantum cryptography, as we will see in Chapter 19. An eavesdropper cannot copy the qubits passing between Alice and Bob in order to measure them at leisure.

And it makes quantum error correction hard. The simplest classical way to protect information is to make several copies and take a majority vote. For quantum information, copying is impossible, and for some years after Shor's algorithm was discovered many physicists believed this meant quantum computers could never be protected from errors. Chapter 21 tells how the problem was solved without copying anything.

12.3 Telling states apart

A close relative of no-cloning is the impossibility of distinguishing non-perpendicular states. Suppose someone hands you a qubit and promises that it is either $|0\rangle$ or $|+\rangle$. Can you find out which?

Not with certainty. These two states are 45° apart in the arrow picture, not 90°. Any measurement corresponds to a pair of perpendicular arrows, and there is no pair for which $|0\rangle$ lies along one and $|+\rangle$ along the other. Measure in the 0/1 basis, for example, and a result of 1 tells you for sure that you had $|+\rangle$, but a result of 0 is ambiguous, since $|+\rangle$ gives 0 half of the time too. Every possible measurement has some chance of being wrong or inconclusive. Only perpendicular states, opposite points on the Bloch sphere, can be told apart perfectly.

12.4 Bell's game

We now return to the question raised in Chapter 10: could the correlations of an entangled pair be explained by instructions planted in each particle when the pair was created, like the left and right gloves in two boxes? John Bell's answer can be explained as a game, in a version invented in 1969 by John Clauser, Michael Horne, Abner Shimony and Richard Holt and known by their initials as CHSH.

Alice and Bob play as a team against a referee. They may plan a strategy together beforehand, but once the game starts they are separated and cannot communicate. The referee sends each of them a random bit, x to Alice and y to Bob, each 0 or 1 with equal chance. Each replies with a bit of their own, a from Alice and b from Bob. They win the round if

$$a \oplus b = x \text{ AND } y.$$

In words: if both questions were 1, they win by giving *different* answers; in the other three cases (x and y not both 1) they win by giving the *same* answer.

What is the best classical strategy? Alice's answer can depend only on her own question x, and Bob's only on y. The simplest idea is for both always to answer 0. Then their answers always agree, and they win in the three cases where x AND y = 0, losing only when x = y = 1. That wins three-quarters of the rounds. Can they do better? Try any rule: Alice says a_0 when asked 0 and a_1 when asked 1; Bob says b_0 and b_1. Winning all four cases would require

$$a_0 \oplus b_0 = 0, \quad a_0 \oplus b_1 = 0, \quad a_1 \oplus b_0 = 0, \quad a_1 \oplus b_1 = 1.$$

XOR all four left-hand sides together. Each of the four letters appears exactly twice, so the total is 0. But the right-hand sides XOR to 1. The four conditions contradict each other, so any deterministic strategy must lose at least one of the four cases, and wins at most 3/4 of the rounds on average. Strategies that use shared random numbers (say, a list of coin tosses agreed in advance) are just random mixtures of deterministic strategies, so they cannot do better either. This includes every possible scheme of "hidden instructions" carried by particles. The limit of 75% is a form of *Bell's inequality*.

case	needed	"always answer 0"
x = 0, y = 0	same	win
x = 0, y = 1	same	win
x = 1, y = 0	same	win
x = 1, y = 1	different	lose

12.5 Winning more often with entanglement

Now give Alice and Bob a shared entangled pair in the state $|\Phi^+\rangle$ before they separate. When they receive their questions, each measures their own qubit in a basis that depends on the question, and answers with the result.

To describe the strategy we need one fact about $|\Phi^+\rangle$. If Alice measures along a direction at angle α in the arrow picture (that is, in the basis consisting of the unit arrow at angle α and the arrow perpendicular to it), and Bob measures along a direction at angle β, then their results agree with probability

$$(\cos(\alpha - \beta))^2.$$

We already know two cases of this. When both measure in the 0/1 basis, $\alpha = \beta = 0$, and they always agree, since $\cos 0 = 1$. When both measure in the diagonal basis, $\alpha = \beta = 45°$, and again they always agree, as we found in Section 10.3.

The quantum strategy is this. Alice measures at angle 0° if asked 0 and at 45° if asked 1. Bob measures at angle 22.5° if asked 0 and at −22.5° if asked 1.

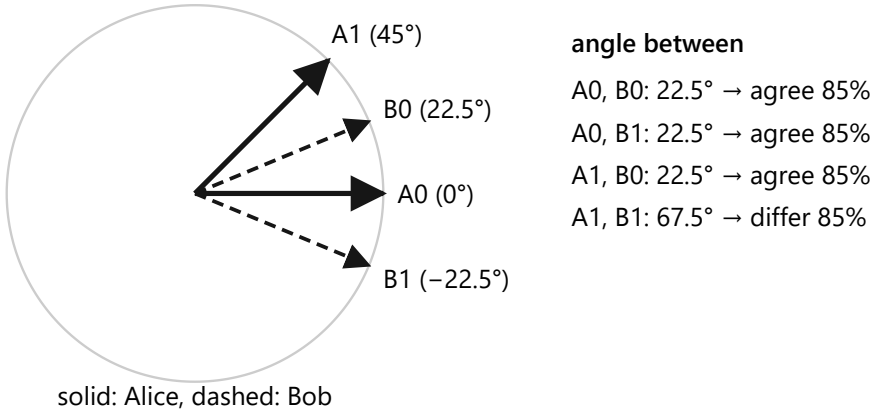


Figure 12.2. The quantum strategy for the CHSH game. Alice measures along A0 or A1 and Bob along B0 or B1, depending on their questions. In the three cases where they need to agree, their directions are 22.5° apart; in the case where they need to differ, the directions are 67.5° apart.

Look at the four cases. When the questions are (0, 0), (0, 1) or (1, 0), the angle between Alice's and Bob's directions is 22.5°, and they agree with probability $(\cos 22.5°)^2 \approx 0.854$. Agreeing is exactly what they need. When the questions are (1, 1), the angle is 67.5°, and they agree with probability $(\cos 67.5°)^2 \approx 0.146$, so they *disagree* with probability about 0.854, which again is exactly what they need. In every case they win with probability about 85.4%.

That is more than the 75% that any classical strategy, any scheme of hidden instructions, can achieve. The entangled pair lets Alice and Bob coordinate their answers in a way that is impossible without it, even though no message passes between them, and even though (as we saw in Section 10.4) each of their answers on its own is a perfectly random bit. The number 85.4% is not an accident of this particular strategy; Boris Tsirelson proved in 1980 that no quantum strategy can do better.

12.6 The experiments

Bell's theorem turns a philosophical debate into an experiment: prepare entangled pairs, let two distant detectors choose their measurement settings at random, and see whether the win rate exceeds 75%. Early experiments by John Clauser and

Stuart Freedman in 1972 and, more convincingly, by Alain Aspect and his colleagues near Paris in the early 1980s found clear violations of Bell's inequality, in agreement with quantum mechanics.

Sceptics could still point to *loopholes*. Perhaps the detectors missed so many particles that the ones they caught were unrepresentative. Perhaps the measurement settings were chosen too slowly, leaving time for a signal at light speed to pass between the two sides. In 2015 three groups, in Delft, Vienna and Boulder, closed both loopholes at once. The Delft experiment placed its two entangled electrons 1.3 kilometres apart on the university campus, and still found the quantum correlations. In 2022 the Nobel Prize in Physics was awarded to Aspect, Clauser and Anton Zeilinger for this line of work.

What should we conclude? Einstein hoped that the outcomes of measurements were determined in advance by properties carried along with each particle, and that nothing done in one place could affect what happens instantaneously far away. This combination of ideas is usually called *local realism*. Bell's theorem and the experiments show that local realism is false: nature does not work that way. Exactly which part of it to give up is still argued over by physicists and philosophers. For quantum computing, the lesson is more practical. Entanglement provides correlations that no classical resource can provide, and a computer built from entangled qubits is not a classical computer in disguise.

Chapter summary. The no-cloning theorem says that no device can copy an arbitrary unknown quantum state, because quantum operations act on superpositions term by term; known states and classical data can still be copied. No-cloning explains why a single qubit yields only one measurement, protects no-signalling and quantum cryptography, and makes error correction difficult. Non-perpendicular states cannot be told apart with certainty. In the CHSH game, any classical strategy, including hidden instructions, wins at most 75% of rounds, while players sharing an entangled pair can win about 85.4%. Experiments since the 1970s, decisively in 2015, confirm the quantum prediction and rule out local realism.

Exercises

12.1 A device copies $|+\rangle$ and $|-\rangle$ perfectly: $|+\rangle|0\rangle \to |+\rangle|+\rangle$ and $|-\rangle|0\rangle \to |-\rangle|-\rangle$. What does it do to $|0\rangle$? (Use $|0\rangle = (|+\rangle + |-\rangle)/\sqrt{2}$.) Is the output two copies of $|0\rangle$?

12.2 List all 16 deterministic strategies for the CHSH game (Alice chooses a_0, a_1; Bob chooses b_0, b_1), or at least a handful of them, and check that none wins all four cases.

12.3 In the quantum strategy, suppose Bob uses angles 45° and −45° instead of ±22.5°. Compute the probability of winning in each of the four cases and the average. Is it better or worse than 75%?

12.4 Explain why the no-cloning theorem does not stop you from making a backup of your hard disk on a quantum computer.

Part III: Quantum algorithms

Recipes that use superposition, entanglement and interference to solve problems with fewer steps than any classical method.

CHAPTER 13

Interference as a way of computing

We have all the ingredients now: qubits, gates, measurement, entanglement. It is time to compute something. This chapter presents the first quantum algorithms ever discovered. The problems they solve are artificial, chosen to show off a quantum advantage as cleanly as possible rather than to be useful. But they reveal, in their simplest form, the pattern that every later algorithm follows: put the input into superposition, let a function act on all of it at once, and then use interference to concentrate the amplitude on an answer that reveals a global property of the function.

13.1 A question about a black box

Imagine a black box with one input and one output, each a single bit. Inside is one of four possible functions f:

function	f(0)	f(1)	type
f_1	0	0	constant
f_2	1	1	constant
f_3	0	1	balanced
f_4	1	0	balanced

Two of the functions give the same output whatever the input; they are called *constant*. The other two give different outputs for the two inputs; they are called *balanced*. Your task is to decide whether the function in the box is constant or balanced. You do not need to know which of the four it is, only which type.

Classically there is no shortcut. You feed in 0 and see f(0). That tells you nothing about the type, because each value of f(0) is consistent with one constant and one balanced function. You must also feed in 1 and compare. Two uses of the box are necessary.

A story can make the problem more vivid. A coin collector is given a coin and wants to know whether it is genuine, with a head on one side and a tail on the other, or a forgery with the same picture on both sides. Looking at one side tells her nothing; she has to look at both. The question "are the two sides the same or different?" is a question about a *relationship* between the two values, and classically a relationship can only be checked by obtaining both values.

In 1985 David Deutsch showed that a quantum computer can answer the question with a single use of the box. A slightly improved version of his algorithm, due to Richard Cleve, Artur Ekert, Chiara Macchiavello and Michele Mosca in 1998, is the one we describe here.

13.2 Phase kickback

First we need to make the black box quantum. Following Section 11.5, we assume it is available as an oracle acting on two qubits:

$$U_f\,|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle.$$

The first qubit is the input and passes through unchanged; the value f(x) is XORed into the second qubit.

Now comes the trick on which the whole chapter rests. Suppose the second qubit is prepared not in $|0\rangle$ or $|1\rangle$ but in the state $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. What does the oracle do?

If f(x) = 0, nothing is XORed in, and the second qubit stays $|-\rangle$. If f(x) = 1, the second qubit is flipped: $|0\rangle - |1\rangle$ becomes $|1\rangle - |0\rangle$, which is $-(|0\rangle - |1\rangle)$. In other words, the second qubit stays in the state $|-\rangle$ either way, but when f(x) = 1 the whole state picks up a minus sign. We can write both cases together as

$$U_f\,|x\rangle|-\rangle = (-1)^{f(x)}\,|x\rangle|-\rangle. \tag{13.1}$$

Here $(-1)^{f(x)}$ means +1 if f(x) = 0 and −1 if f(x) = 1. The value of the function has been turned into a sign. And although the sign arose from flipping the second qubit, it can just as well be thought of as belonging to the first qubit, since a sign

multiplies the whole state. This effect is called *phase kickback*: the target qubit stays as it was, and the phase is "kicked back" onto the control.

On its own, with the first qubit in a basis state, the minus sign is a harmless global phase. But if the first qubit is in a superposition, different terms of the superposition can pick up different signs, and then the signs are relative phases, which, as we know, can be detected by interference.

13.3 Deutsch's algorithm

The algorithm uses two qubits and four steps.

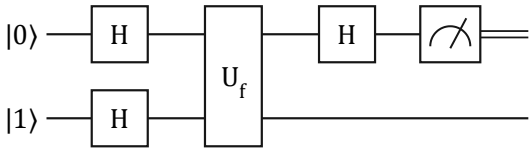


Figure 13.1. Deutsch's algorithm. The second qubit starts in |1⟩ and a Hadamard turns it into |−⟩, ready for phase kickback. The first qubit is put into |+⟩, passed through the oracle once, and measured in the diagonal basis (via a Hadamard). The outcome is 0 if f is constant and 1 if f is balanced.

Step 1. Prepare the first qubit in |0⟩ and the second in |1⟩, and apply a Hadamard to each. The first becomes $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ and the second becomes |−⟩.

Step 2. Apply the oracle once. By phase kickback, each term of the first qubit's superposition picks up the sign $(-1)^{f(x)}$. The first qubit is now in the state

$$\left((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle \right) / \sqrt{2},$$

and the second qubit, still |−⟩, plays no further part.

Step 3. Look at the state of the first qubit. If f is constant, f(0) and f(1) are equal, so both terms have the same sign, and the state is ±|+⟩. If f is balanced, the two signs differ, and the state is ±|−⟩. The overall ± is a global phase and does not matter. So the type of function has been recorded in the relative sign between the amplitudes of |0⟩ and |1⟩, exactly the kind of information that distinguishes |+⟩ from |−⟩. Apply a Hadamard to the first qubit, which turns |+⟩ into |0⟩ and |−⟩ into |1⟩.

Step 4. Measure the first qubit. If the result is 0, the function is constant; if 1, it is balanced. The answer is certain, not merely probable.

Let us follow one case explicitly, to see the interference at work. Take the balanced function f_3, with f(0) = 0 and f(1) = 1. After step 2 the first qubit is $(|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle$. The final Hadamard turns it into $|1\rangle$, so the measurement gives 1, "balanced". For the constant function f_1, both signs are +, the qubit is $|+\rangle$, the Hadamard turns it into $|0\rangle$, and the measurement gives 0, "constant".

The oracle was used once. Classically, it had to be used twice. The quantum algorithm did not learn f(0) or f(1). It learned only whether they are equal, which is exactly what was asked, and it learned it by evaluating f on a superposition of both inputs and letting the two results interfere. In the language of the coin collector, it looked at both sides of the coin at once and asked only whether they matched.

Box 13.1: Where did the second query go?

It is tempting to say that the quantum computer evaluated f twice, once for each term of the superposition, "in parallel". That is a reasonable way of speaking, provided one remembers that the two values are never available separately. If we measured the first qubit in the 0/1 basis after step 2, we would see 0 or 1 at random and learn nothing about f at all. The power of the algorithm lies not in having both values but in combining them through interference into a single bit that answers a question about their relationship. That is the pattern to watch for in every quantum algorithm.

13.4 The Deutsch–Jozsa algorithm

Saving one query out of two is not much to boast about. In 1992 Deutsch and Richard Jozsa generalised the problem so that the gap becomes dramatic.

Now the black box takes an input of n bits, so there are 2^n possible inputs, and it outputs a single bit. We are promised that the function is either *constant* (the same output for every input) or *balanced* (output 0 for exactly half the inputs and 1 for the other half). Our task is again to decide which.

Classically, in the worst case, this is laborious. Suppose you query the box and get 0 every time. You cannot be sure the function is constant until you have checked more than half of the inputs, because a balanced function could have

hidden all its 1s among the inputs you have not yet tried. So a classical computer that must be certain needs $2^{n-1} + 1$ queries in the worst case. For n = 100 that is more than 10^{29}.

The quantum algorithm needs exactly one. It is the same as Deutsch's algorithm, with the single input qubit replaced by n of them.

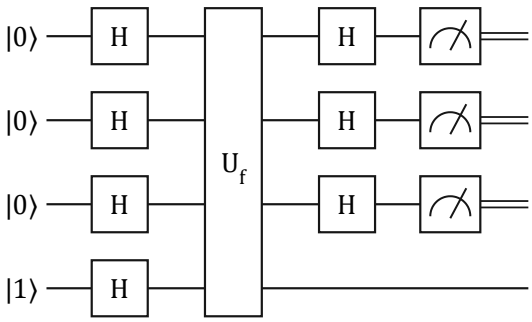


Figure 13.2. The Deutsch–Jozsa algorithm, drawn for n = 3 input qubits. Hadamards prepare an equal superposition of all 2^n inputs, one oracle call kicks the value of f into the signs, and a second layer of Hadamards followed by measurement reveals whether f is constant (all results 0) or balanced (at least one result 1).

After the first layer of Hadamards, the input register holds an equal superposition of all 2^n inputs, as in Section 9.4. After the oracle, phase kickback has attached the sign $(-1)^{f(x)}$ to each input x. After the second layer of Hadamards, we ask: what is the amplitude of the outcome where every qubit reads 0?

It turns out (Box 13.2 explains why) that this amplitude is simply the *average* of all the signs $(-1)^{f(x)}$. If f is constant, all the signs are the same, and the average is +1 or −1; the outcome 00...0 then occurs with certainty. If f is balanced, half the signs are +1 and half are −1, and their average is exactly 0; the outcome 00...0 never occurs. So we measure all the input qubits once. If they are all 0, the function is constant. If any of them is 1, it is balanced.

This is interference on a large scale. In the balanced case, 2^n contributions to the amplitude of 00...0 arrive, half with a plus sign and half with a minus sign, and they cancel perfectly, just as the two routes cancelled at detector B of the interferometer.

Box 13.2: Why the average?

A Hadamard sends $|0\rangle$ to $(|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ to $(|0\rangle - |1\rangle)/\sqrt{2}$. Either way, the amplitude for ending up in $|0\rangle$ is $+1/\sqrt{2}$. So when a layer of n Hadamards acts on any basis state $|x\rangle$, the amplitude for ending up in $|00\ldots0\rangle$ is $(1/\sqrt{2})^n$, with a plus sign, whatever x was. Before the second layer, each of the 2^n inputs x had amplitude $(-1)^{f(x)}/\sqrt{(2^n)}$. Multiplying and adding over all x, the final amplitude of $|00\ldots0\rangle$ is the sum of $(-1)^{f(x)}/2^n$, which is precisely the average of the signs.

How impressive is this? The problem is contrived, and if we allow a classical computer a small chance of error the advantage largely disappears: query a handful of random inputs, and if they all agree, declare "constant". A balanced function will be caught out with high probability after a few tries. So the Deutsch–Jozsa algorithm does not demonstrate a practically useful speed-up. Its importance is conceptual. It was the first example in which a quantum computer provably outperformed any deterministic classical computer by an exponential factor, and it showed in miniature how interference can extract a global property of a function from a single query.

13.5 Finding a hidden string

A small variation of the same circuit solves a problem where the quantum advantage survives even against randomised classical computers. It was found by Ethan Bernstein and Umesh Vazirani in 1993.

Suppose the black box hides a secret string s of n bits, and on input x it returns the XOR of those bits of x for which the corresponding bit of s is 1. (Mathematically, f(x) is the "dot product" of s and x, counted modulo 2.) For example, with s = 101, the box returns $x_1 \oplus x_3$, ignoring the middle bit of x. The task is to find s.

Classically, each query returns a single bit, so at least n queries are needed; the natural method is to query with x = 100...0, then 010...0 and so on, reading off one bit of s each time. The Bernstein–Vazirani algorithm uses exactly the circuit of Figure 13.2, with one query, and the measurement outcome *is* s, with certainty. One query against n. The reason, briefly, is that phase kickback attaches to each input x the sign $(-1)^{s\cdot x}$, and the pattern of signs produced by such a function is

precisely the pattern that a layer of Hadamards produces from the basis state |s⟩. Since Hadamards undo themselves, the second layer turns the pattern back into |s⟩.

13.6 The pattern

All the algorithms in this chapter share a single shape: Hadamards, oracle, Hadamards, measure. The first Hadamards spread the computation over all possible inputs. The oracle marks each input with a sign that depends on the function. The second Hadamards make all those signs interfere, so that the amplitude collects on outcomes that reveal something about the function as a whole. The Hadamard layer is acting here as a very simple kind of *Fourier transform*, a mathematical tool for detecting patterns and regularities. In Chapter 15 we will meet the full quantum Fourier transform, which can detect far richer patterns, and in Chapter 16 we will see it reveal the hidden period at the heart of Shor's factoring algorithm.

Chapter summary. Deutsch's problem asks whether a one-bit function is constant or balanced; classically this needs two queries, quantumly one. The key trick is phase kickback: with the target qubit in |−⟩, the oracle turns the value f(x) into a sign $(-1)^{f(x)}$ on the input. A Hadamard then converts the relative sign into a definite measurement outcome. The Deutsch–Jozsa algorithm extends this to n-bit functions with one query instead of up to $2^{n-1} + 1$, although randomised classical methods narrow the gap; the Bernstein–Vazirani algorithm finds a hidden n-bit string with one query instead of n. All follow the pattern Hadamards, oracle, Hadamards, measure, in which interference reveals a global property of the function.

Exercises

13.1 Work through Deutsch's algorithm for the functions f_2 and f_4 in the table of Section 13.1. What is the state of the first qubit after step 2 in each case, and what does the measurement give?

13.2 In step 1, what would go wrong if the second qubit were prepared in |0⟩ (so that after the Hadamard it was in |+⟩) instead of |1⟩? (Hint: what does the oracle do to |+⟩ when f(x) = 1?)

13.3 For n = 2, write down a balanced function and check, by computing the average of the signs, that the amplitude of the outcome 00 is zero.

13.4 In the Bernstein–Vazirani problem with s = 110, what does the box output for the inputs 000, 011 and 111?

13.5 A randomised classical computer queries the Deutsch–Jozsa box on k random inputs and declares "constant" if all outputs agree. If the function is actually balanced, roughly how likely is it to be fooled after k = 10 queries? (Hint: each new output has about a one-half chance of matching the first.)

CHAPTER 14

Teleportation and superdense coding

The word “teleportation” conjures up images from science fiction: a person dissolving into sparkling light in one place and reassembling somewhere else. Quantum teleportation is less dramatic but in its own way more surprising. It is a procedure for transferring the exact quantum state of a qubit from one place to another, without sending the qubit itself, using only a shared entangled pair and two bits of ordinary classical communication. It was discovered in 1993 by a group of six physicists led by Charles Bennett, demonstrated in the laboratory in 1997, and is now a standard tool in quantum computing and quantum networking. Its mirror image, superdense coding, uses entanglement to send two classical bits by transmitting only one qubit. Both protocols are short, and working through them is the best possible exercise in the ideas of Part II.

14.1 The problem

Alice has a qubit in some state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. She does not know α and β; the qubit might be the output of a computation, or something handed to her by someone else. She wants Bob, who is far away, to end up with a qubit in exactly the same state.

The obvious solution is to post the qubit, or send it as a photon down an optical fibre. Often that is not possible: the qubit might be an ion that cannot leave its trap, or the channel between Alice and Bob might be too noisy to carry fragile quantum states. Can Alice instead send Bob a *description* of the state, over an ordinary telephone line, so that he can build a copy?

She cannot. To describe the state she would need to know α and β, and as we saw in Chapter 6, measuring a single qubit tells her only one bit and destroys the rest. Even if she somehow knew them, they are continuous numbers, and describing them exactly would take infinitely many bits.

Teleportation solves the problem with the help of one extra resource: an entangled pair shared in advance, one qubit with Alice and one with Bob.

14.2 The protocol

At the start, Alice holds two qubits: the qubit to be teleported, in the state $|\psi\rangle$, and her half of a Bell pair $|\Phi^+\rangle$. Bob holds the other half of the Bell pair. The protocol has four steps.

1. Alice applies a CNOT, with the $|\psi\rangle$ qubit as control and her half of the Bell pair as target.
2. Alice applies a Hadamard to the $|\psi\rangle$ qubit.
3. Alice measures both of her qubits, getting two classical bits, and sends them to Bob by any ordinary means (a phone call, an email).
4. Depending on the two bits, Bob applies X, Z, both or neither to his qubit. After this, his qubit is in the state $|\psi\rangle$.

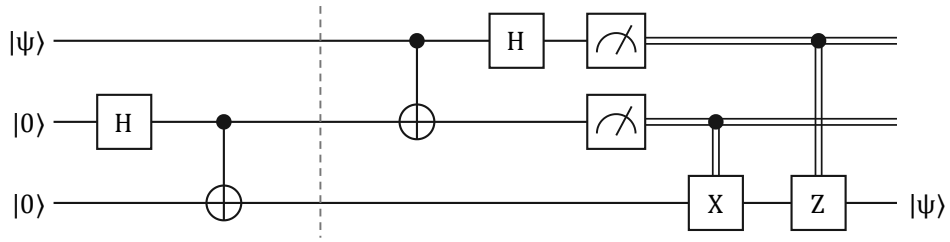


Figure 14.1. Quantum teleportation. Left of the dashed line, a Bell pair is created between the middle qubit (Alice's) and the bottom qubit (Bob's); this can happen long in advance. Right of the line, Alice performs a CNOT and a Hadamard on her two qubits and measures them. Her two results travel to Bob along classical wires (double lines) and decide which corrections he applies. Bob's qubit ends up in the state $|\psi\rangle$.

Notice what Alice's first two steps are: a CNOT followed by a Hadamard. That is precisely the reversed Bell-pair circuit of Section 11.2, the circuit that performs a Bell measurement. So in words, the protocol is: Alice performs a Bell measurement

on the unknown qubit and her half of the entangled pair, tells Bob the result, and Bob makes a correction.

14.3 Why it works

Let us follow the state through the protocol. It is the longest calculation in this book, but every step uses only what we already know. Label the three qubits from top to bottom of Figure 14.1, so that the first is the unknown qubit, the second is Alice's half of the pair and the third is Bob's.

The starting state, written out by multiplying the brackets, is

$$(\alpha|0\rangle + \beta|1\rangle)\,(|00\rangle + |11\rangle)/\sqrt{2} = (\,\alpha|000\rangle + \alpha|011\rangle + \beta|100\rangle + \beta|111\rangle\,)\,/\,\sqrt{2}.$$

After the CNOT (first qubit controls the second): in the two terms where the first qubit is 1, the second qubit flips.

$$(\,\alpha|000\rangle + \alpha|011\rangle + \beta|110\rangle + \beta|101\rangle\,)\,/\,\sqrt{2}.$$

After the Hadamard on the first qubit: each $|0\rangle$ in the first position becomes $(|0\rangle + |1\rangle)/\sqrt{2}$ and each $|1\rangle$ becomes $(|0\rangle - |1\rangle)/\sqrt{2}$. Expanding gives eight terms:

$$\tfrac{1}{2}\,[\,\alpha|000\rangle + \alpha|100\rangle + \alpha|011\rangle + \alpha|111\rangle + \beta|010\rangle - \beta|110\rangle + \beta|001\rangle - \beta|101\rangle\,].$$

Now comes the step that makes everything clear. Group the terms according to the values of the first two qubits, which are the ones Alice is about to measure:

$$\tfrac{1}{2}\,[\,|00\rangle(\alpha|0\rangle + \beta|1\rangle) + |01\rangle(\alpha|1\rangle + \beta|0\rangle) + |10\rangle(\alpha|0\rangle - \beta|1\rangle) + |11\rangle(\alpha|1\rangle - \beta|0\rangle)\,].$$

Read this line slowly. Each of Alice's four possible results, 00, 01, 10 and 11, occurs with probability $(\tfrac{1}{2})^2 = \tfrac{1}{4}$. And next to each result, the state of Bob's qubit is a slightly scrambled version of $|\psi\rangle$.

Alice's result	Bob's qubit	Bob's correction
00	$\alpha\|0\rangle + \beta\|1\rangle$	none
01	$\alpha\|1\rangle + \beta\|0\rangle$	X
10	$\alpha\|0\rangle - \beta\|1\rangle$	Z
11	$\alpha\|1\rangle - \beta\|0\rangle$	X, then Z

When Alice measures, the state collapses to one of the four lines, and Bob's qubit is left in the corresponding scrambled state. In the first case it is already $|\psi\rangle$. In the second, the amplitudes are swapped, which an X gate undoes. In the third, the sign of β is wrong, which a Z gate undoes. In the fourth, both have happened, and Bob applies X and then Z: the X turns $\alpha|1\rangle - \beta|0\rangle$ into $\alpha|0\rangle - \beta|1\rangle$, and the Z turns that into $\alpha|0\rangle + \beta|1\rangle$, which is exactly $|\psi\rangle$. In every case Bob ends up with $|\psi\rangle$, as long as he knows which of the four cases occurred. That is what Alice's two bits tell him.

14.4 What teleportation does not do

Teleportation is easily misunderstood, so it is worth listing what it does not do.

It does not send anything faster than light. Until Bob receives Alice's two classical bits, which travel at or below light speed, his qubit is in one of four scrambled states, each equally likely, and from his point of view it is completely random. Averaged over the four possibilities, his qubit is at the centre of the Bloch sphere and carries no information about $|\psi\rangle$ at all. Only when the bits arrive can he unscramble it.

It does not copy the state. Alice's measurement leaves her two qubits in the definite basis states she observed, with no trace of α and β. The original is destroyed in the process of creating the new one, in perfect agreement with the no-cloning theorem of Chapter 12.

It does not transmit matter or energy. Nothing physical travels from Alice to Bob except the classical bits. The qubit that ends up in the state $|\psi\rangle$ is the one Bob had all along; what has been transferred is the state, the pattern of amplitudes, not the particle.

And it does not need anyone to know the state. Neither Alice nor Bob learns anything about α and β. Alice's measurement results are completely random (each outcome has probability ¼ whatever $|\psi\rangle$ is), so they reveal nothing about the state, and Bob just follows the table.

There is something rather wonderful about the accounting. A qubit's state is described by continuous amplitudes that would take infinitely many bits to write down. Teleportation transfers it with exactly two bits of classical communication, plus one shared entangled pair, which is used up in the process. Entanglement, it seems, is a genuine resource, like fuel, and teleportation is one way of spending it.

Box 14.1: Teleportation in practice

The first teleportation experiments, in 1997 in Innsbruck and Rome, teleported the polarisation state of photons across a laboratory table. Since then the distances have grown steadily: across the Danube in Vienna, between two Canary Islands 143 kilometres apart in 2012, and from a ground station in Tibet to the Micius satellite in orbit in 2017. Inside quantum computers, teleportation is used to move quantum states between parts of a processor that cannot interact directly, and it is a key ingredient in proposals for linking separate quantum computers into networks. Some error-correction schemes even use a form of teleportation to apply hard-to-implement gates, by teleporting a qubit "through" a specially prepared state.

14.5 Superdense coding

Teleportation uses two classical bits and one entangled pair to send one qubit. *Superdense coding*, discovered by Bennett and Stephen Wiesner in 1992, does the reverse: it uses one qubit and one entangled pair to send two classical bits.

Again Alice and Bob share a Bell pair $|\Phi^+\rangle$. Alice wants to send Bob two bits, say b_1 and b_2. Recall from Section 10.5 that by acting on her qubit alone, Alice can turn the shared pair into any of the four Bell states. So she does this: if $b_2 = 1$ she applies X to her qubit, and then if $b_1 = 1$ she applies Z. The pair is now in one of four Bell states, one for each of the four possible messages. She then sends her qubit to Bob.

Bob now holds both qubits of the pair. He performs a Bell measurement, the CNOT followed by a Hadamard of Section 11.2, and measures both qubits. Because the four Bell states are perfectly distinguishable, he learns which one he has, and hence both of Alice's bits.

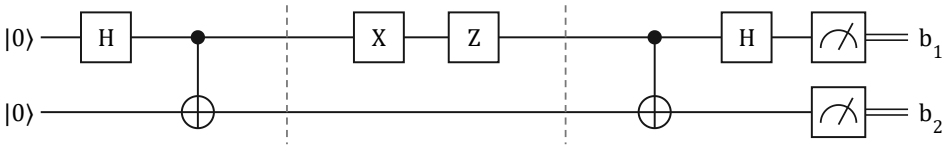


Figure 14.2. Superdense coding. A Bell pair is shared in advance (left). Alice encodes two bits by applying X and/or Z to her qubit (middle; each gate is applied only if the corresponding bit is 1). She sends her qubit to Bob, who performs a Bell measurement (right) and reads off both bits.

Only one qubit travelled from Alice to Bob, yet it carried two bits. Does this contradict Holevo's theorem from Section 5.6, which says that one qubit can carry at most one bit? Not quite. The second qubit, Bob's half of the pair, also carries information, and it had to reach Bob at some earlier time. Counting both qubits, two qubits delivered two bits. What superdense coding shows is that one of the two qubits can be sent in advance, before the message has even been chosen, and the message still fits into the one qubit sent afterwards. Shared entanglement doubles the capacity of the quantum channel.

14.6 A symmetry

Teleportation and superdense coding fit together neatly:

$$1 \text{ entangled pair} + 2 \text{ classical bits} \rightarrow 1 \text{ qubit transmitted (teleportation)}$$
$$1 \text{ entangled pair} + 1 \text{ qubit transmitted} \rightarrow 2 \text{ classical bits (superdense coding)}$$

Each converts one kind of resource into another, with entanglement as the catalyst that is consumed along the way. This way of thinking, treating qubits, bits and entangled pairs as interchangeable currencies with fixed exchange rates, has become one of the central themes of quantum information theory. It also explains why the experimental ability to create and distribute entanglement reliably is regarded as so important: entanglement is the currency in which the quantum internet will trade.

Chapter summary. Quantum teleportation transfers an unknown qubit state from Alice to Bob using a shared Bell pair and two classical bits. Alice performs a Bell measurement (CNOT, Hadamard, measure) on the unknown qubit and her half of the pair; Bob's qubit is left in one of four scrambled versions of the state, and Alice's two bits tell him which X and Z corrections undo the scrambling. Teleportation does not send information faster than light, does not copy the state (the original is destroyed) and reveals nothing about the amplitudes. Superdense coding reverses the exchange: by applying X and/or Z to her half of a shared pair and sending it, Alice transmits two classical bits with one qubit. Entanglement acts as a consumable resource in both protocols.

Exercises

14.1 Check the third row of the table in Section 14.3: apply Z to $\alpha|0\rangle - \beta|1\rangle$ and confirm that the result is $|\psi\rangle$.

14.2 Suppose Alice teleports the state $|1\rangle$ (so $\alpha = 0$ and $\beta = 1$). Using the grouped expression in Section 14.3, write down Bob's qubit for each of Alice's four results. Does Bob's qubit before correction tell him anything about whether Alice sent $|0\rangle$ or $|1\rangle$?

14.3 In superdense coding, which Bell state does the pair end up in for each of the four messages 00, 01, 10 and 11? Use the conversions of Section 10.5.

14.4 Teleportation uses up one entangled pair per qubit. If Alice and Bob want to teleport a register of 100 qubits, how many pairs and how many classical bits do they need?

14.5 A friend claims that teleportation proves information can travel instantly, because Bob's qubit "becomes" a scrambled copy of $|\psi\rangle$ the moment Alice measures. Write a short reply.

CHAPTER 15

Finding rhythm: the quantum Fourier transform

The algorithms of Chapter 13 revealed simple patterns in a function, such as whether its values were all equal. Shor's factoring algorithm, the subject of the next chapter, needs to detect a subtler kind of pattern: a hidden *rhythm*, a sequence of values that repeats itself at regular intervals. The tool for detecting rhythm is the Fourier transform, one of the most widely used ideas in all of science and engineering. In this chapter we meet it first in its everyday form and then in its quantum form, which can be carried out on a quantum computer exponentially faster than its classical counterpart can be carried out on a classical one.

15.1 Hearing the notes in a chord

When a pianist plays a chord, the sound that reaches your ear is a single complicated vibration of the air. Yet a trained musician can hear the individual notes in it, and so, with a little practice, can most people. The ear is doing something remarkable: it is decomposing a complicated wave into the simple, regularly repeating waves (pure tones of particular frequencies) from which it is built.

The mathematical version of this decomposition was developed by the French mathematician Joseph Fourier around 1807, while studying the flow of heat. Fourier showed that practically any signal can be written as a sum of simple repeating waves of different frequencies, and that the recipe for doing so, now called the *Fourier transform*, tells you how much of each frequency the signal contains.

Fourier transforms are everywhere. A prism splitting sunlight into a rainbow is performing one, physically: it separates light by frequency. The graphic equaliser on a music player displays one. MP3 and JPEG compression rely on close relatives of it, keeping the frequencies that matter to our ears and eyes and discarding the rest. Mobile phones, Wi-Fi, medical scanners and earthquake detectors all run Fourier transforms constantly.

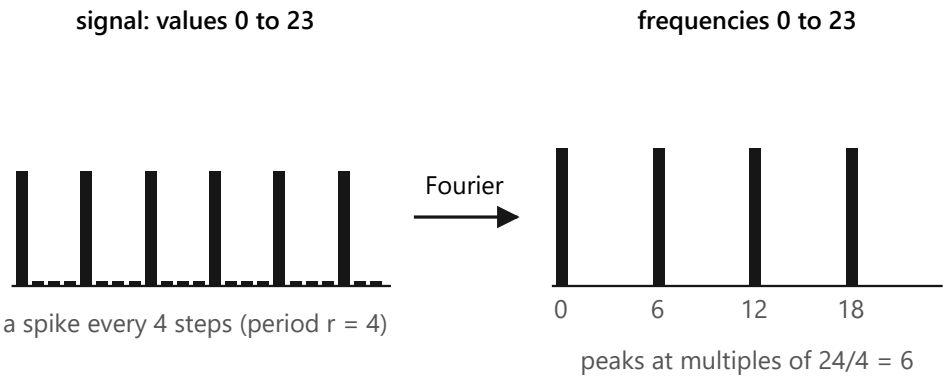


Figure 15.1. A signal of 24 values that repeats with period 4 (left) and its Fourier transform (right). All the weight of the transform sits at multiples of 24 ÷ 4 = 6. Reading off the spacing of the peaks reveals the period.

For our purposes the key property is shown in Figure 15.1. Take a sequence of N numbers that repeats with some period r. Its Fourier transform is zero everywhere except at multiples of N/r. So the Fourier transform turns a hidden period into visible, evenly spaced peaks, and from the spacing of the peaks the period can be read off.

15.2 Arrows going round a clock

To see how the Fourier transform works we return to the arrows of Chapter 3. Imagine a clock face with N positions instead of twelve. Draw a unit arrow from the centre to each position. For N = 8, the arrows point at 0°, 45°, 90°, and so on, a turn of one-eighth of a circle between each. These arrows, thought of as complex numbers, are called the N-th *roots of unity*, and they are the basic building blocks of the Fourier transform.

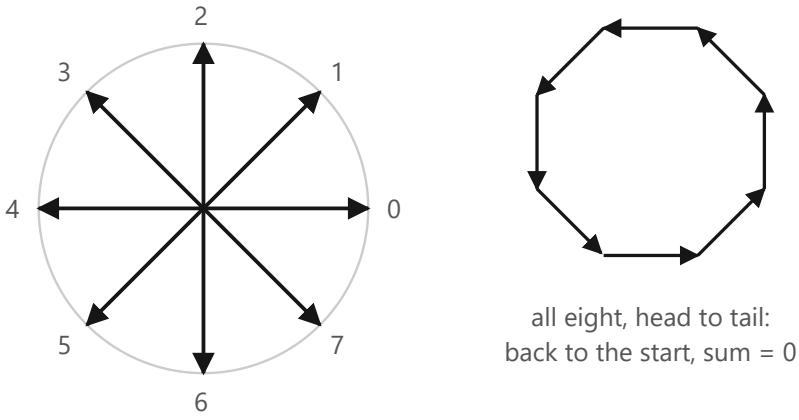


Figure 15.2. Left: the eight arrows of length 1 spaced evenly around a circle, labelled 0 to 7. Right: placed head to tail, they form a closed octagon, so their sum is zero. Evenly spread arrows always cancel completely.

The crucial fact is shown on the right of Figure 15.2. If you add up arrows that are spread evenly all the way round the circle, they cancel exactly; placed head to tail, they form a closed polygon and bring you back to where you started. If, on the other hand, the arrows all point in the same direction, they add up to a long arrow. This is destructive and constructive interference once more, in its most general form.

15.3 How the transform detects a period

The Fourier transform of a sequence of N values asks, for each frequency k from 0 to N − 1, the following question. Walk along the sequence, and at step x turn an arrow round the clock by k positions per step (so at step x it has turned k × x positions in all, wrapping round as necessary). Multiply each value of the sequence by the arrow at that step, and add everything up. The length of the result says how much of frequency k the sequence contains.

Now suppose the sequence is a spike every r steps, as in Figure 15.1, with N = 24 and r = 4. Only the spikes, at steps 0, 4, 8, 12, 16 and 20, contribute to the sum. For frequency k, the arrow at step 4j has been turned by 4jk positions on a 24-position clock.

If k = 6, each spike's arrow has turned by a multiple of 24 positions, which is a whole number of full turns, so all six arrows point in the starting direction and add up to a long arrow. The same happens for k = 12 and 18. These are the peaks.

If k is anything else, say k = 1, the spikes' arrows are at positions 0, 4, 8, 12, 16 and 20 on the 24-position clock, spread evenly all the way round, and they cancel completely. Every frequency that is not a multiple of 6 gives zero. The period has been converted into a pattern of peaks by interference.

15.4 The quantum version

In the quantum Fourier transform, the sequence is not a list of numbers stored in memory but the list of *amplitudes* of a quantum register. A register of n qubits has $N = 2^n$ amplitudes, one for each basis state $|0\rangle$, $|1\rangle$, ..., $|N-1\rangle$ (reading the bit strings as numbers, as in Box 2.1). The quantum Fourier transform, or QFT, is a quantum operation that replaces these amplitudes by their Fourier transform.

On a single basis state $|x\rangle$ it does this: it produces an equal superposition of all N basis states, in which the amplitude of $|k\rangle$ is an arrow that has been turned $x \times k$ positions round the N-position clock. In symbols,

$$\mathrm{QFT}\,|x\rangle = (1/\sqrt{N})\,(\,|0\rangle + \omega^{x}|1\rangle + \omega^{2x}|2\rangle + \ldots + \omega^{(N-1)x}|N-1\rangle\,),$$

where ω (omega) stands for the arrow one position round the clock. For one qubit, N = 2, the clock has only two positions, and ω = −1 (a half-turn). Then $\mathrm{QFT}|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ and $\mathrm{QFT}|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. The one-qubit QFT is simply the Hadamard gate. This is why the Hadamard layers of Chapter 13 were able to detect simple patterns: they are the Fourier transform's smallest relative.

Because the QFT acts on superpositions term by term, applying it to a register whose amplitudes form a periodic pattern produces a register whose amplitudes are peaked at the multiples of N/r. Measuring the register then gives one of the peaks at random, a multiple of N/r, from which the period r can usually be worked out.

15.5 An exponentially fast circuit

How many gates does the QFT need? Here we meet one of the most remarkable facts in the subject. The best classical method for computing a Fourier transform of N numbers, the *fast Fourier transform*, popularised by James Cooley and John Tukey in 1965, takes roughly N × log N steps, where log N is the number of bits needed to write N. That is already excellent and is one of the most important algorithms ever devised; without it, digital music and mobile phones would be impractical.

The quantum Fourier transform on n qubits, which transforms $N = 2^n$ amplitudes, needs only about $n^2/2$ gates. For n = 30, the classical FFT on a billion numbers needs about thirty billion steps; the QFT on 30 qubits needs a few hundred gates. The gap is exponential.

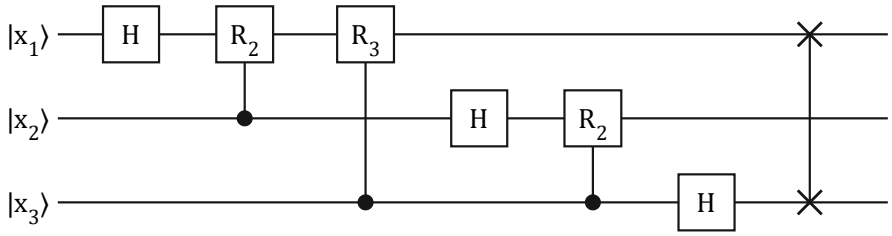


Figure 15.3. The quantum Fourier transform on three qubits. Each qubit receives a Hadamard followed by controlled phase rotations from the qubits below it; R_k turns the phase of $|1\rangle$ by $360°/2^k$. A final SWAP reverses the order of the qubits. For n qubits the same pattern uses about $n^2/2$ gates.

The circuit (Figure 15.3) has a simple, regular structure. Each qubit in turn receives a Hadamard and then a series of controlled phase rotations, each half the size of the previous one, controlled by the qubits further down. At the end, the order of the qubits is reversed with SWAP gates. The reason this works, without going into the algebra, is that the QFT of a basis state turns out to be a *product state*: each qubit ends up in its own superposition ($|0\rangle$ + some phase × $|1\rangle$)/$\sqrt{2}$, with a phase that depends on the input bits in a simple way. The Hadamards create the superpositions and the controlled rotations install the phases. The very smallest rotations contribute almost nothing and can be left out with little loss of accuracy, which reduces the gate count further.

15.6 The fine print

An exponential speed-up for the Fourier transform sounds as if it should revolutionise signal processing. It does not, and the reason is the familiar one. The quantum Fourier transform produces the transformed amplitudes inside the quantum register, where they cannot be read. Measuring the register gives a single random outcome, with probabilities given by the squared amplitudes. To recover all N transformed values we would need to repeat the experiment an enormous number of times, losing the advantage. And loading N classical numbers into the amplitudes of a register in the first place is itself generally expensive.

So the QFT is not a faster way of processing audio. It is useful in situations where the input to the transform is *created* by the quantum computer itself, typically by evaluating a function on a superposition of inputs, and where a single sample from the output, one of the peaks, is all we need. Period finding is the prime example, and Shor's algorithm is built on it.

15.7 Phase estimation

One more application of the QFT deserves mention, because it underlies many of the most important quantum algorithms. Suppose a quantum operation U leaves some particular state $|u\rangle$ unchanged apart from multiplying it by a phase: an arrow turned by some angle θ. Such a state is called an *eigenstate* of U, and the phase is its *eigenvalue*. Finding θ is a common problem in physics, because the energy levels of molecules and materials appear exactly as such phases.

The *phase estimation* algorithm, due to Kitaev in 1995, finds θ to high precision. It applies controlled versions of U, U^2, U^4, U^8 and so on to $|u\rangle$, using a register of control qubits. By phase kickback, the angle θ is written into the phases of the control register as a rhythm whose frequency is θ. An inverse QFT then converts that rhythm into a number, which is read out by measurement and gives the binary digits of θ. Shor's algorithm can be described as phase estimation applied to multiplication, and the quantum chemistry algorithms of Chapter 18 use phase estimation to compute the energies of molecules.

Chapter summary. The Fourier transform decomposes a signal into frequencies and turns a hidden period r into peaks spaced N/r apart. It works by interference: arrows turned round a clock add up when they point the same way and cancel when they are spread evenly. The quantum Fourier transform performs this transformation on the 2^n amplitudes of an n-qubit register using only about $n^2/2$ gates, exponentially fewer than the classical fast Fourier transform. Because the amplitudes cannot be read directly, the QFT is useful only when the input is prepared by the quantum computer and one sample of the output suffices, as in period finding. The one-qubit QFT is the Hadamard gate. Phase estimation uses the QFT to find the phase that an operation applies to its eigenstate.

Exercises

15.1 On a 12-position clock, add up the arrows at positions 0, 3, 6 and 9. Then add up the arrows at positions 0, 6, 0 and 6 (that is, the arrows at 0 and 6, twice each). What do you get in each case?

15.2 A sequence of N = 16 values has a spike every 4 steps. At which frequencies does its Fourier transform have peaks?

15.3 Write out QFT|x⟩ for two qubits (N = 4, so ω is a quarter-turn, which is the complex number i) for x = 1. What are the four amplitudes?

15.4 How many gates (Hadamards plus controlled rotations, not counting the final swaps) does the circuit pattern of Figure 15.3 use for 4 qubits? For 10?

15.5 Explain in your own words why the quantum Fourier transform cannot be used to speed up an MP3 encoder.

CHAPTER 16

Shor's algorithm: breaking numbers apart

In 1994 Peter Shor showed that a quantum computer could factor large numbers efficiently. More than any other single result, this turned quantum computing from a physicist's curiosity into a subject of national security briefings. In this chapter we will see why factoring matters, how a problem about prime numbers can be turned into a problem about rhythm, and how the quantum Fourier transform of Chapter 15 finds the rhythm. We will work through the whole algorithm for a small example, and end with an honest look at how far we are from running it on numbers that matter.

16.1 Why factoring matters

Every time you see a small padlock symbol next to a web address, your browser and the website have agreed on a secret key that nobody else can know, even though every message they exchanged to set it up could have been read by anyone on the network. This seemingly impossible feat is made possible by *public-key cryptography*, invented in the 1970s.

The idea can be pictured with real padlocks. Suppose I manufacture thousands of open padlocks to which only I have the key, and leave them in a basket outside my door. Anyone who wants to send me a secret puts it in a box, snaps one of my padlocks shut on it, and posts it. Snapping the padlock shut requires no key; opening it does. Even the sender cannot reopen the box. Only I can.

In the most famous public-key system, RSA (named after Ron Rivest, Adi Shamir and Leonard Adleman, who described it in 1977), the open padlock is a large number N, typically more than 600 decimal digits long, which is published for anyone to use. The key that opens the lock is knowledge of the two prime

numbers p and q whose product is N. Making N from p and q is easy: just multiply. Recovering p and q from N is, as far as anyone knows, impossibly hard for classical computers. The largest RSA-style number ever factored publicly had 250 decimal digits, in 2020, and it took a team of researchers the equivalent of about 2,700 years of computing time on a single processor core. Numbers in everyday use are far larger, and every additional digit makes the task harder.

Shor's algorithm would factor such numbers in a matter of hours or days on a sufficiently large quantum computer. RSA, and its cousins based on similar mathematics (including the elliptic-curve systems that protect much of today's internet traffic and many cryptocurrencies), would be broken.

16.2 Clock arithmetic

Shor's algorithm needs one piece of mathematics that you already use every day without calling it mathematics. If it is 10 o'clock now, what time will it be in 5 hours? Not 15 o'clock but 3 o'clock, because on a 12-hour clock we wrap around after 12. Mathematicians write this as

$$10 + 5 = 3 \pmod{12},$$

read "modulo 12". To work out any number modulo 12, divide by 12 and keep the remainder: 15 mod 12 = 3, 27 mod 12 = 3, 24 mod 12 = 0. This kind of arithmetic, called *modular arithmetic*, works for any clock size and for multiplication as well as addition. For example, 7 × 7 = 49, and 49 mod 15 = 4, since 49 = 3 × 15 + 4.

16.3 From factoring to rhythm

Here is the key idea, which was known to mathematicians before Shor. Suppose we want to factor N. Pick a number a smaller than N, more or less at random, and look at its powers modulo N:

$$a^0, a^1, a^2, a^3, \ldots \quad \text{(all mod N)}.$$

Because there are only N possible remainders, this sequence must eventually repeat, and once it returns to 1 it starts over and repeats forever with a fixed period. Let us try N = 15 and a = 7:

x	0	1	2	3	4	5	6	7	8
$7^x \bmod 15$	1	7	4	13	1	7	4	13	1

Each entry is 7 times the previous one, modulo 15: $7 \times 7 = 49 \to 4$; $4 \times 7 = 28 \to 13$; $13 \times 7 = 91 \to 1$. The sequence has period $r = 4$.

Why should the period help us factor? Because $a^r \bmod N = 1$ means that N divides $a^r - 1$. If r is even, we can write

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1),$$

a difference of two squares, which you may remember from school algebra. So N divides the product of two numbers, $a^{r/2} - 1$ and $a^{r/2} + 1$. Usually it divides neither of them alone, which means that some of N's prime factors are in one and some in the other. Then the *greatest common divisor* of N with each of the two numbers reveals a factor.

In our example, $a^{r/2} = 7^2 = 49$, which is 4 mod 15. The two numbers are $4 - 1 = 3$ and $4 + 1 = 5$. The greatest common divisor of 3 and 15 is 3; that of 5 and 15 is 5. And indeed $15 = 3 \times 5$.

Sometimes this fails: r might be odd, or one of the two numbers might already be a multiple of N. Then we simply choose another a and try again. It can be proved that for a randomly chosen a, the method succeeds at least half of the time, so a few attempts are almost always enough.

Box 16.1: Euclid's algorithm

The greatest common divisor of two numbers is the largest number that divides both. Computing it is easy, even for numbers with thousands of digits, thanks to a method described by Euclid around 300 BC. To find the greatest common divisor of two numbers, repeatedly replace the larger by its remainder when divided by the smaller, until one of them becomes zero; the other is the answer. For 1071 and 462: 1071 mod 462 = 147; 462 mod 147 = 21; 147 mod 21 = 0. So the answer is 21. Euclid's algorithm is one of the oldest algorithms still in daily use, and every step of Shor's method except the period finding is performed with it and other fast classical tools.

So factoring reduces to period finding. The catch is that for a number N with hundreds of digits, the period r is typically itself a number with hundreds of digits. Finding it by computing the powers one by one until the sequence repeats would take far longer than the age of the universe. No efficient classical method for finding the period is known. This is exactly where the quantum computer comes in.

16.4 Quantum period finding

Shor's algorithm uses two registers of qubits. The first, the *input* register, will hold the exponent x. The second, the *output* register, will hold the value $a^x \bmod N$. The steps are the ones Chapter 13 trained us to expect.

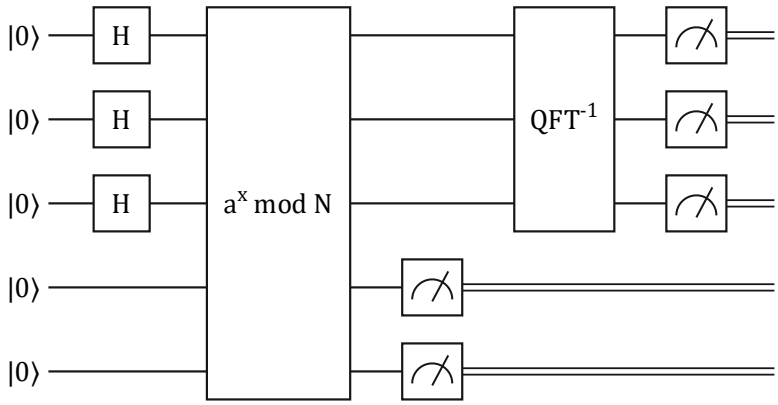


Figure 16.1. The structure of Shor's algorithm (drawn with only three input qubits and two output qubits; real instances need thousands). Hadamards create a superposition of all exponents x. A modular exponentiation circuit computes $a^x \bmod N$ into the output register. Measuring the output register is optional but helps to picture what happens. An inverse quantum Fourier transform on the input register converts the hidden period into peaks, and a final measurement samples one of them.

Step 1: superposition. Apply Hadamards to every qubit of the input register, producing an equal superposition of all exponents from 0 up to Q − 1, where $Q = 2^n$ for n input qubits. (Q is chosen to be somewhat larger than N^2, for reasons that will appear shortly.)

Step 2: compute. Apply a circuit that computes $a^x \bmod N$ into the output register. This is ordinary arithmetic, built from reversible gates as described in Chapter 11, and it is by far the most expensive part of the algorithm in terms of gate count. Because it acts on the superposition term by term, the result is the entangled state

$$(1/\sqrt{Q})\,(\,|0\rangle|a^0 \bmod N\rangle + |1\rangle|a^1 \bmod N\rangle + |2\rangle|a^2 \bmod N\rangle + \ldots\,).$$

Step 3: measure the output (optional). Suppose we now measure the output register. We get some value, say v, at random. By the rule of Section 9.5, the input register collapses to the superposition of all exponents x for which $a^x \bmod N = v$. Because the sequence is periodic, these exponents are evenly spaced, r apart: something like x_0, $x_0 + r$, $x_0 + 2r$, and so on. The input register now holds a periodic pattern, a spike every r steps, exactly like the signal in Figure 15.1. The starting offset x_0 is random, so measuring the input register now would tell us nothing useful. But the *spacing* of the spikes is the period we want.

(It turns out that this measurement is not actually necessary; the algorithm works just as well without it, because the output register's entanglement with the input has the same effect. But imagining the measurement makes it much easier to see what is going on.)

Step 4: Fourier transform. Apply the inverse quantum Fourier transform to the input register. (The inverse QFT turns the arrows the other way round the clock; for our purposes it behaves just like the QFT.) As we saw in Chapter 15, it turns a sequence with period r into peaks at multiples of Q/r. The random offset x_0 only affects the phases of the peaks, not their positions, so it does not matter.

Step 5: measure and deduce. Measure the input register. The result is, with high probability, close to a multiple of Q/r, say a number k with k/Q close to j/r for some whole number j. A classical calculation, using a technique called *continued fractions* that finds the simplest fraction close to a given number, recovers r from k/Q. This is why Q is chosen to be larger than N^2: it makes the peaks sharp enough that the simplest nearby fraction is reliably the right one.

Step 6: factor. With r in hand, compute $a^{r/2} \pm 1$ and use Euclid's algorithm to find factors of N, as in Section 16.3. If anything went wrong (r odd, unlucky measurement), repeat with a new a.

16.5 A small example

Let us run through the algorithm for N = 15 and a = 7, with an input register of four qubits, so Q = 16. (For such a small N we can get away with a smaller Q than N^2, because the period happens to divide Q exactly.)

After steps 1 and 2, the state pairs each x from 0 to 15 with 7^x mod 15, which runs through 1, 7, 4, 13, 1, 7, 4, 13, ... four times.

In step 3, suppose we measure the output and get 4. The exponents with 7^x mod 15 = 4 are x = 2, 6, 10 and 14. The input register collapses to

$$(|2\rangle + |6\rangle + |10\rangle + |14\rangle) / 2.$$

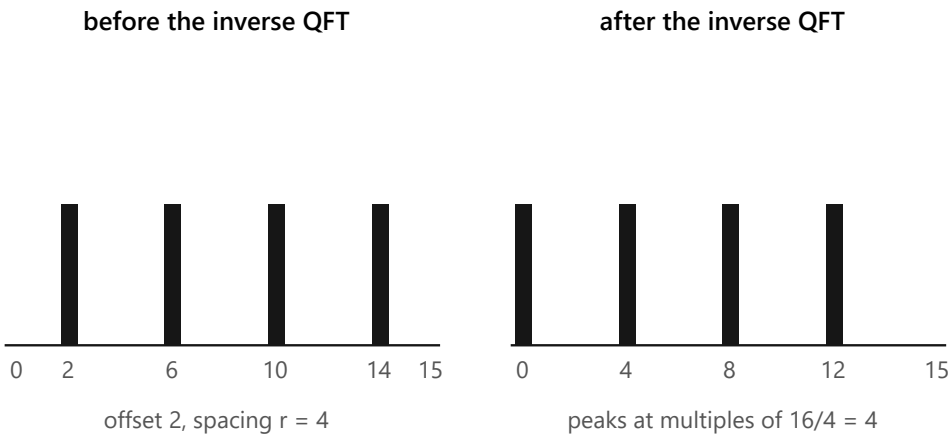


Figure 16.2. Shor's algorithm for N = 15, a = 7, with a four-qubit input register. Left: the probabilities of the input register after the output has been measured as 4; the spikes are at 2, 6, 10 and 14. Right: after the inverse QFT, the probability sits on 0, 4, 8 and 12, the multiples of Q/r = 16/4. The random offset has disappeared.

In step 4, the inverse QFT turns this into a superposition of |0⟩, |4⟩, |8⟩ and |12⟩, each with probability 1/4 (the offset of 2 shows up only as phases on these four terms).

In step 5 we measure. Suppose we get 12. Then k/Q = 12/16 = 3/4, and the denominator suggests r = 4. We check: 7^4 mod 15 = 1. Correct. If instead we had got 4, we would have 4/16 = 1/4, again giving r = 4. If we had got 8, we would have

8/16 = 1/2, suggesting r = 2, which fails the check (7^2 mod 15 = 4, not 1); we would then simply run the algorithm again. If we had got 0, we would learn nothing and run again.

In step 6, with r = 4, we compute 7^2 mod 15 = 4, and the greatest common divisors of 3 and 5 with 15 give the factors 3 and 5.

Of course, nobody needs a quantum computer to factor 15. But every step of this example works in the same way for numbers with hundreds of digits, and the number of gates needed grows only as a modest power of the number of digits. Shor's algorithm needs roughly n^3 gates to factor an n-bit number, compared with the best classical methods, whose running time grows faster than any power of n.

16.6 How close are we?

In 2001 a team at IBM factored 15 using Shor's algorithm on a seven-qubit device built from molecules in a test tube, controlled with the techniques of nuclear magnetic resonance. It was a landmark demonstration, but the experiment was simplified using prior knowledge of the answer, and so have most of the small demonstrations that followed. More than two decades later, no quantum computer has run Shor's algorithm in full on a number that a schoolchild could not factor in their head.

The reason is error. Factoring a 2048-bit RSA number requires billions of quantum gates, applied to thousands of logical qubits, with essentially no errors along the way. Today's qubits make an error roughly once every few hundred to few thousand operations. As Chapter 21 will explain, this can be overcome with quantum error correction, which combines many noisy physical qubits into one reliable logical qubit, but the overhead is large. Published estimates of what it would take to break RSA-2048 have fallen steadily as algorithms and error correction have improved: from around a billion physical qubits in estimates of the early 2010s, to about twenty million in an influential 2019 analysis by Craig Gidney and Martin Ekerå, to under a million noisy qubits running for about a week in a 2025 estimate by Gidney. The largest machines built so far have on the order of a thousand physical qubits, and their error rates and error-correction capabilities are not yet anywhere near what this task requires.

No one can say with confidence when the gap will close. What is clear is that the threat is taken seriously. Messages encrypted today could be recorded now and decrypted later, once a large quantum computer exists, an attack known as "harvest now, decrypt later". For that reason, in 2024 the United States National Institute of Standards and Technology published the first standards for *post-quantum cryptography*: new public-key systems based on mathematical problems, such as finding short vectors in high-dimensional lattices, for which no efficient quantum algorithm is known. Browsers, messaging apps and operating systems have already begun switching to them. In a sense, Shor's algorithm has changed the world's cryptography before ever being run.

Chapter summary. RSA and related public-key systems are secure only because factoring large numbers is hard for classical computers. Factoring N reduces to finding the period r of the sequence a^x mod N: if r is even, the greatest common divisors of N with $a^{r/2} \pm 1$ usually reveal factors. Shor's algorithm finds the period with a quantum computer by preparing a superposition of exponents, computing a^x mod N on all of them at once, applying an inverse QFT to turn the period into peaks, and measuring; a classical continued-fraction calculation then extracts r. The algorithm needs roughly n^3 gates for an n-bit number. Running it on cryptographically relevant numbers requires error-corrected machines far larger than any built so far, but the threat has already driven the adoption of post-quantum cryptography.

Exercises

16.1 Compute 2^x mod 15 for x = 0 to 6. What is the period? Use it to find the factors of 15.

16.2 Compute 14^x mod 15 for x = 0 to 4. What is the period r? What goes wrong when you try to use it? (Compute $a^{r/2} + 1$ and see what its greatest common divisor with 15 is.)

16.3 Use Euclid's algorithm to find the greatest common divisor of 221 and 91. What does that tell you about the factors of 221?

16.4 In the example of Section 16.5, suppose the output register had been measured as 13 instead of 4. Which exponents x would the input register then hold? Would the peaks after the inverse QFT be in different places?

16.5 Find the factors of 21 by hand using the method of Section 16.3 with a = 2. (Hint: compute 2^x mod 21 until you get back to 1.)

CHAPTER 17

Grover's algorithm: a needle in a haystack

We began this book in a library with no catalogue, searching for a single slip of paper among a trillion books. In 1996 Lov Grover, a researcher at Bell Labs, found that a quantum computer could search such a library in far fewer steps than any classical machine: roughly the square root of the number of books, a million steps instead of a trillion. Grover's algorithm is less dramatic than Shor's, since its speed-up is not exponential. But it applies to an enormous range of problems, it is easy to picture, and it introduces a technique, *amplitude amplification*, that has become one of the standard tools of quantum algorithm design.

17.1 The search problem

Suppose there are N possible items, numbered 0 to N − 1, and exactly one of them is "marked". We have a test that, given an item, tells us whether it is the marked one; we can think of it as a function f with f(x) = 1 for the marked item and f(x) = 0 for every other. We know nothing else: there is no ordering, no pattern, no hint. The task is to find the marked item using as few tests as possible.

Many real problems have this shape, at least approximately. Finding a combination of settings that satisfies a long list of constraints, finding the input to a cryptographic function that produces a given output, finding a schedule with no clashes: in each case it is easy to *check* a proposed answer but there may be no better way of *finding* one than to try candidates.

Classically, with no structure to exploit, there is nothing to do but test items one by one. On average this takes N/2 tests, and in the worst case N. A quantum computer can do it with about $(\pi/4)\sqrt{N}$ tests. For a million items, that is about 785 instead of half a million.

17.2 The idea: grow the right amplitude

The quantum computer will hold the items in a register of n qubits, with $N = 2^n$. We start, as usual, with a Hadamard on every qubit, so that the register is in an equal superposition of all N items. Each item has amplitude $1/\sqrt{N}$. If we measured now, we would get each item with probability $1/N$, which is just a random guess.

Grover's algorithm repeats two operations, in alternation, that gradually move amplitude from the unmarked items onto the marked one. After the right number of repetitions, almost all of the amplitude sits on the marked item, and a measurement finds it with high probability.

Operation 1: mark. Use the test, in the form of an oracle with phase kickback (Section 13.2), to flip the sign of the marked item's amplitude. All other amplitudes are left alone. After this step the marked item has amplitude $-1/\sqrt{N}$ and the others still have $+1/\sqrt{N}$. Measuring now would still give a random item, since squaring removes the sign. But the marked item has been tagged, invisibly, in a way that interference can detect.

Operation 2: reflect about the average. Compute the average of all the amplitudes. Then replace each amplitude by its mirror image on the other side of the average: an amplitude that was a little above the average becomes a little below it, and an amplitude that was far below the average becomes far above it. In symbols, each amplitude a becomes $2m - a$, where m is the average. This operation is called *inversion about the mean*, and it can be carried out with a short circuit, as we will see below.

Let us see what this does with $N = 8$. Initially each amplitude is $1/\sqrt{8} \approx 0.354$.

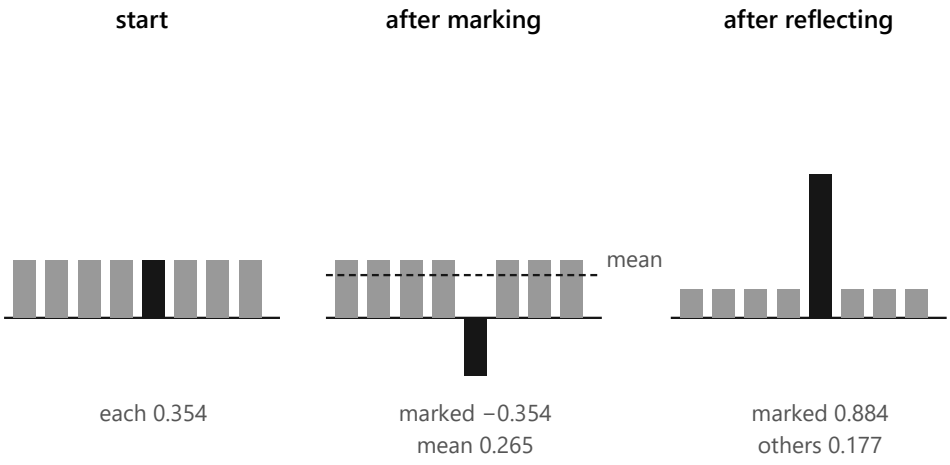


Figure 17.1. One round of Grover's algorithm with eight items (amplitudes drawn as bars; the black bar is the marked item). Marking flips the sign of its amplitude, pulling the average (dashed) down slightly. Reflecting every amplitude about the average lifts the marked item far above the rest.

After marking, the amplitudes are seven times 0.354 and once −0.354. Their average is $(7 \times 0.354 - 0.354)/8 \approx 0.265$. Reflecting about this average, each unmarked amplitude becomes $2 \times 0.265 - 0.354 \approx 0.177$, and the marked amplitude becomes $2 \times 0.265 + 0.354 \approx 0.884$. The probability of finding the marked item has jumped from $1/8 = 12.5\%$ to $0.884^2 \approx 78\%$.

Do it once more. Marking makes the marked amplitude −0.884. The new average is $(7 \times 0.177 - 0.884)/8 \approx 0.044$. Reflecting gives unmarked amplitudes of about $2 \times 0.044 - 0.177 \approx -0.088$ and a marked amplitude of about $2 \times 0.044 + 0.884 \approx 0.972$. The probability of success is now $0.972^2 \approx 94.5\%$. Two rounds, two uses of the test, and the marked item is found with near certainty. A classical search would need, on average, more than four tests.

17.3 A rotation in a plane

There is a beautiful geometric way to understand why this works and how many rounds are needed. At every stage of the algorithm, all of the unmarked items have the same amplitude as each other. So the state is always a combination of just two states: the marked item $|w\rangle$ (for "winner"), and the equal superposition of all the

unmarked items, which we will call |r⟩ (for "rest"). These two states are perpendicular arrows, and the state of the register is a unit arrow in the plane they span.

The starting state, the equal superposition of all N items, lies very close to |r⟩, because nearly all of its weight is on unmarked items. It makes a small angle θ with |r⟩, where $\sin\theta = 1/\sqrt{N}$.

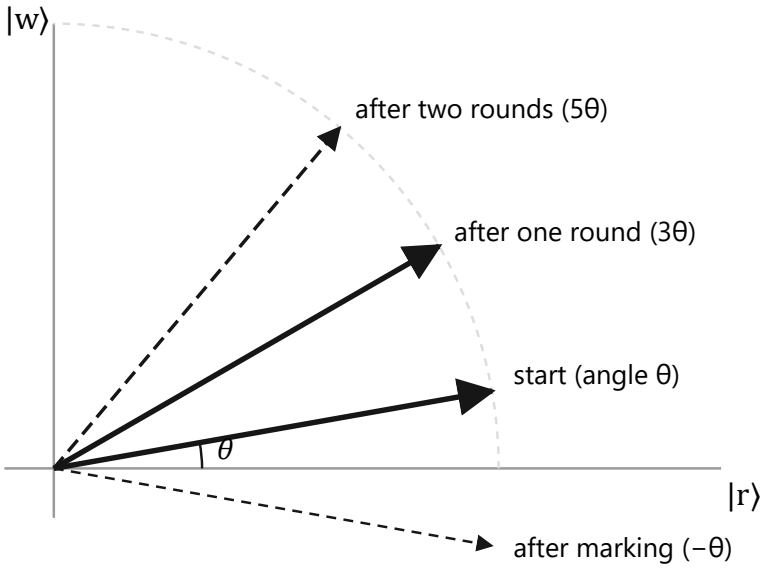


Figure 17.2. Grover's algorithm as a rotation, drawn for a small N so that the angles are visible. The state starts at a small angle θ above |r⟩. Marking reflects it to angle −θ, below |r⟩. Reflecting about the average reflects it back across the starting direction, to angle 3θ. Each round turns the state by 2θ towards the marked item |w⟩.

Now look at what the two operations do in this plane. Marking flips the sign of the |w⟩ component, which is a reflection across the |r⟩ axis: the arrow at angle θ goes to angle −θ. Reflecting about the average turns out to be a reflection across the direction of the *starting state*: the arrow at −θ, which is 2θ below the starting direction, goes to 2θ above it, at angle 3θ. Two reflections make a rotation, and each round rotates the state by 2θ towards |w⟩.

After k rounds the state is at angle (2k + 1)θ. We want it to point as nearly as possible along |w⟩, at 90°. For large N, the angle θ is approximately $1/\sqrt{N}$ (in the unit of angle mathematicians call radians, where a right angle is π/2), so the number of rounds needed is about

$$k \approx (\pi/4)\sqrt{N}.$$

This also explains a curious feature of the algorithm: running it too long makes things *worse*. If we keep rotating past $|w\rangle$, the state swings away again, and the probability of success falls. Unlike most search methods, where more effort never hurts, Grover's algorithm has to be stopped at the right moment, like a pendulum that must be caught at the top of its swing.

17.4 The circuit

Both operations of a Grover round can be built from the gates we know.

The marking step is the oracle for f, used with an extra qubit in the state $|-\rangle$ so that phase kickback turns "f(x) = 1" into a sign flip, exactly as in Chapter 13. In practice the oracle is a reversible circuit that evaluates whatever test defines the search problem, cleaned up with Bennett's trick.

The reflection about the average is called the *diffusion* operator. It can be built from three layers: Hadamards on every qubit, then an operation that flips the sign of every basis state *except* $|00...0\rangle$, then Hadamards again. The Hadamards translate between the item basis and a basis in which the equal superposition is simply $|00...0\rangle$; in that basis, reflecting about the starting direction just means flipping the sign of everything that is not $|00...0\rangle$.

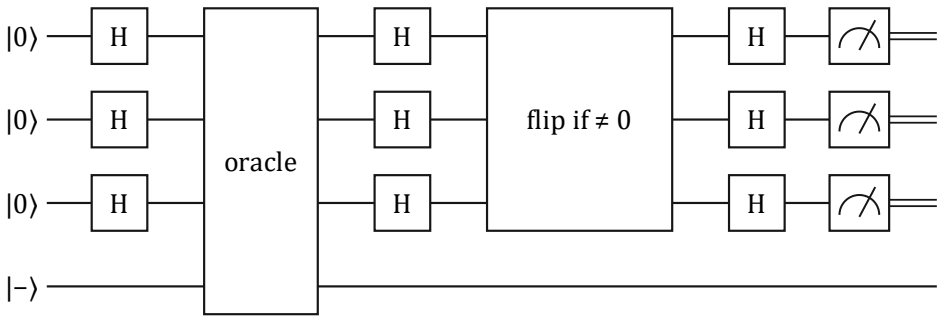


Figure 17.3. One round of Grover's algorithm on three qubits (eight items). After the initial Hadamards, the oracle marks the winning item by flipping its sign (via an extra qubit in $|-\rangle$). The diffusion step, consisting of Hadamards, a sign flip on every state except $|000\rangle$, and Hadamards, reflects all amplitudes about their average. For larger searches, the oracle and diffusion steps are repeated about $(\pi/4)\sqrt{N}$ times before measuring.

17.5 How good is it?

Grover's algorithm gives a *quadratic* speed-up: $\sqrt{N}$ steps instead of N. That is a substantial gain for large N, but much less dramatic than the exponential speed-up of Shor's algorithm. If a classical search takes 10^{12} steps, Grover's takes about 10^{6}; if the classical search takes 2^{100} steps, Grover's still takes 2^{50}, which remains utterly infeasible.

Could a cleverer quantum algorithm do better? No. In 1997, Charles Bennett, Ethan Bernstein, Gilles Brassard and Umesh Vazirani proved that any quantum algorithm for unstructured search must use the oracle at least on the order of $\sqrt{N}$ times. Grover's algorithm is optimal. This is an important result in its own right, because it shows that quantum computers cannot simply try all possibilities at once and pick the right one. If they could, search would take a single step. For problems with no structure, the best a quantum computer can do is a square-root improvement.

There are also practical caveats. Each oracle call might be an expensive circuit, much slower on a quantum computer than the equivalent test on a classical one. The rounds must be performed one after another, so the algorithm cannot be spread across many machines as easily as a classical search. And on an error-corrected quantum computer, each logical operation may be thousands of times slower than a classical operation. Detailed analyses suggest that for many search problems, a quadratic speed-up will only beat classical hardware once problems are very large indeed, and possibly only on much better quantum hardware than is now foreseeable. Grover's algorithm is a theoretical landmark and a versatile building block, but it is not by itself a reason to expect a near-term revolution.

Box 17.1: Grover and cryptography

Grover's algorithm can search for a secret key: try every possible key and mark the one that decrypts a known message correctly. Against a symmetric cipher with a 128-bit key, a classical brute-force attack needs about 2^{128} trials, while Grover's algorithm needs about 2^{64} rounds. That sounds like a serious weakening. The usual response is simply to double the key length: with 256-bit keys, Grover's algorithm needs 2^{128} rounds, as many as a classical attack on a 128-bit key. Modern standards such as AES-256 already offer such key lengths, so symmetric encryption is considered relatively safe from quantum attack. It is public-key cryptography, threatened by Shor's algorithm, that needs replacing.

17.6 Amplitude amplification

The idea behind Grover's algorithm is more general than search. Suppose we have *any* quantum procedure that produces a good answer with some small probability p, and a way of recognising good answers. Classically, we would have to repeat the procedure about 1/p times to find a good answer. Using the same pair of reflections, one marking good answers and one reflecting about the procedure's output state, we can boost the probability of success to nearly 1 in about $1/\sqrt{p}$ rounds. This technique, called *amplitude amplification*, was worked out by Brassard, Peter Høyer, Michele Mosca and Alain Tapp around 2000. It can be bolted onto many quantum algorithms to speed them up quadratically, and related techniques allow a quantum computer to *count* the number of marked items, or estimate a probability, quadratically faster than random sampling could.

Chapter summary. Grover's algorithm searches N unstructured items for a marked one using about $(\pi/4)\sqrt{N}$ oracle calls, compared with about $N/2$ classically. Starting from an equal superposition, it repeats two steps: the oracle flips the sign of the marked item's amplitude, and the diffusion operator reflects all amplitudes about their average, which transfers amplitude to the marked item. Geometrically, each round rotates the state by a small angle towards the marked item, so the algorithm must stop at the right time. The quadratic speed-up is provably optimal for unstructured search, showing that quantum computers cannot try all possibilities at once. Practical gains are limited by oracle cost and error-correction overhead. Amplitude amplification generalises the method to boost the success probability of other quantum procedures.

Exercises

17.1 Repeat the calculation of Section 17.2 for $N = 4$. Show that after one round the marked item has amplitude 1, so it is found with certainty.

17.2 For $N = 8$, carry out a third round, starting from the amplitudes at the end of Section 17.2 (-0.088 for each unmarked item, 0.972 for the marked one). What is the probability of success now? Compare with your answer after two rounds.

17.3 About how many rounds does Grover's algorithm need for $N = 10^6$? For $N = 10^{12}$?

17.4 Suppose there are M marked items among N instead of one. Guess how the number of rounds should change, using the geometric picture. (In fact $\sin\theta = \sqrt{(M/N)}$.)

17.5 A news article claims that Grover's algorithm allows a quantum computer to "instantly search the entire internet". List two reasons why this is wrong.

CHAPTER 18

Simulating nature

Feynman's original motivation for quantum computers, as we saw in Chapter 1, was not factoring or searching but simulation: using one quantum system to predict the behaviour of another. Four decades later, many researchers still believe that this will be the first truly valuable application of quantum computers, and possibly the most important one. This chapter explains why simulating molecules and materials is so hard for classical computers, how a quantum computer would go about it, and what has to happen before it pays off.

18.1 Why chemistry is hard

Chemistry is, at bottom, the physics of electrons. Whether two atoms bond, how strongly a drug sticks to a protein, which way a reaction goes and how fast, what colour a dye is, whether a material conducts electricity: all of these are decided by the behaviour of the electrons in the molecules involved. And electrons are quantum objects. To predict what they do, we need to know the quantum state of all of them together.

That is where the trouble starts. Each electron in a molecule can occupy various *orbitals*, regions around the nuclei where it is likely to be found. A useful description of a molecule might allow, say, 100 orbitals, each of which can be empty or occupied. The state of the electrons is then a superposition over all the ways of filling those orbitals, and there can easily be more such ways than 2^{100}. We met this explosion in Chapter 9: describing such a state on a classical computer would need more numbers than there are atoms in the universe.

Chemists are resourceful, and over the past century they have developed powerful approximate methods that sidestep the explosion for many molecules. The most widely used, *density functional theory*, replaces the full quantum state

with a much simpler quantity and works remarkably well for a broad range of problems; its developers were awarded the Nobel Prize in Chemistry in 1998. But the approximations fail precisely where things get interesting: in molecules where several electrons are strongly coupled to each other, so that the state cannot be broken into simpler pieces. Such *strongly correlated* systems include many catalysts, the active sites of important enzymes, certain magnetic materials, and high-temperature superconductors.

Box 18.1: A molecule worth simulating

Plants need nitrogen to grow, and the air is nearly 80% nitrogen gas. But the two atoms of a nitrogen molecule are held together by one of the strongest bonds in chemistry, and breaking it to make fertiliser is hard. Industry does it with the Haber–Bosch process, which combines nitrogen and hydrogen at about 450°C and hundreds of times atmospheric pressure. It feeds roughly half of the world's population, and it consumes on the order of one to two percent of all the energy humanity uses.

Certain bacteria do the same job at room temperature, using an enzyme called nitrogenase. Its active site contains a cluster of iron, molybdenum and sulfur atoms, known by its abbreviation FeMoco, whose electrons are strongly correlated. Despite decades of research, nobody fully understands how it works, in part because its electronic structure is beyond the reach of reliable classical calculation. FeMoco has become a standard benchmark for quantum chemistry on quantum computers. Understanding it might, one day, point the way to gentler ways of making fertiliser.

18.2 How quantum states change in time

So far in this book, quantum states have changed only when we applied gates to them. In nature, states change continuously, all by themselves. A molecule's electrons, left alone, evolve smoothly from one superposition to another, with their amplitudes rotating at rates set by the energies involved.

The rule governing this evolution is the *Schrödinger equation*, found by Erwin Schrödinger in 1926. We will not write it down, but its content can be stated in words. Every physical system has a quantity called its *Hamiltonian*, named after the Irish mathematician William Rowan Hamilton, which describes all the ways its energy depends on its state: how strongly the electrons repel each other, how strongly they are attracted to the nuclei, how they move between orbitals. The

Schrödinger equation says that, over each short interval of time, the state is rotated by an amount determined by the Hamiltonian. Since rotations are exactly what quantum gates do, the time evolution of any quantum system is, in principle, a quantum circuit.

A quantum computer simulates nature by building that circuit: it encodes the state of the molecule in its qubits, and then applies gates that rotate the qubits in the same way that the molecule's Hamiltonian would rotate the molecule.

18.3 Small steps

There is a complication. The Hamiltonian of a molecule is a sum of many pieces, each describing one interaction: this electron with that nucleus, that pair of electrons with each other, and so on. Each piece on its own corresponds to a simple rotation that is easy to build from a few gates. But the pieces do not commute (Box 3.1): applying them one after another in different orders gives different results. So we cannot simply apply each piece's full rotation once and be done.

The solution, often named after the mathematician Hale Trotter, is to break time into many small steps, and in each step apply a small rotation for each piece in turn. When the steps are small enough, the error caused by applying the pieces one at a time instead of all together becomes negligible.

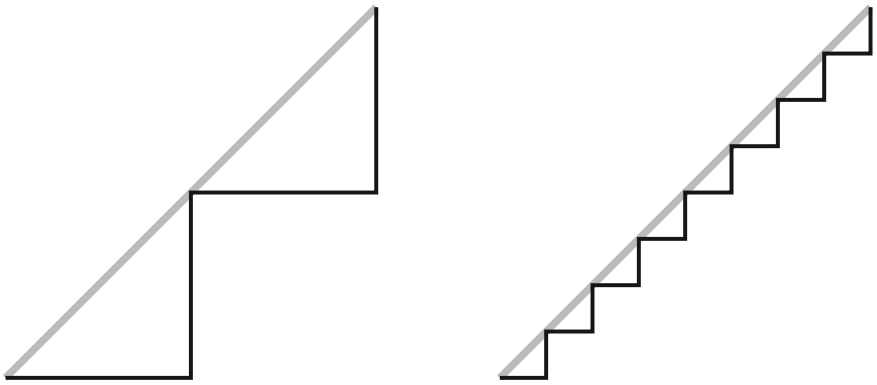


Figure 18.1. An analogy for Trotter's method. To travel diagonally (grey) using only east and north moves, alternate between them. With a few large steps the path strays far from the diagonal; with many small steps it stays close. In a quantum simulation, the "east" and "north" moves are the rotations for different parts of the Hamiltonian.

Figure 18.1 shows the idea with an analogy. If you can only walk east or north, you can still travel diagonally by alternating short steps in each direction. The shorter the steps, the closer you stay to the diagonal. More refined schemes, which arrange the steps more cleverly, achieve the same accuracy with fewer gates, and much research has gone into reducing the cost of simulation in this way.

18.4 From molecules to qubits

To run a simulation, the molecule's state must be written in terms of qubits. The most natural way, for electrons, is to use one qubit per orbital: $|1\rangle$ means the orbital is occupied and $|0\rangle$ means it is empty. A molecule described with 100 orbitals then needs 100 qubits to hold its electronic state. That is the crucial point: the number of qubits grows only in proportion to the size of the description, while the number of amplitudes grows exponentially. A quantum computer with a few hundred good qubits could hold states that no classical computer could ever store.

There is a subtlety that makes electrons more awkward than they look. Electrons are *fermions*, particles that obey a strict rule, the Pauli exclusion principle, under which swapping two electrons flips the sign of the state. Qubits do not naturally behave that way, so the mapping has to keep track of these signs.

Standard techniques, such as one devised by Pascual Jordan and Eugene Wigner in 1928, long before anyone imagined quantum computers, handle this bookkeeping at the cost of some extra gates.

18.5 Finding the energy

Often the most important thing a chemist wants to know about a molecule is not how it evolves in time but what its *energy levels* are, in particular the lowest one, the *ground state energy*. Differences between ground state energies of different arrangements of atoms tell us which reactions release energy, how strongly molecules bind, and how fast reactions go.

On a large, error-corrected quantum computer, the natural tool is the phase estimation algorithm of Section 15.7. If the qubits are prepared in (or close to) the ground state, then simulating the molecule's time evolution simply multiplies the state by a phase, which turns at a rate proportional to the energy. Phase estimation reads off that rate, and hence the energy, to high precision. This approach is expected to give definitive answers for molecules like FeMoco, but it needs long, deep circuits and therefore full error correction.

For the noisy, smaller machines available today, researchers have developed a hybrid approach called the *variational quantum eigensolver*, or VQE, proposed by Alberto Peruzzo and colleagues in 2014. Here a short quantum circuit with adjustable rotation angles prepares a trial state, the quantum computer measures its energy, and a classical computer adjusts the angles to try to lower the energy, round and round, like a person tuning an old radio by ear.

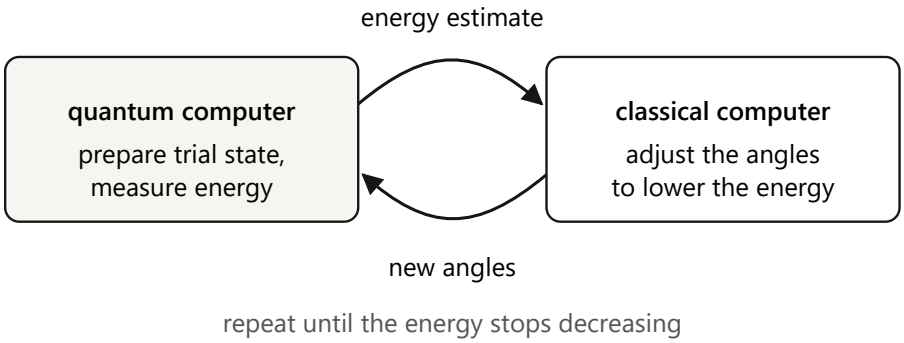


Figure 18.2. The variational quantum eigensolver. A quantum computer prepares a trial state using a short circuit with adjustable angles and estimates its energy by repeated measurement; a classical optimiser chooses new angles. The loop continues until the energy no longer goes down, giving an estimate of the ground state energy.

VQE and similar variational methods have been demonstrated on small molecules, such as hydrogen and lithium hydride, on many kinds of hardware. They are useful as testbeds and have taught researchers a great deal. But they also have serious limitations. The energy has to be estimated by repeating measurements an enormous number of times, the classical optimisation can get stuck, and for the molecules studied so far classical methods give answers at least as good. Whether variational methods will ever beat classical chemistry on a problem that matters is an open question.

18.6 Where things stand

Simulation is the application for which the case for quantum computers is strongest, for a simple reason: the problem is itself quantum mechanical, and the explosion of amplitudes is not a feature of a clever algorithm but of nature. Even so, an honest assessment has to include some caveats.

Classical methods keep improving, and claims of quantum advantage have repeatedly been answered by better classical algorithms that reproduced the quantum results. The molecules for which a quantum computer would clearly help are the strongly correlated ones, and identifying which practically important problems truly need such accuracy is itself a research question. And the resource

estimates for definitive calculations on molecules like FeMoco, while they have fallen considerably as algorithms have improved, still call for error-corrected machines with on the order of a million physical qubits or more.

Beyond chemistry, quantum simulation is likely to find uses in materials science (designing better batteries, magnets or superconductors), in nuclear and particle physics, and in the study of exotic phases of matter. Some of the most convincing experiments so far have been performed not on general-purpose quantum computers but on *analogue quantum simulators*: arrays of atoms or ions arranged to mimic a particular model of a material, which then simply evolve and are measured. This is Feynman's water-flume idea in its purest form.

Box 18.2: What about optimisation and machine learning?

Quantum computers are often said to be about to transform logistics, finance and artificial intelligence. The reality is more cautious. For optimisation problems, such as finding the best route for delivery vans, there are quantum heuristics (for example the *quantum approximate optimisation algorithm*, proposed in 2014), but no convincing evidence yet that they outperform the best classical methods on real problems. Grover-type speed-ups are quadratic at best. For machine learning, several proposed exponential speed-ups turned out, on closer inspection, to rely on assumptions about loading data that made the comparison unfair, and classical algorithms inspired by the quantum ones (a process called *dequantisation*, pioneered by Ewin Tang in 2018) matched them. Research continues, and surprises are possible, but simulation of quantum systems and the breaking of certain cryptosystems remain the applications with the strongest theoretical foundations.

Chapter summary. Predicting the behaviour of molecules requires the quantum state of their electrons, which involves exponentially many amplitudes; classical approximations work well in many cases but fail for strongly correlated systems such as some catalysts and enzymes. Quantum states evolve according to the Schrödinger equation, which rotates the state in a way set by the Hamiltonian, so time evolution can be built as a quantum circuit, using many small Trotter steps. Molecules are encoded with roughly one qubit per orbital, with extra bookkeeping for fermion signs. Ground state energies can be found by phase estimation on error-corrected machines, or approximately by variational hybrid methods on today's noisy ones. Simulation is the most promising application, but classical methods are strong and useful chemistry will likely need large error-corrected computers.

Exercises

18.1 A molecule is described using 50 orbitals. How many qubits would the encoding of Section 18.4 need? How many amplitudes could the state of those qubits involve, and how much classical memory would they need at 16 bytes each? (Use the table in Section 9.3.)

18.2 In the analogy of Figure 18.1, the greatest distance between the staircase and the diagonal is proportional to the step size. If doubling the number of steps halves the error, how many steps are needed to make the error 1,000 times smaller than with a single step?

18.3 Explain in your own words why simulating a molecule is a more natural task for a quantum computer than searching a database.

18.4 In a VQE calculation, each energy estimate is obtained by repeating a measurement many times. If the statistical error falls like one over the square root of the number of repetitions (as for coin tosses), how many more repetitions are needed to make the error ten times smaller?

Part IV: Information, noise and machines

Secret keys guaranteed by physics, the enemy of every qubit, the clever defence against it, and the real machines being built today.

CHAPTER 19

Quantum cryptography

Shor's algorithm threatens the cryptography that protects today's communications. In a pleasing twist, quantum mechanics also offers a remedy: a way of creating secret keys whose security rests not on the assumed difficulty of a mathematical problem but on the laws of physics themselves. The idea is older than quantum computing as we know it, it uses only single qubits and simple measurements, and it has been a commercial product for some years. It is also the most direct application of the strange rules of measurement we met in Chapter 6.

19.1 The one-time pad

There is one encryption method that has been known for over a century to be perfectly unbreakable. It is called the *one-time pad*. Alice and Bob share a secret key: a long string of random bits, as long as the message. To encrypt, Alice XORs each bit of her message with the corresponding bit of the key. To decrypt, Bob XORs the ciphertext with the same key, which undoes the encryption, since XORing twice with the same bit gives back the original.

message	**1 0 1 1 0 0 1 0**
key	0 1 1 0 1 0 0 1
ciphertext (message XOR key)	1 1 0 1 1 0 1 1

To an eavesdropper who does not know the key, the ciphertext is a string of random bits. Every possible message of the right length is equally consistent with it. No amount of computing power, classical or quantum, can extract any information, as Claude Shannon proved in the 1940s. The one-time pad is not merely hard to break; it is impossible to break.

The difficulty is the key. It must be truly random, as long as all the messages that will ever be sent with it, never reused, and shared in advance between Alice and Bob without anyone else seeing it. During the Cold War, keys for diplomatic hotlines were carried by trusted couriers in locked briefcases. For ordinary internet traffic that is out of the question, which is why public-key cryptography, with all its mathematical assumptions, took over.

Quantum key distribution, or QKD, solves the key-sharing problem. It lets Alice and Bob create a shared random key over a public channel, and it guarantees that if anyone has been listening, they will find out.

19.2 Two bases, again

The method was invented in 1984 by Charles Bennett and Gilles Brassard and is known as BB84. It builds on an earlier idea of Stephen Wiesner's, written around 1970 but rejected by journals and only published in 1983, about using two incompatible bases to make banknotes that could not be counterfeited.

BB84 relies on a fact from Section 6.4. A qubit prepared in the 0/1 basis, as $|0\rangle$ or $|1\rangle$, gives a definite answer when measured in that basis but a completely random one when measured in the diagonal basis. A qubit prepared in the diagonal basis, as $|+\rangle$ or $|-\rangle$, behaves the other way round. If you do not know which basis a qubit was prepared in, you cannot measure it reliably, and if you guess the basis wrong, your measurement destroys the original state.

In practice the qubits are usually photons, and the two bases are two different orientations of polarisation: horizontal and vertical for one basis, the two diagonals for the other. For the description we will simply call them the "+" basis (0/1) and the "×" basis (diagonal).

19.3 The protocol

Step 1. Alice generates two long strings of random bits. For each position, the first string gives a *bit value* and the second a *basis choice*. She prepares one photon per position: if the basis choice is +, she sends $|0\rangle$ for bit value 0 and $|1\rangle$ for bit value 1; if the basis choice is ×, she sends $|+\rangle$ for 0 and $|-\rangle$ for 1. She sends the photons to Bob.

Step 2. Bob, who does not know Alice's basis choices, picks a basis at random for each photon and measures in it. He records his results.

Step 3. Over an ordinary public channel, such as the internet, Alice and Bob announce which *basis* they used for each photon, but not the bit values. They keep only the positions where they happened to choose the same basis (about half of them) and discard the rest. This step is called *sifting*.

In the positions they keep, Bob measured in the same basis that Alice prepared in, so his result equals her bit value. They now share a string of random bits that has never been sent over the public channel in readable form.

photon	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Alice's bit	0	1	1	0	1	0	0	1
Alice's basis	+	×	+	+	×	×	+	×
photon sent	$\|0\rangle$	$\|-\rangle$	$\|1\rangle$	$\|0\rangle$	$\|-\rangle$	$\|+\rangle$	$\|0\rangle$	$\|-\rangle$
Bob's basis	+	+	+	×	×	+	+	×
Bob's result	0	0 or 1	1	0 or 1	1	0 or 1	0	1
same basis?	yes	no	yes	no	yes	no	yes	yes
key bit	0		1		1		0	1

In the table, "0 or 1" means that Bob's result is random, because he measured in the wrong basis; those positions are discarded.

Step 4. Alice and Bob check for eavesdropping. They choose a random sample of their key bits, announce them publicly, and compare. If there has been no interference, the sample bits should agree perfectly (apart from occasional errors caused by imperfect equipment). They then discard the announced bits, since they are no longer secret, and keep the rest.

19.4 Catching the eavesdropper

Suppose an eavesdropper, traditionally called Eve, intercepts the photons on their way to Bob. She wants to learn the bit values, so she must measure the photons; but she does not know Alice's basis choices, which are announced only later. She can do no better than guess a basis for each photon, measure, and send Bob a new photon prepared according to her result, hoping he will not notice.

Half of the time Eve guesses the right basis. Then she learns the bit value and sends Bob an exact replacement, and nothing goes wrong. The other half of the time she guesses wrong. Her measurement gives a random result, and the photon she sends on is prepared in the wrong basis. When Bob then measures in Alice's basis (in the positions that survive sifting), he gets a random result, which disagrees with Alice's bit half of the time.

So, for each key bit that Eve tampered with, there is a probability of $1/2 \times 1/2 = 1/4$ that Bob's bit differs from Alice's. If Eve intercepts every photon, about a quarter of the sifted key will be wrong. When Alice and Bob compare a sample of a few hundred bits, an error rate of 25% will stand out unmistakably. They will know someone was listening, throw the key away and try again, perhaps over a different channel.

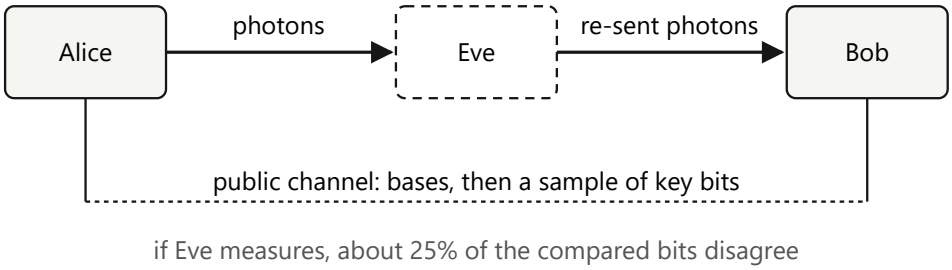


Figure 19.1. Eavesdropping on BB84. Photons travel through a channel that Eve may tap; the basis announcements and the check of sample bits travel over a public channel, which Eve can read but not alter. Any attempt to measure the photons introduces detectable errors.

Could Eve be cleverer? She could intercept only a fraction of the photons, introducing fewer errors but also learning less. She could try a measurement in some intermediate basis. She could try to copy each photon and measure the copy later, after the bases are announced; but the no-cloning theorem of Chapter 12 forbids exactly this. In every case, the amount of information Eve gains is tied to the amount of disturbance she causes. This trade-off is a direct consequence of the rules of quantum measurement, and it has been made mathematically precise: full security proofs of BB84 against every attack allowed by quantum mechanics were completed around 2000, by Dominic Mayers, by Hoi-Kwong Lo and H. F. Chau, and by Peter Shor and John Preskill.

19.5 From a noisy key to a secret key

Real equipment is never perfect, so Alice and Bob always see some errors, even without an eavesdropper, and they cannot tell which errors are innocent. The protocol therefore ends with two classical steps. In *error correction*, Alice and Bob exchange some carefully chosen information about their keys, such as the parity of blocks of bits, to find and fix the disagreements. In *privacy amplification*, they compress the corrected key into a shorter one, mixing the bits together with a public random function, so that whatever partial information Eve might have about the long key becomes almost no information about the short one. The security proofs say exactly how much shorter the key must be made for a given error rate. As long as the error rate is below about 11%, a secret key can still be distilled.

19.6 Entanglement-based QKD

In 1991 Artur Ekert proposed a different approach, now called E91, which uses entangled pairs. A source sends one photon of each pair to Alice and the other to Bob, and each measures in a randomly chosen basis. When their bases match, the entanglement guarantees that their results are correlated, giving key bits. On the other positions, they check whether their results violate a Bell inequality, as in Chapter 12. A violation certifies that the photons really were entangled, and since entanglement cannot be shared with a third party beyond a certain limit (a property called the *monogamy of entanglement*), it also certifies that Eve cannot hold much information about the key.

This idea leads to the remarkable possibility of *device-independent* QKD, in which Alice and Bob need not trust the inner workings of their own equipment at all, since the Bell violation alone proves security. Experimental demonstrations of device-independent QKD were reported in 2022, though at very low key rates.

19.7 Quantum cryptography in practice

Commercial QKD systems have been on sale since the early 2000s, and QKD networks have been built in several countries. A network in China links Beijing and Shanghai over about 2,000 kilometres of fibre via a chain of relay stations, and the Micius satellite demonstrated QKD between ground stations on different continents in 2017.

The main practical limitation is distance. Photons in an optical fibre are gradually absorbed, and after a few hundred kilometres too few arrive for a useful key rate. Classical signals are boosted by amplifiers, but a quantum signal cannot be amplified, because amplifying it would amount to copying it. Today's long-distance networks therefore rely on "trusted nodes", relay stations where the key is decoded and re-encrypted, which must themselves be kept secure. The long-term solution is the *quantum repeater*, a device that uses entanglement and teleportation to extend quantum links without measuring the qubits along the way, but practical repeaters do not yet exist.

There is also an important gap between the ideal protocol and real hardware. Security proofs assume idealised devices, and real ones have quirks: detectors that can be blinded by bright light, sources that occasionally emit two photons instead of one. Researchers have demonstrated attacks on commercial systems that exploit such quirks, and countermeasures have been developed in turn. For this reason, security agencies in several countries currently recommend post-quantum cryptography (Section 16.6) rather than QKD for most purposes, regarding QKD as a promising technology for specialised high-security links rather than a general replacement.

Chapter summary. The one-time pad is perfectly secure but needs a random shared key as long as the message. Quantum key distribution creates such a key over a public channel. In BB84, Alice sends single photons prepared in randomly chosen bases, Bob measures in randomly chosen bases, and they keep the bits where the bases matched. An eavesdropper who measures the photons must guess the bases, and her wrong guesses cause errors in about a quarter of the key bits she touches, which Alice and Bob detect by comparing a sample. Error correction and privacy amplification turn a noisy key into a secret one. Entanglement-based protocols can use Bell tests to certify security, even with untrusted devices. Practical QKD is limited by distance and by imperfections in real hardware.

Exercises

19.1 Using the one-time pad, decrypt the ciphertext 0110 with the key 1100. Why must a one-time pad key never be reused? (Hint: what is the XOR of two ciphertexts encrypted with the same key?)

19.2 In the table of Section 19.3, how many of the eight photons survived sifting? On average, what fraction survive?

19.3 Eve intercepts every photon. Alice and Bob compare a sample of 4 key bits. What is the probability that all 4 agree, so that Eve goes undetected? What if they compare 100 bits?

19.4 Eve intercepts only one photon in ten. What error rate would Alice and Bob see in the sifted key?

19.5 Explain why a classical amplifier cannot be used to boost a quantum signal in a fibre, in terms of a theorem from Chapter 12.

CHAPTER 20

Noise: the enemy of every qubit

Everything in Parts II and III assumed perfect qubits: gates that do exactly what they should, states that stay put between gates, and measurements that never lie. Real qubits are nothing like that. They are delicate physical objects, and the world around them is constantly jostling them. This chapter is about what goes wrong, and why it goes wrong so easily. It is the bad news that makes the good news of Chapter 21 so remarkable.

20.1 Every leak is a measurement

In Section 6.6 we noted that a measurement does not need a physicist. Whenever information about a qubit's state leaks into its surroundings, the effect on the qubit is the same as if someone had measured it. The leaked information need not be read by anyone; it is enough that it exists somewhere, in principle, recorded in a stray photon or the motion of a nearby atom.

Think back to the interferometer of Chapter 4. Interference disappeared as soon as there was any record, anywhere, of which route the photon took. Now imagine that the photon's two routes pass through slightly different air, so that an air molecule is nudged a little differently depending on the route. That nudged molecule is a record of the route. It is a very faint record, so it destroys the interference only partially. But the photon passes through billions of molecules, each of which can keep a faint record, and together they can wipe the interference out completely.

This process is called *decoherence*. The superposition is not destroyed so much as smeared out: the qubit becomes entangled with its environment, and the delicate phase relations that made interference possible are spread across countless particles that we have no hope of gathering back together. From the point of view of the qubit alone, its pure state becomes a mixed state, a point inside

the Bloch sphere rather than on its surface (Section 7.6). It is exactly what happened to Schrödinger's cat in Box 5.1. The cat interacts so strongly with its surroundings that any superposition of alive and dead would decohere in an unimaginably short time.

For a quantum computer, decoherence is the central problem. A qubit must be isolated well enough that its environment does not keep records of its state, and yet it must be accessible enough that we can apply gates to it and, at the end, measure it. These two demands pull in opposite directions.

20.2 Two ways to forget

Physicists describe decoherence in real qubits with two numbers, both times.

The first, called T_1 or the *relaxation time*, describes how long a qubit takes to lose energy. In most qubits, $|1\rangle$ has more energy than $|0\rangle$, and a qubit in $|1\rangle$ will sooner or later drop to $|0\rangle$, giving its excess energy to the environment, for example by emitting a photon. On the Bloch sphere, relaxation drags every state towards the north pole.

The second, called T_2 or the *dephasing time*, describes how long the qubit keeps track of its relative phase. Suppose the energy difference between $|0\rangle$ and $|1\rangle$ fluctuates slightly, because of a wobbling magnetic field or a stray electric charge nearby. Then the relative phase between the amplitudes turns slightly faster or slower than it should, in a way we cannot predict. On the Bloch sphere, the state drifts randomly around the equator. After a while we no longer know where on the equator it is, and our best description of the qubit is an average over all the places it might be, which lies on the axis, inside the sphere.

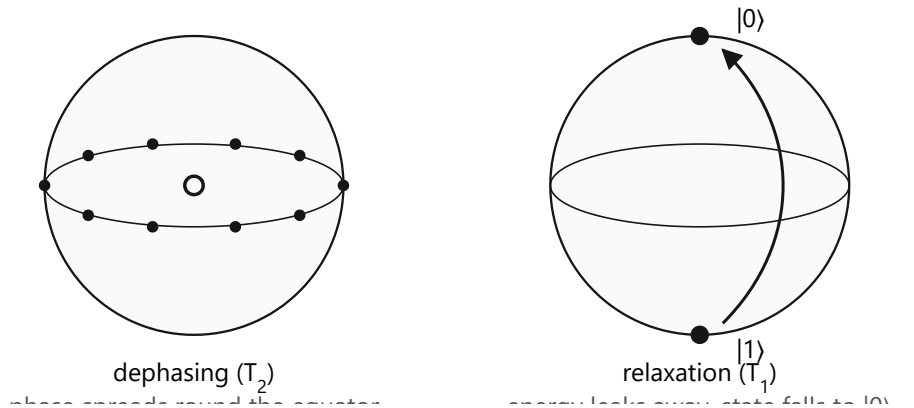


Figure 20.1. The two kinds of decoherence on the Bloch sphere. Left: dephasing. A state prepared on the equator drifts randomly around it (dots show where it might be after some time); averaged over the possibilities, it ends up at the centre (circle). Right: relaxation. A state decays from |1⟩ towards |0⟩ as the qubit loses energy to its surroundings.

An analogy for dephasing is a group of runners who start a race together around a circular track. At first they run as a tight pack, and if you know where one is, you know where they all are. But each runs at a slightly different pace, and after a few laps they are spread all around the track. The pack has "dephased". In a quantum computer, the "runners" are the different possible histories of the qubit's phase, and once they have spread out, the interference that depended on their being together is lost.

Dephasing usually happens faster than relaxation, because it does not require the qubit to give up any energy; a gentle, random wobble in the environment is enough.

20.3 Errors from our own hands

Not all errors come from the environment. Many come from the imperfect way we control the qubits.

A gate is applied by a pulse of microwaves, laser light or voltage, and if the pulse is slightly too strong, too weak or too long, the rotation it produces will be slightly too large or too small. A Hadamard that is meant to turn the Bloch sphere by exactly 180° might turn it by 179.5°. Such *control errors* are small for each gate, but they accumulate over the thousands of gates in a computation, and unlike random noise they can add up systematically in the same direction.

Two-qubit gates are much harder to get right than single-qubit gates, because the two qubits must be made to interact strongly for a brief, precisely timed moment and then left alone. They typically have error rates ten times higher than single-qubit gates. There are also *crosstalk* errors, in which a gate applied to one qubit slightly disturbs its neighbours; *leakage*, in which a qubit escapes from its two working levels into some other level (recall the grey third level of the atom in Figure 5.1); and *measurement errors*, in which the readout reports 0 when the qubit was 1, or vice versa.

20.4 How good are qubits today?

Comparing qubit technologies is complicated, and the figures change every year, but a few orders of magnitude give a sense of the situation as of the mid-2020s.

	superconducting circuits	**trapped ions**	**neutral atoms**
coherence time	0.1–1 ms	seconds or longer	seconds
two-qubit gate time	tens of ns	tens to hundreds of μs	about 1 μs
two-qubit gate error (best)	about 0.1–0.5%	about 0.1% or better	about 0.5%

(One ms is a thousandth of a second, one μs a millionth and one ns a billionth.) What matters is not the coherence time on its own but how many gates can be performed before the qubit decoheres, and above all the error rate per gate. With an error rate of 0.1% per two-qubit gate, roughly one gate in a thousand goes wrong.

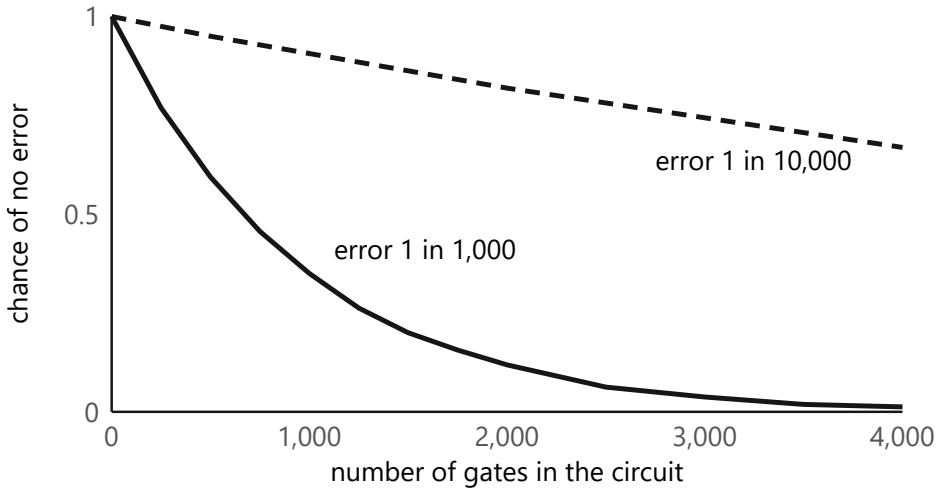


Figure 20.2. The probability that a circuit runs without a single error, as a function of the number of gates, for error rates of 1 in 1,000 (solid) and 1 in 10,000 (dashed) per gate. Even with excellent gates, long computations are almost certain to contain errors.

If each gate fails independently with probability p, the chance that a circuit of G gates runs without any error is $(1 - p)^G$. With p = 0.001 and G = 1,000, that is about 37%. With G = 10,000 it is about 0.005%. Shor's algorithm for a cryptographically relevant number needs billions of gates; the probability of running it without error on unprotected qubits is, for all practical purposes, zero. And it is not realistic to expect physical error rates to fall to one in a billion, because the environment can never be shut out completely.

20.5 The noisy era

In 2018 the physicist John Preskill coined the term *NISQ*, for *Noisy Intermediate-Scale Quantum*, to describe the machines then emerging: tens to hundreds of qubits, too few and too noisy for error correction, but too many to simulate easily on classical computers. Much research in the years since has gone into finding out whether such machines can do anything useful on their own.

One line of work is *error mitigation*: rather than preventing errors, measure their effect and subtract it statistically. For example, one can deliberately make the noise worse by a known factor, run the circuit at several noise levels, and

extrapolate the results back to zero noise. Such tricks can improve the accuracy of short computations, but their cost in repeated runs grows exponentially with the size of the circuit, so they cannot rescue long computations.

Another is the search for *quantum advantage*: a task, however artificial, on which a noisy quantum computer demonstrably beats the best classical computers. In 2019 a team at Google reported that their 53-qubit processor Sycamore had sampled from the output of random circuits in about 200 seconds, a task they estimated would take a top supercomputer ten thousand years. Classical researchers responded with much faster simulation methods, cutting that estimate dramatically, and a lively back-and-forth followed, with larger and better quantum experiments and cleverer classical algorithms on each side. These experiments were landmarks in controlling large quantum systems, but the tasks have no practical use, and the debate showed how difficult it is to establish an advantage that holds up.

The broad consensus today is that the most valuable applications of quantum computing, such as Shor's algorithm and accurate chemistry, will require error correction. That is the subject of the next chapter.

20.6 Why not just build better qubits?

Engineers are, of course, building better qubits, and error rates have fallen steadily for two decades. But there is a fundamental limit to how far this can go. A qubit that is perfectly isolated from its environment cannot be controlled, because control is itself an interaction with the outside world. The control lines that deliver gate pulses also carry noise; the measurement apparatus that reads the qubit at the end can disturb it in the middle. Every qubit technology involves trade-offs between isolation and control, and no known technology offers error rates anywhere near the one-in-a-billion level that long algorithms need.

For many years this seemed a fatal objection to quantum computing. Classical computers face noise too, but they deal with it by constantly measuring and resetting bits: a voltage that drifts slightly towards the middle is snapped back to a clean 0 or 1 at every step. Quantum states cannot be treated like that, because measuring them destroys them. And the obvious alternative, copying information so that a damaged copy can be outvoted by the others, is forbidden by the no-

cloning theorem. In 1995 several prominent physicists published arguments that quantum computation would never survive noise. They were proved wrong, as we will see next.

Chapter summary. Decoherence occurs when information about a qubit's state leaks into its environment, which acts like an unintended measurement and turns pure states into mixed ones. It has two main forms: relaxation (T_1), in which the qubit loses energy and decays to $|0\rangle$, and dephasing (T_2), in which the relative phase drifts randomly. Further errors come from imperfect control, crosstalk, leakage and faulty measurement; two-qubit gates are the most error-prone. With today's best error rates of around one in a thousand per gate, circuits of more than a few thousand gates almost certainly fail. Noisy intermediate-scale machines have achieved impressive demonstrations, but valuable applications require error correction, which must work without measuring or copying the protected information.

Exercises

20.1 A qubit has a coherence time of 100 μs and a two-qubit gate takes 50 ns. Roughly how many gates can be performed within the coherence time?

20.2 Using $(1 - p)^G \approx e^{-pG}$ for small p, estimate the probability of no error for a circuit of 500 gates with $p = 0.002$. ($e^{-1} \approx 0.37$.)

20.3 On the Bloch sphere, where does the state $|+\rangle$ end up after complete dephasing? After complete relaxation? Where does $|1\rangle$ end up after complete dephasing?

20.4 Explain the difference between a control error and decoherence, and why control errors may be more harmful in long circuits even if they are smaller per gate.

CHAPTER 21

Protecting quantum information

At the end of the last chapter we were in a bind. Real qubits make errors far too often for long computations. Classical computers fix errors by copying information and checking it, but copying a quantum state is forbidden, and checking it by measurement destroys it. On top of that, quantum errors are continuous: a qubit can be rotated by 179.5° instead of 180°, or by any other amount, so there seem to be infinitely many different errors to worry about.

In 1995 Peter Shor found a way around all three obstacles at once, and within a couple of years a general theory of *quantum error correction* had been developed by Shor, Andrew Steane, Raymond Laflamme, Emanuel Knill, Daniel Gottesman, Alexei Kitaev and many others. It is one of the deepest and most surprising ideas in the subject, and it is the reason that large-scale quantum computing is regarded as possible at all. In this chapter we build up to it in small steps, starting from the simplest classical code.

21.1 A classical warm-up

Suppose you need to send a single bit over a noisy telephone line that flips each bit with probability p, say 1%. The simplest protection is to repeat yourself: send 000 for 0 and 111 for 1. The receiver takes a majority vote. If one of the three bits is flipped, say 000 arrives as 010, the majority still says 0 and the error is corrected. Only if two or more bits are flipped does the vote go wrong.

The chance of two or more flips out of three is about $3p^2$ (three ways to choose which two bits flip, each with probability p^2), which for p = 1% is 0.03%, more than thirty times better than sending the bit once. That is the essence of error

correction: spread the information across several carriers, so that a few errors can be detected and undone. The price is redundancy; here, three bits are sent for every one bit of information.

21.2 Encoding without copying

The naive quantum analogue would be to turn $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ into three copies $|\psi\rangle|\psi\rangle|\psi\rangle$. That is impossible, by no-cloning. But there is something else we *can* do. Use two CNOT gates, with the original qubit as control and two fresh qubits in $|0\rangle$ as targets:

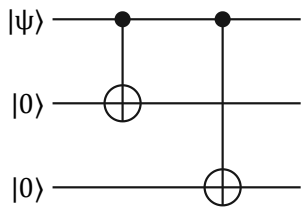


Figure 21.1. Encoding one qubit into three for the bit-flip code. The CNOTs copy the basis-state values, not the state itself, producing $\alpha|000\rangle + \beta|111\rangle$.

Acting term by term, the CNOTs turn $\alpha|0\rangle|00\rangle + \beta|1\rangle|00\rangle$ into

$$\alpha|000\rangle + \beta|111\rangle.$$

This is not three copies of $|\psi\rangle$. (Three copies would be the product state $|\psi\rangle|\psi\rangle|\psi\rangle$, with eight terms when multiplied out.) It is a single entangled state of three qubits that stores the two amplitudes α and β, once each, in a redundant way: the value 0 is represented by 000 and the value 1 by 111. We say that the three physical qubits now hold one *logical qubit*, with logical basis states $|0_L\rangle = |000\rangle$ and $|1_L\rangle = |111\rangle$.

21.3 Asking the right question

Now suppose one of the three qubits suffers a bit flip, say the middle one. The state becomes

$$\alpha|010\rangle + \beta|101\rangle.$$

How can we find and fix the error? We must not measure the qubits individually. That would collapse the superposition and destroy α and β. The trick is to ask a less nosy question. Instead of asking "what is the value of each qubit?" we ask "do qubits 1 and 2 agree?" and "do qubits 2 and 3 agree?". These are called *parity checks*.

Look at what the questions reveal. In the state $\alpha|010\rangle + \beta|101\rangle$, qubits 1 and 2 disagree in *both* terms (0 versus 1 in the first, 1 versus 0 in the second). Qubits 2 and 3 also disagree in both terms. So both answers are definite, "disagree" and "disagree", and the measurement collapses nothing: both terms of the superposition give the same answers, so neither is singled out. The answers reveal *where the error is* without revealing anything about α and β.

The pair of answers is called the *error syndrome*, and it identifies the culprit:

qubits 1 and 2	**qubits 2 and 3**	**diagnosis**	**correction**
agree	agree	no error	none
disagree	agree	qubit 1 flipped	X on qubit 1
disagree	disagree	qubit 2 flipped	X on qubit 2
agree	disagree	qubit 3 flipped	X on qubit 3

The parity checks are carried out with the help of extra qubits, called *ancillas*. To check whether qubits 1 and 2 agree, an ancilla starting in $|0\rangle$ receives a CNOT from each of them; if they agree, the ancilla is flipped twice or not at all and ends in $|0\rangle$; if they disagree, it is flipped once and ends in $|1\rangle$. Measuring the ancilla gives the answer. Only the ancilla is measured, and it holds only the parity, not the data.

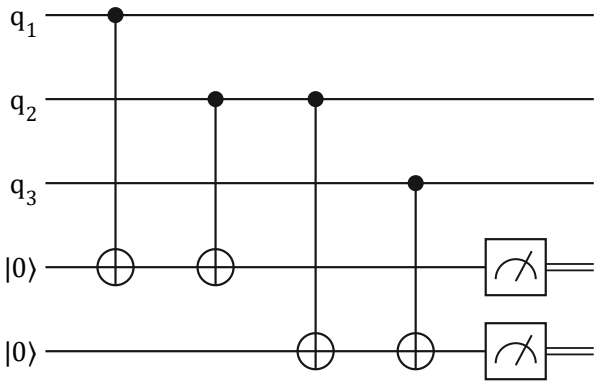


Figure 21.2. Measuring the error syndrome of the bit-flip code. The first ancilla (fourth line) records whether q_1 and q_2 agree; the second records whether q_2 and q_3 agree. Measuring the ancillas reveals the syndrome without disturbing the encoded amplitudes.

21.4 Continuous errors become discrete

Now for the most surprising part. Suppose the error on qubit 2 is not a full bit flip but a small unwanted rotation, a slight tilt of its state on the Bloch sphere. A small rotation can be written as a superposition: mostly "nothing happened" plus a little bit of "a bit flip happened". The damaged state is therefore a superposition of the undamaged code state and the code state with qubit 2 flipped.

When we measure the syndrome, the measurement must give one of the definite answers in the table. With high probability it reports "no error", and the state collapses onto the undamaged part of the superposition: the small error has simply vanished. With small probability it reports "qubit 2 flipped", the state collapses onto the flipped part, and we correct it with an X gate. Either way, we end up with the correct encoded state.

The act of measuring the syndrome *forces* the error to decide: either it happened fully, in which case we know about it and can undo it, or it did not happen at all. Infinitely many possible continuous errors are reduced to a handful of discrete ones. This *digitisation of errors* is what makes quantum error correction possible, and it is a lovely example of the measurement postulate, usually a nuisance, working in our favour.

21.5 Phase flips too

The bit-flip code has a weakness: it does nothing against phase flips. A Z error on any of the three qubits turns $\alpha|000\rangle + \beta|111\rangle$ into $\alpha|000\rangle - \beta|111\rangle$, which is a perfectly valid code state, just the wrong one, and the parity checks cannot see it.

But recall from Chapter 8 that phase flips are simply bit flips in the diagonal basis: Z swaps $|+\rangle$ and $|-\rangle$ just as X swaps $|0\rangle$ and $|1\rangle$. So a code that encodes $\alpha|+++\rangle + \beta|---\rangle$ instead, with parity checks done in the diagonal basis, protects against phase flips (but not bit flips). Shor's 1995 code combines the two ideas in layers. First the qubit is encoded against phase flips into three qubits; then each of those three is encoded against bit flips into three more. The result uses nine physical qubits per logical qubit and can correct any single error, bit flip, phase flip or both, on any one of the nine.

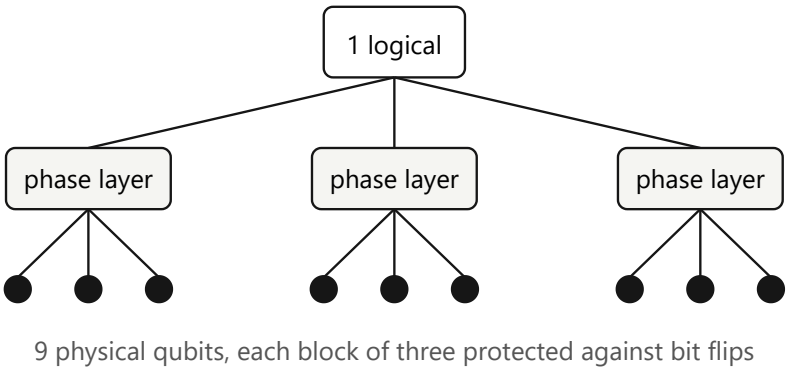


Figure 21.3. The structure of Shor's nine-qubit code. One logical qubit is spread over three blocks to protect against phase flips, and each block is spread over three physical qubits to protect against bit flips.

Since any single-qubit error can be written as a combination of "no error", a bit flip, a phase flip and both together, and since syndrome measurement digitises errors as described above, a code that corrects bit flips and phase flips on any one qubit corrects *every* possible error on that qubit. This was the breakthrough: quantum information can be protected against arbitrary noise, as long as the noise does not hit too many qubits at once.

21.6 The threshold theorem

Error correction is itself carried out by gates and measurements, which are also noisy. Could the correction process introduce more errors than it removes? It could, if done carelessly. An error in one of the CNOTs of Figure 21.2, for instance, could spread to several data qubits at once. The art of *fault-tolerant* design is to arrange every step of the computation, including the error correction itself, so that a single fault can only ever cause a correctable amount of damage.

With fault-tolerant design, a remarkable result holds, proved independently by several groups around 1996–97 and known as the *threshold theorem*. There is a critical error rate, the *threshold*, such that if every physical component fails with probability below the threshold, then by using larger and larger codes the error rate of the logical qubits can be made as small as we like, while the overhead in qubits and gates grows only modestly (as a power of the logarithm of the computation's size). Below threshold, adding more qubits makes things better. Above threshold, it makes things worse.

The threshold theorem changed the question facing experimentalists. It was no longer "can qubits be made perfect?", which they cannot, but "can qubits be made good enough to get below the threshold?", which is a matter of engineering.

21.7 The surface code

The early codes, like Shor's, had thresholds that were frustratingly low and required operations between qubits far apart. The code that most experimental groups are now pursuing is the *surface code*, which grew out of work by Alexei Kitaev in 1997 and was developed further by many researchers. Its great advantages are that it needs only a flat, two-dimensional grid of qubits with interactions between nearest neighbours, which suits chip-based hardware, and that its threshold is high, around 1% per operation, within reach of today's best devices.

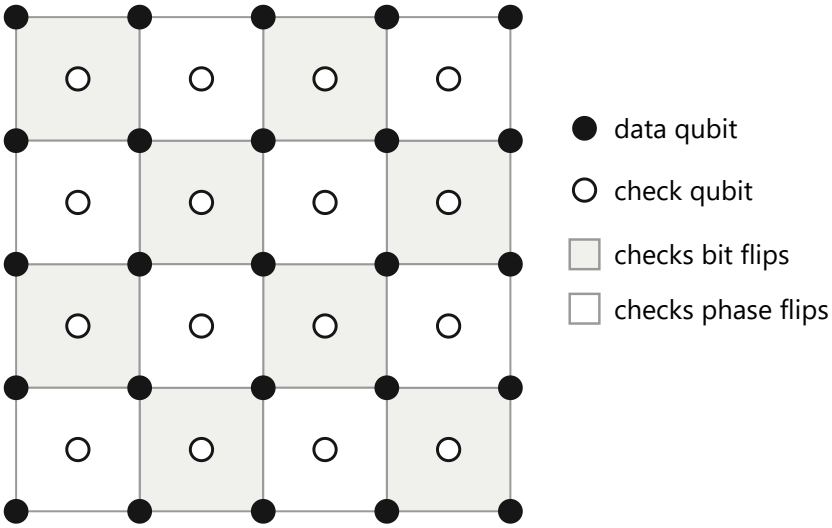


Figure 21.4. A simplified picture of a surface code patch. Data qubits (black) sit at the corners of a checkerboard; a check qubit (white) in the middle of each square repeatedly measures the parity of its four neighbours, shaded squares checking for bit flips and unshaded ones for phase flips. The pattern of checks that report a change reveals where errors have occurred. (Real layouts differ in details at the edges.)

In the surface code, data qubits sit on a checkerboard grid, with a check qubit in the middle of each square. Each check qubit repeatedly measures the parity of its four neighbours, some of the squares checking for bit flips and the others for phase flips. An error on a data qubit changes the results of the checks next to it, lighting them up like a burglar alarm. A classical computer watches the pattern of alarms in real time and works out the most likely set of errors that would explain it, a task called *decoding*. One logical qubit is stored in the whole patch, in a way that cannot be damaged by any small cluster of errors: to corrupt it, errors must form a chain stretching all the way across the patch.

The size of the patch is measured by its *distance* d, the number of data qubits along one side. A patch of distance d can correct any pattern of up to about d/2 errors. Below threshold, every increase of d by 2 divides the logical error rate by a constant factor. In late 2024, a team at Google reported exactly this behaviour on their *Willow* processor: going from distance 3 to 5 to 7, the logical error rate fell by roughly a factor of two at each step, with the distance-7 logical qubit outliving the

best of its physical qubits. It was the first convincing demonstration of a surface code operating below threshold, and a milestone the field had pursued for decades.

The cost is high. To bring the logical error rate down to the level needed for something like Shor's algorithm, patches of distance 25 or more may be needed, each using on the order of a thousand physical qubits for a single logical qubit. A useful fault-tolerant quantum computer with a few thousand logical qubits would therefore need around a million physical ones. Much current research aims to reduce this overhead, through better physical qubits, cleverer codes (several families of codes promise many logical qubits per patch, at the price of longer-range connections) and better decoders.

Box 21.1: Magic states

Some gates are easy to perform on encoded qubits. In the surface code, the Pauli gates, the Hadamard and the CNOT can all be carried out reliably on logical qubits. Unfortunately these "easy" gates are not universal on their own; in fact, circuits made only from them can be simulated efficiently on a classical computer, a result known as the Gottesman–Knill theorem. To complete the set, the T gate is needed, and it cannot be done directly in a fault-tolerant way. The standard solution, proposed by Sergey Bravyi and Alexei Kitaev in 2005, is to prepare special ancilla states, called *magic states*, purify them by a process called *distillation* that turns many noisy copies into a few good ones, and then use them, with a teleportation-like trick, to apply T gates. Magic-state factories may take up a large part of a future quantum computer, and the number of T gates is often the main measure of an algorithm's cost.

Chapter summary. Quantum error correction protects information without copying it, by encoding one logical qubit in an entangled state of several physical qubits, such as $\alpha|000\rangle + \beta|111\rangle$. Parity checks measured via ancilla qubits reveal where an error occurred (the syndrome) without revealing the encoded amplitudes. Syndrome measurement digitises continuous errors into a few discrete ones that can be corrected. Codes that handle both bit flips and phase flips, such as Shor's nine-qubit code, correct arbitrary single-qubit errors. The threshold theorem says that if physical error rates are below a threshold, logical errors can be made arbitrarily small with modest overhead. The surface code, with a threshold near 1% and only nearest-neighbour interactions, has been demonstrated below threshold; a useful machine may need around a thousand physical qubits per logical qubit.

Exercises

21.1 For the classical repetition code with $p = 0.1$, compute the probability that the majority vote is wrong. (Count the cases with two flips and three flips.) Is the code helpful at this error rate? What about $p = 0.6$?

21.2 The state $\alpha|000\rangle + \beta|111\rangle$ suffers a bit flip on qubit 3. Write the resulting state and work out the two parity-check results.

21.3 Why does measuring "do qubits 1 and 2 agree?" not disturb the encoded state, while measuring "is qubit 1 equal to 0?" would?

21.4 Show that a phase flip (Z) on the first qubit of $\alpha|000\rangle + \beta|111\rangle$ gives the same result as a phase flip on the second qubit. Why does this mean the bit-flip code cannot detect phase errors?

21.5 A surface code patch of distance d has about $2d^2$ qubits in total (data plus check). How many qubits are in a patch of distance 7? Of distance 25?

CHAPTER 22

Building a quantum computer

So far we have treated qubits as abstract objects obeying mathematical rules. Somebody, somewhere, has to build them. This chapter is a tour of the laboratories. We will look at what a physical system needs in order to serve as a qubit, and then at the main technologies being pursued today. No single technology has won yet, and each has its own strengths and difficulties. The details change quickly, so the emphasis here is on the underlying physics, which does not.

22.1 What a qubit needs

In 2000 the physicist David DiVincenzo listed the requirements that any physical implementation of a quantum computer must meet. His list is still a useful checklist. In plain words, we need:

1. **Well-defined qubits, and lots of them.** Each qubit must have two clearly identified states, and the design must allow the number of qubits to grow.
2. **A way to start.** We must be able to put every qubit reliably into a known state, usually $|0\rangle$, at the start of a computation.
3. **Long coherence.** The qubits must keep their quantum states for much longer than it takes to perform a gate.
4. **A universal set of gates.** We must be able to apply single-qubit rotations and at least one entangling two-qubit gate, with high accuracy.
5. **A way to read the answer.** We must be able to measure individual qubits reliably.

Two further requirements concern sending qubits between machines, which matters for quantum networks. Every technology in this chapter meets the five main requirements to some degree. The differences lie in how well, and in how

easily each can be scaled up to millions of qubits.

22.2 Superconducting circuits

The technology used by several of the largest industrial efforts, including those of Google and IBM, is the *superconducting qubit*. It is an electrical circuit, a few tenths of a millimetre across, printed onto a chip using the same kinds of techniques used to make conventional microchips. It is made of a metal such as aluminium, which, when cooled close to absolute zero, becomes a *superconductor*: electric current flows through it with no resistance at all.

A superconducting circuit containing a capacitor and an inductor behaves like an electrical pendulum, with current sloshing back and forth at a frequency of a few billion times per second (a few gigahertz, the same range as Wi-Fi). At very low temperatures this oscillation is quantised, with discrete energy levels, like an atom. On its own, the levels of such an oscillator are evenly spaced, which would make it impossible to drive the transition between the lowest two levels without also exciting the next ones. The fix is a special component called a *Josephson junction*, two layers of superconductor separated by an insulating barrier about a nanometre thick, which makes the level spacing uneven. The lowest two levels then form a qubit. The most common design is called the *transmon*.

Superconducting qubits are controlled with pulses of microwaves sent down coaxial cables, and their state is read out by coupling each qubit to a small microwave resonator whose frequency shifts slightly depending on whether the qubit is 0 or 1. Gates are fast, taking tens of nanoseconds. The chips are made by established fabrication methods, which is an advantage for scaling.

The disadvantages are cold and wiring. To keep thermal noise from disturbing the qubits, the chip must be cooled to about 10 millikelvin, a hundredth of a degree above absolute zero and far colder than outer space, in a device called a *dilution refrigerator*. The characteristic golden "chandelier" seen in photographs of quantum computers is the inside of such a refrigerator, with its stages of progressively colder plates and the cables that bring control signals down to the chip. Each qubit needs its own control lines, and running thousands of cables into a refrigerator, without bringing in heat, is a major engineering problem. Superconducting qubits also differ slightly from one another, because of tiny variations in fabrication, and they are sensitive to microscopic defects in the materials.

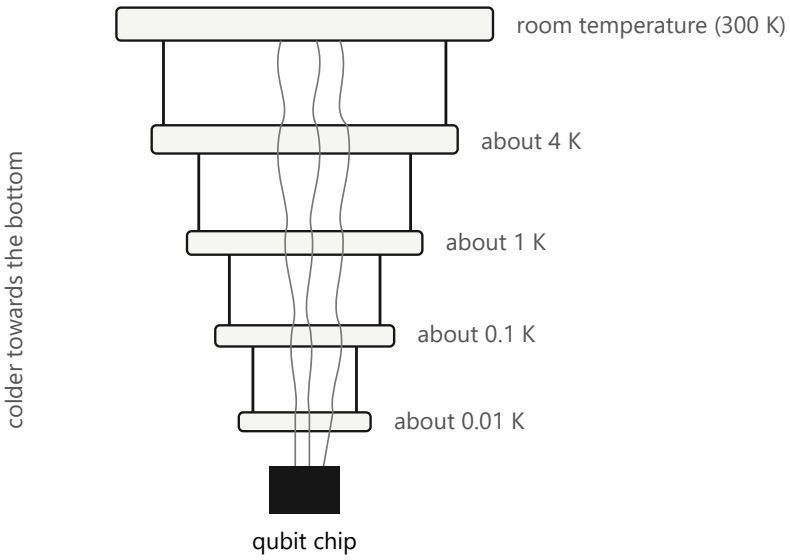


Figure 22.1. A schematic of a dilution refrigerator for superconducting qubits. Successive plates are held at progressively lower temperatures, from room temperature at the top to about 10 millikelvin at the bottom, where the qubit chip sits. Control and readout cables (grey) wind down through each stage, filtered and cooled along the way.

22.3 Trapped ions

An ion is an atom that has lost or gained an electron and so carries an electric charge. Charged particles can be held in place by electric fields, and in an *ion trap* a string of ions, typically of elements like ytterbium, calcium or barium, floats in a vacuum chamber, suspended by oscillating electric fields from electrodes a fraction of a millimetre away. The ions repel each other and settle into a neat line, a few micrometres apart.

Each ion is a qubit, with two of its internal energy levels as $|0\rangle$ and $|1\rangle$. Because every ion of a given isotope is identical (nature manufactures them to perfect tolerances), all the qubits are the same, a significant advantage over fabricated qubits. Gates are applied with precisely tuned laser beams or microwaves. Two-qubit gates use the fact that the ions in the string push on each other: a laser can make the whole string vibrate in a way that depends on the state of one ion, and that shared vibration then affects another ion, entangling the two. Readout uses a laser that makes an ion glow if it is in one state and stay dark if it is in the other; a camera sees which ions are glowing.

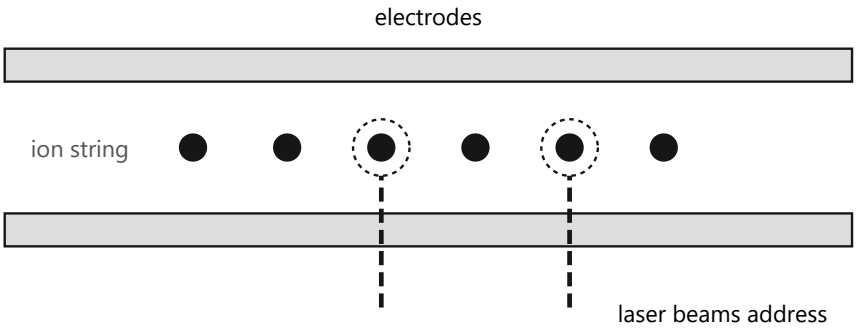


Figure 22.2. A trapped-ion processor, schematically. Ions (black) are held in a line by electric fields from nearby electrodes, a few micrometres apart. Focused laser beams address individual ions to perform gates; here two ions are being entangled through their shared motion.

Trapped ions have some of the best figures of merit of any technology: coherence times of seconds or more, and the highest gate fidelities reported for any qubit. Any ion in a string can be entangled with any other, not just its neighbours. Their main drawbacks are speed, since gates take tens to hundreds of microseconds, a thousand times slower than superconducting gates, and scaling, since a single string becomes hard to control beyond a few dozen ions. Companies such as Quantinuum and IonQ are pursuing architectures in which ions are shuttled between separate zones of a chip-like trap, or in which separate traps are linked by photons.

22.4 Neutral atoms

A related approach uses uncharged atoms, often rubidium, caesium or strontium. Neutral atoms cannot be held by electric fields, but they can be trapped by tightly focused laser beams, called *optical tweezers*. Hundreds or thousands of tweezers can be created at once, arranged in any pattern, and each holds a single atom. The atoms are cooled to microkelvin temperatures by lasers, without the need for a dilution refrigerator.

To make two atoms interact, a laser excites them briefly into a *Rydberg state*, in which one electron is lifted into a huge orbit, making the atom enormous by atomic standards and causing it to interact strongly with its neighbours. This interaction is used to perform two-qubit gates. Because the tweezers can be moved, atoms can be rearranged during a computation, bringing any pair together.

This flexibility has made neutral atoms one of the fastest-moving technologies; arrays of thousands of atoms have been demonstrated, and in 2023 and 2024 groups at Harvard and MIT, working with the company QuEra, ran small error-corrected computations on dozens of logical qubits. Challenges include atom loss (atoms occasionally escape their tweezers and must be replaced) and slower operations than superconducting circuits.

22.5 Photons

Photons are natural carriers of quantum information. They barely interact with their surroundings, so they decohere very little, and they travel at the speed of light, which makes them ideal for communication. Their weakness is the flip side of their strength: because photons barely interact with anything, it is hard to make two photons interact with each other, which is what a two-qubit gate requires.

Photonic quantum computing gets around this with an idea called *measurement-based* or *fusion-based* quantum computing. Instead of applying gates in sequence, the machine first creates a large entangled state of many photons, and then performs the computation entirely by measuring photons one by one, with each measurement basis chosen according to earlier results. Entanglement plus measurement turns out to be as powerful as a circuit of gates. Companies such as PsiQuantum and Xanadu are pursuing photonic machines, which could be built with the mature manufacturing methods of the telecommunications industry and operated at temperatures less extreme than superconducting chips need. The main difficulties are losing photons along the way and generating single photons reliably on demand.

22.6 Spins in silicon and other approaches

The semiconductor industry has spent seventy years learning how to control electrons in silicon, and it would be attractive to build qubits the same way. In *spin qubits*, a single electron is trapped in a tiny region of a silicon chip, called a *quantum dot*, and its spin (Section 5.1) serves as the qubit. Spin qubits are extremely small, potentially allowing millions on a single chip, and they can be made in conventional chip factories. Making them uniform enough and wiring them up remain challenges.

Further approaches include qubits stored in defects in diamond (useful for sensing and networking), and *topological qubits*, which aim to store information in exotic collective states of matter, spread out in such a way that local noise cannot disturb it. Topological qubits would be naturally protected against errors, but they have proved very hard to create, and claims of having made their basic ingredients, so-called Majorana modes, have been the subject of intense scientific debate. Microsoft has invested heavily in this approach.

22.7 The rest of the machine

A quantum computer is much more than its qubits. Around the chip, trap or tweezer array sits a substantial classical system: lasers or microwave generators that produce precisely shaped control pulses, electronics that read out measurement signals, and classical computers that compile programs into pulses, schedule operations, and, in an error-corrected machine, decode syndrome measurements fast enough to keep up with the errors. For a superconducting surface code, decoding must happen in about a microsecond per round, continuously, for every logical qubit.

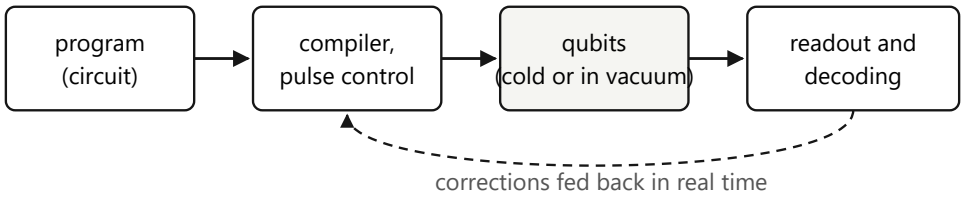


Figure 22.3. The control stack of a quantum computer. A program is compiled into physical control pulses that act on the qubits; measurement signals are read out and, in an error-corrected machine, decoded by a classical computer that feeds corrections back within microseconds.

Programs for quantum computers are written today in software frameworks, many of them open source, that let a user describe a circuit in a conventional programming language, simulate it on an ordinary computer and then send it to real hardware over the internet. Anyone can try this, free of charge, on small devices, and doing so is an excellent next step after this book.

22.8 Which will win?

Nobody knows. Superconducting circuits benefit from fast gates and established fabrication but struggle with wiring and uniformity. Ions offer superb quality but slow gates. Neutral atoms offer flexibility and large numbers but have their own speed and loss issues. Photons and spins offer paths to manufacturing at enormous scale, but are less mature. It is quite possible that different technologies will dominate different roles, with, for example, photons linking processors built from other kinds of qubits. The history of classical computing, in which vacuum tubes gave way to transistors and transistors to integrated circuits, suggests that the technology of a mature quantum computer may not be any of those we can now see clearly.

Chapter summary. A physical qubit technology must offer well-defined, scalable qubits, reliable initialisation, long coherence, a universal set of accurate gates and reliable measurement. Superconducting circuits are chip-based, fast and fabricated like microchips but need dilution refrigerators at about 10 millikelvin. Trapped ions are identical, long-lived and very accurate but slow, and hard to scale in a single string. Neutral atoms in optical tweezers can be rearranged and number in the thousands, and have run small error-corrected computations. Photons excel at communication and can compute via measurements on large entangled states. Spin qubits in silicon promise very dense chips. Every quantum computer is embedded in an extensive classical control system, and no technology has yet emerged as the clear winner.

Exercises

22.1 Absolute zero is −273.15 °C. What is 10 millikelvin in degrees Celsius? Compare it with the temperature of deep space, about 2.7 K.

22.2 A superconducting two-qubit gate takes 50 ns and a trapped-ion gate takes 200 μs. How many gates can each perform per second? If an algorithm needs 10^9 gates in sequence, how long does each take, ignoring error correction?

22.3 Go through DiVincenzo's five requirements for photons. Which ones are easy to satisfy and which are hard?

22.4 Explain why the fact that all ions of a given isotope are identical is an advantage for building a quantum computer.

CHAPTER 23

Measuring information

Throughout this book we have talked loosely about information: a bit holds one unit of it, a measurement extracts some of it, decoherence leaks it into the environment, an eavesdropper gains some of it. In this chapter we make the idea more precise. The theory of information was created for classical communication in 1948, and its quantum extension underlies much of what we have seen, from Holevo's limit on what a qubit can carry to the security of quantum key distribution. We will keep the mathematics light and focus on the ideas.

23.1 Surprise

How much information is in a message? In 1948 Claude Shannon, an engineer at Bell Labs, proposed an answer that founded a new science. Shannon's key insight was to ignore the *meaning* of a message entirely and ask only how *surprising* it is. A message you could have predicted in advance tells you nothing new. A message that could have been any one of a million possibilities tells you a lot.

Shannon measured information in bits. Learning the outcome of a fair coin toss, which could have gone either way, gives you exactly one bit. Learning which of four equally likely possibilities occurred gives two bits, since you could have encoded the answer as two coin tosses. Learning which of 2^n equally likely possibilities occurred gives n bits. In general, an outcome that had probability p carries $\log_2(1/p)$ bits of information, where $\log_2$ means "the power of 2 that gives this number". An outcome with probability 1/8 carries $\log_2 8 = 3$ bits. An outcome with probability 1, which was certain, carries 0 bits.

23.2 Entropy

The *entropy* of a source of messages is the average amount of information per message, the average surprise. For a source that produces outcome i with probability p_i, Shannon's entropy is

$$H = p_1 \log_2(1/p_1) + p_2 \log_2(1/p_2) + \ldots$$

A fair coin has entropy 1 bit. A coin that always lands heads has entropy 0. A biased coin that lands heads 90% of the time has entropy of about 0.47 bits: its tosses are mostly predictable, so each carries less than a full bit of surprise.

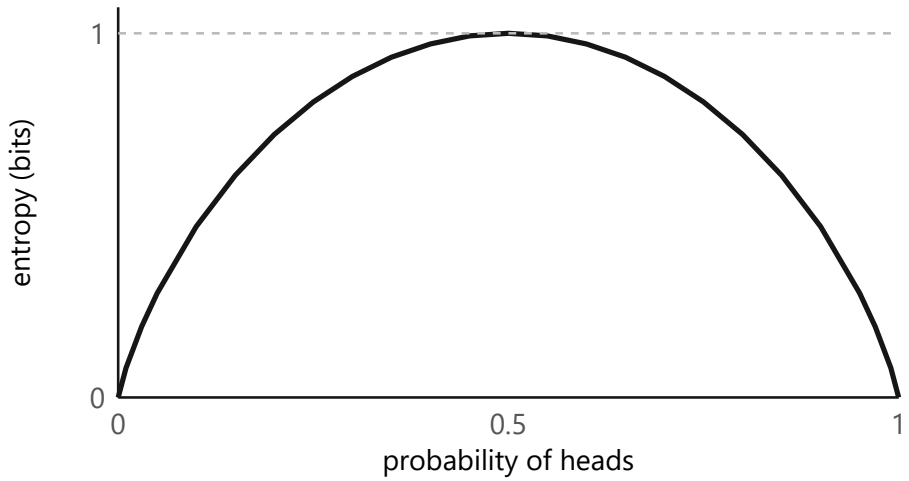


Figure 23.1. The entropy of a coin as a function of its probability of landing heads. It is largest, one full bit, for a fair coin, and zero for a coin whose outcome is certain.

Shannon proved that entropy has a very practical meaning: it is the limit to which a message can be compressed. A long sequence of tosses of the 90% coin can be squeezed, on average, into about 0.47 bits per toss, and no further. Every file compression program, from ZIP to the formats used for images and audio, is an application of this result. Shannon also showed that entropy sets the rate at which information can be sent reliably over a noisy channel, and that error-correcting codes can achieve that rate, launching the field of coding theory on which modern communications depend.

23.3 Quantum entropy

In 1932, long before Shannon, the mathematician John von Neumann had defined an entropy for quantum states. For a pure state, a point on the surface of the Bloch sphere, the *von Neumann entropy* is zero: there is nothing uncertain about it, even though measuring it may give random results. For a mixed state, a point inside the sphere, it is positive, and it is largest, one bit for a single qubit, at the centre of the sphere, where the state is a completely random mixture.

Von Neumann entropy measures how much we do *not* know about how the qubit was prepared. It plays the same role for quantum information that Shannon's entropy plays for classical information: Benjamin Schumacher showed in 1995 that it is the limit to which quantum states can be compressed. (Schumacher is also the person who coined the word "qubit", in conversation with William Wootters in 1992.)

23.4 Entanglement makes parts uncertain

Here is one of the strangest facts in quantum information. Take the Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$. As a whole, it is a pure state; we know it exactly, so its entropy is zero. But what about one of its qubits on its own?

We saw in Chapter 10 that the first qubit alone gives 0 or 1 at random, and the same in every other basis. Its state, considered alone, is the completely mixed state at the centre of the Bloch sphere, with one full bit of entropy. So the whole has zero entropy, while each part has one bit. We know everything about the pair and nothing about each member.

This could never happen classically. In classical information theory, the uncertainty of a whole is always at least as large as the uncertainty of any of its parts: if you know everything about a pair of dice, you certainly know everything about each die. In quantum mechanics, the whole can be more certain than its parts, and the amount by which this happens is a measure of entanglement. For a pure state of two systems, the entropy of either part is called the *entanglement entropy*; it is zero for product states and largest for maximally entangled states like the Bell states.

This gives a fresh way of seeing decoherence (Chapter 20). When a qubit becomes entangled with its environment, the joint state of qubit and environment may still be pure, but the qubit on its own acquires entropy. From the qubit's point

of view, it has become mixed; its quantum information has not been destroyed but has been spread into correlations with the environment, where we cannot get at it.

23.5 Holevo's limit revisited

We can now restate the result of Section 5.6 more precisely. Suppose Alice encodes classical messages into the states of qubits and sends them to Bob. Holevo's theorem says that the amount of classical information Bob can extract is at most the von Neumann entropy of the states Alice sends, averaged appropriately, which for n qubits is at most n bits. However the encoding is chosen, and however cleverly Bob measures, n qubits carry no more than n bits of accessible classical information. Superdense coding (Section 14.5) appears to beat this, but only because the entangled qubit sent in advance also counts.

This limit is a warning against the most naive picture of quantum computing. The 2^n amplitudes of an n-qubit register are not a store of 2^n numbers that we can read out. They are a vast space in which interference can happen, but the information that can be extracted at the end is limited to n bits per run.

23.6 Information is physical

Rolf Landauer, whom we met in Chapter 2, was fond of the slogan "information is physical". Information is always carried by something, a voltage, a magnetised grain, a photon, an ion, and the laws of physics govern what can be done with it. Landauer's principle, that erasing a bit releases a minimum amount of heat, links information theory to thermodynamics. Quantum information theory takes the slogan further: since the world is quantum, the ultimate rules for storing, sending and processing information are the rules of quantum mechanics. Classical information theory turns out to be a special case, the case in which we never use superposition or entanglement. Seen this way, quantum computing is not an exotic add-on to computer science but a more complete account of what computation can be.

Chapter summary. Shannon measured information by surprise: an outcome of probability p carries $\log_2(1/p)$ bits, and the entropy of a source, its average information, sets the limit for data compression. Von Neumann entropy extends this to quantum states: zero for pure states, up to one bit per qubit for mixed states. For an entangled pure state, the whole has zero entropy while each part can have positive entropy, something impossible classically; the entropy of a part measures entanglement. Decoherence spreads a qubit's information into correlations with the environment. Holevo's theorem limits the accessible information in n qubits to n bits. Information is physical, and quantum information theory gives the most general account of its possibilities.

Exercises

23.1 How many bits of information are gained by learning the outcome of rolling a fair eight-sided die? A fair six-sided die (use $\log_2 6 \approx 2.58$)?

23.2 Compute the entropy of a coin that lands heads with probability 1/4. (Use $\log_2 4 = 2$ and $\log_2(4/3) \approx 0.415$.)

23.3 What is the von Neumann entropy of a qubit in the state $|+\rangle$? Of a qubit at the centre of the Bloch sphere?

23.4 Explain in your own words why "knowing everything about the whole but nothing about the parts" is impossible for two ordinary dice, but possible for two entangled qubits.

CHAPTER 24

What quantum computers can and cannot do

We have now seen several quantum algorithms, some with spectacular speed-ups and some with modest ones, and we have seen that for unstructured search no quantum algorithm can do better than a square-root improvement. It is natural to ask for the big picture. Which problems will quantum computers solve efficiently, and which will remain hard for them too? This is the territory of *computational complexity theory*, the branch of computer science that sorts problems by how difficult they are. Its answers are partly proved, partly conjectured and partly unknown, and this chapter tries to be clear about which is which.

24.1 Sorting problems by difficulty

Complexity theory groups problems into *complexity classes*, according to the resources needed to solve them. Recall from Chapter 2 that a problem is regarded as "easy" if it can be solved by an algorithm whose running time grows polynomially with the size of the input, and "hard" if every algorithm needs exponential time or something close to it.

The class of problems that a classical computer can solve in polynomial time is called **P**. Sorting a list, multiplying numbers, finding the shortest route between two places on a map, and checking whether a number is prime (a fact proved only in 2002) are all in P.

A second, larger class is called **NP**. It contains the problems whose solutions can be *checked* in polynomial time, even if finding them might be hard. Sudoku is a good example. Given a filled-in grid, checking that it obeys the rules is quick. Finding a solution for a large Sudoku from scratch appears to be much harder. Factoring is also in NP: given a proposed factorisation of N, you can check it by multiplying. So

is the *travelling salesman problem* in its yes-or-no form: is there a route visiting all of these cities with total length below some limit? Given a route, it is easy to add up its length.

Every problem in P is also in NP, since if you can find a solution quickly you can certainly check one. The most famous open question in computer science, and one of the seven Millennium Prize Problems for which the Clay Mathematics Institute offers a million dollars, is whether P equals NP: whether every problem whose solutions are easy to check also has solutions that are easy to find. Nearly everyone believes the answer is no, but no one has proved it.

Within NP sit the *NP-complete* problems, the hardest problems in NP, in a precise sense: if any one of them could be solved in polynomial time, then so could every problem in NP. Sudoku (generalised to large grids), the travelling salesman problem and thousands of other practically important problems in scheduling, logistics, circuit design and biology are NP-complete. Factoring, interestingly, is *not* believed to be NP-complete. It seems to sit in a middle ground: probably not in P, but probably not among the hardest problems in NP either.

24.2 The quantum class

The problems that a quantum computer can solve in polynomial time, with a high probability of giving the right answer, form the class **BQP**, for "bounded-error quantum polynomial time". What do we know about it?

BQP contains P, because a quantum computer can simulate any classical computation efficiently (Chapter 11). It contains factoring and a few related problems from number theory and algebra, thanks to Shor. And it contains the simulation of quantum physical systems, as Feynman suggested and later work confirmed.

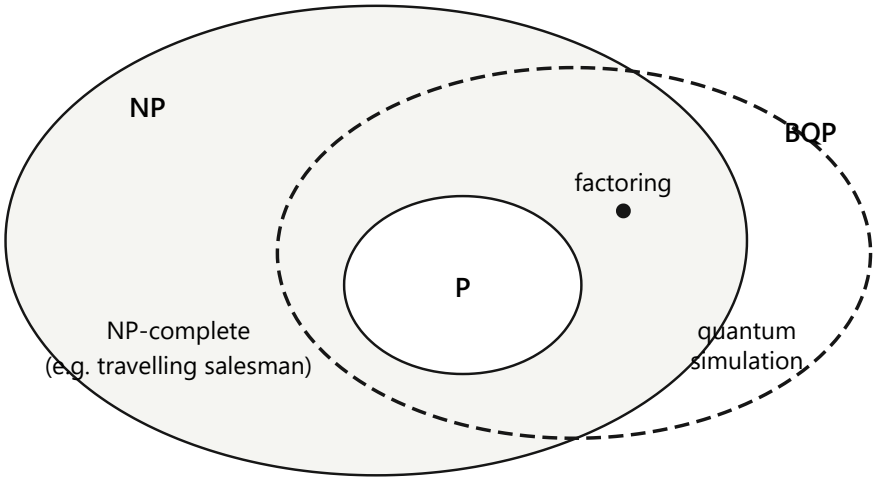


Figure 24.1. The relationship between complexity classes as most researchers believe it to be (none of the separations shown is proved). P sits inside both NP and BQP. Factoring is in NP and in BQP but is believed to lie outside P. NP-complete problems are believed to lie outside BQP. BQP also contains problems, such as simulating quantum systems, that are not believed to be in NP.

The crucial question is whether BQP contains the NP-complete problems. If it did, quantum computers could efficiently solve the travelling salesman problem, design optimal schedules and crack every puzzle whose solutions are easy to check. The answer is not known for certain, but almost all researchers believe that *quantum computers cannot efficiently solve NP-complete problems*. The strongest evidence is the optimality of Grover's algorithm (Section 17.5). An NP-complete problem, stripped of all structure, is essentially a search through exponentially many candidates, and for unstructured search a quantum computer gains only a square root: 2^n becomes $2^{n/2}$, which is still exponential. To do better, a quantum algorithm would have to exploit structure in the problem, and no one has found a way to do that for NP-complete problems.

This deserves emphasis, because it contradicts one of the most common claims made about quantum computers. They are not machines that can solve any hard problem by trying all possibilities at once. Their power appears to be confined to problems with special structure, such as hidden periodicity, that interference can exploit.

24.3 Evidence for a quantum advantage

Is BQP really larger than P? That is, are there problems that quantum computers can solve efficiently but classical computers cannot? This, too, has not been proved. Proving it would imply that P is not equal to PSPACE (a larger classical class), which is itself a famous unsolved problem believed to be far beyond current mathematical techniques. So nobody can prove that factoring is hard for classical computers; it remains possible, though very few experts think it likely, that a fast classical factoring algorithm will be found tomorrow.

The evidence for a genuine quantum advantage comes from several directions. Factoring has resisted the efforts of mathematicians for centuries, and the best classical algorithms remain super-polynomial. In the *oracle* setting, where an algorithm is given a black box and only the number of queries is counted (as in Chapters 13 and 17), it can be proved that quantum computers need exponentially fewer queries than classical ones for some problems. And for certain sampling tasks, it has been shown that an efficient classical simulation of quantum computers would cause a collapse of complexity classes that theorists consider highly implausible. This last kind of argument underpinned the quantum advantage experiments mentioned in Section 20.5.

24.4 The extended Church–Turing thesis

In the 1930s Alonzo Church and Alan Turing independently gave precise definitions of what it means for something to be computable, and showed that their definitions agreed. The *Church–Turing thesis* asserts that anything that can be computed by any physical process can be computed by a Turing machine, Turing's idealised model of a computer. Quantum computers do not challenge this thesis. As we said in Chapter 1, a classical computer can simulate a quantum computer, given enough time, so quantum computers cannot compute anything uncomputable.

There is a stronger version, called the *extended* Church–Turing thesis, which asserts that any physical process can be simulated by a classical computer *efficiently*, with at most a polynomial slow-down. For decades it seemed a reasonable bet; every proposed alternative model of computation turned out to be no more powerful, in this efficient sense, than ordinary computers. Quantum computing is the first serious challenge to the extended thesis. If large quantum computers can be built and Shor's algorithm runs on them, then nature permits

efficient computations that classical computers apparently cannot imitate, and the extended Church–Turing thesis is false. Some physicists have argued that building a quantum computer is worthwhile for this reason alone: either it works, and we learn that the universe computes in a fundamentally richer way than we assumed; or it fails for some reason we do not yet understand, and we discover new physics.

24.5 A rough map of applications

Putting the theory together with the algorithms of Part III gives a rough map of where quantum computers are expected to help.

Exponential speed-ups are known, or strongly believed, for simulating quantum systems in physics, chemistry and materials science, and for certain problems in number theory and algebra, notably factoring and the related *discrete logarithm problem* (which underlies elliptic-curve cryptography). These are the applications with the strongest theoretical foundations.

Polynomial speed-ups, often quadratic, are available for a much broader range of problems, through Grover's algorithm, amplitude amplification and related techniques: search, some optimisation problems, Monte Carlo estimation and others. Whether these will be useful in practice depends on the overheads of error correction, and for many of them the answer may be no for a long time.

For many other problems, including the bulk of everyday computing, there is no reason to expect any quantum advantage at all. And for NP-complete problems in general, an exponential quantum speed-up is believed to be impossible.

This map has changed before and will change again. New quantum algorithms continue to be discovered, and so do new classical algorithms that erase claimed quantum advantages. One of the pleasures of the field is that some of its most basic questions are still open.

Chapter summary. P is the class of problems solvable efficiently by classical computers; NP is the class whose solutions can be checked efficiently; NP-complete problems are the hardest in NP. Whether P equals NP is unknown. BQP, the class of problems solvable efficiently by quantum computers, contains P, factoring and quantum simulation. Quantum computers are believed not to solve NP-complete problems efficiently, since unstructured search allows only a quadratic speed-up. That BQP is larger than P is not proved but is supported by strong evidence. Quantum computers do not violate the Church–Turing thesis but would refute its extended, efficiency-based version. Exponential speed-ups are expected mainly for simulation and number-theoretic problems; quadratic ones for search-like problems.

Exercises

24.1 Give an everyday example of a problem whose solutions are easy to check but seem hard to find.

24.2 A problem has 2^{60} candidate solutions and no structure. How many oracle calls does Grover's algorithm need, roughly? Is that a practical number if each call takes one microsecond?

24.3 Explain why a proof that factoring is not in P would be a major result in mathematics, regardless of quantum computing.

24.4 A start-up claims that its quantum computer "solves NP-complete problems exponentially faster than any classical computer". Based on this chapter, what questions would you ask?

CHAPTER 25

Where we stand and where we might go

We began with a newspaper article promising that quantum computers would soon break every password, design new medicines and try every answer at once. You are now in a position to judge such claims for yourself. In this final chapter we take stock: how the field got here, what remains to be done, how to read the news, and where to go next if you want to learn more.

25.1 Forty years in one page

The history of quantum computing is short, and most of it has happened within living memory.

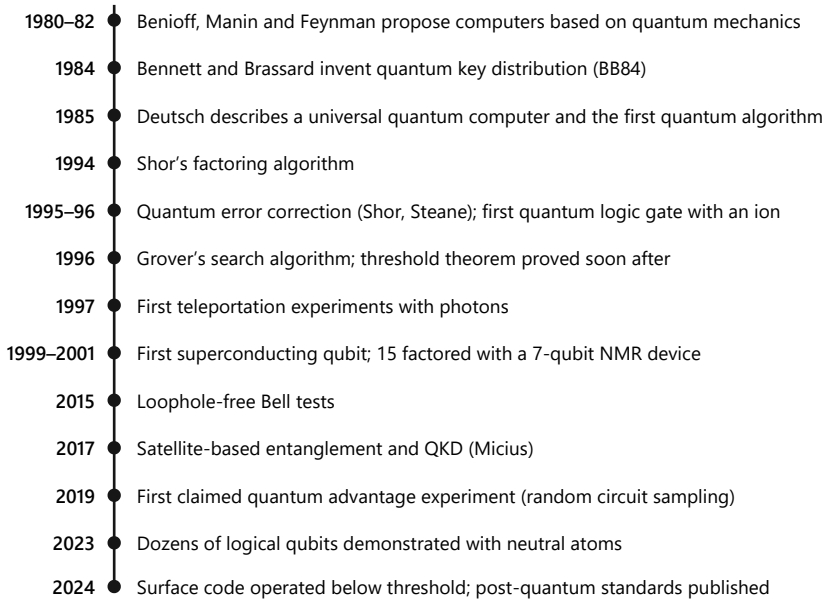


Figure 25.1. Some milestones in quantum computing. The list is necessarily selective; many important contributions are omitted.

Looking back over the timeline, two things stand out. First, the theory came long before the hardware. Shor's algorithm, error correction and the threshold theorem were all worked out at a time when the best experiments could control one or two qubits. Second, progress in hardware has been steady rather than sudden. Each step, from the first two-qubit gate to error correction below threshold, took years of incremental improvement in materials, control electronics and fabrication. There is every reason to expect the next steps to follow the same pattern.

25.2 The road to useful machines

It is common to describe the path ahead in three stages, though the boundaries between them are blurry.

The first stage is the noisy era described in Section 20.5, in which machines with up to a few thousand physical qubits run short circuits without full error correction. Its main achievements have been scientific: demonstrating control of large quantum systems, testing ideas from physics, and developing the techniques

that later machines will need. Whether NISQ machines will solve any commercially valuable problem better than classical computers is doubtful, though not impossible.

The second stage, which is beginning now, is *early fault tolerance*: machines with tens to hundreds of logical qubits, protected by error correction, able to run circuits of millions of operations. These could tackle scientific problems in physics and chemistry that are beyond classical simulation, even if they cannot yet run Shor's algorithm on cryptographic numbers.

The third stage is *large-scale fault tolerance*: thousands of logical qubits and billions of operations, enough for Shor's algorithm on real cryptographic keys and for detailed chemistry on molecules like FeMoco. Estimates suggest that such machines will need hundreds of thousands to millions of physical qubits with error rates comfortably below threshold. Several companies and national programmes have published roadmaps aiming for this stage in the 2030s. Roadmaps are statements of ambition, not forecasts; but the steady fall in resource estimates, combined with the steady improvement in hardware, has made the goal look considerably less distant than it did a decade ago.

25.3 Beyond computing

Quantum computing is the most famous part of a broader set of quantum technologies, and some of the others may arrive sooner.

Quantum sensing uses the extreme sensitivity of quantum states, the very fragility that is such a problem for computing, to make exceptionally precise measurements. Atomic clocks, which already keep time for satellite navigation systems, are quantum devices; newer ones are so precise that they can detect the difference in the flow of time between two heights a few centimetres apart, as predicted by Einstein's theory of gravity. Quantum magnetometers can detect the faint magnetic fields of the brain and heart, and quantum gravimeters can find structures hidden underground. Entanglement can push the precision of some measurements beyond what independent particles allow.

Quantum networks aim to connect quantum devices with entanglement, using the techniques of Chapters 14 and 19: teleportation, quantum repeaters and quantum key distribution. A future quantum internet could provide

communication secured by physics, link small quantum computers into larger ones, and connect quantum sensors to improve their precision.

25.4 How to read the news

Every week brings new announcements about quantum computers. Some report genuine progress, some are marketing, and many are a mixture. Here are some questions that this book should have prepared you to ask.

How many qubits, and how good are they? The number of qubits alone means little. A thousand noisy qubits may be less useful than a hundred good ones. Look for two-qubit gate error rates, and for whether the qubits are physical or logical (error-corrected).

Faster than what, at what? A claim of quantum advantage should name a specific task and compare against the best known classical method, not a naive one. Ask whether the task is useful, and whether classical researchers have had a chance to respond.

Is it Shor, simulation, or something else? Exponential speed-ups are expected for factoring and related problems and for simulating quantum systems. Claims of exponential speed-ups for optimisation, machine learning or NP-complete problems deserve particular scepticism.

Was error correction involved? Since useful applications require fault tolerance, progress in logical qubits and logical error rates is often more significant than headline qubit counts.

Is "quantum" doing any work? Some products described as quantum, from "quantum-inspired" algorithms (which run on ordinary computers) to "quantum" consumer gadgets, have little or nothing to do with the subject of this book.

25.5 Where to go from here

If this book has done its job, you now understand the core ideas of quantum computing: qubits, superposition, measurement, gates, entanglement, interference, the main algorithms, and why noise and error correction dominate the engineering. There are several directions you could go next.

You can *try it yourself*. Several companies offer free access to small quantum computers over the internet, along with open-source programming frameworks such as Qiskit, Cirq and others, which let you build circuits like the ones in this book, simulate them on your own computer and run them on real hardware. Building a Bell pair, teleporting a qubit or running Grover's algorithm on two or three qubits are excellent first projects.

You can *learn the mathematics properly*. The natural next step is a course or textbook on linear algebra, which is the language of quantum mechanics: vectors, matrices, eigenvalues and complex numbers, all of which you have met here in gentle form. With linear algebra in hand, the standard graduate textbooks on quantum computation become accessible. Some suggestions are given in the Further Reading section at the end of this book.

You can *go deeper into the physics*, through an introductory course on quantum mechanics, or into the computer science, through courses on algorithms and complexity. Quantum computing sits at the meeting point of physics, mathematics, computer science and engineering, and people come to it from all of them.

25.6 A last word

At the start of this book we said that some of its ideas are genuinely strange, and that you would be in excellent company if you found them so. That remains true. Nobody fully understands why nature uses amplitudes instead of probabilities, why measurement works as it does, or what exactly entanglement tells us about reality. What has changed over the past forty years is that these strange features have moved from the pages of philosophy journals into laboratories and, increasingly, into engineering. We no longer just marvel at superposition and entanglement; we build machines out of them.

Whether those machines will transform chemistry, cryptography and our understanding of computation, or whether they will prove more limited than their champions hope, is something the coming decades will reveal. Either way, the questions are fascinating, and anyone willing to take them one step at a time can understand them. We hope this book has shown you that.

Chapter summary. The theory of quantum computing, including Shor's and Grover's algorithms and error correction, was largely in place by the late 1990s; hardware has progressed steadily since, culminating in demonstrations of error correction below threshold. The path to useful machines runs from noisy devices through early fault-tolerant machines with tens to hundreds of logical qubits to large-scale machines with millions of physical qubits. Quantum sensing and quantum networks are related technologies that may mature sooner. When reading news about quantum computers, ask about qubit quality, the comparison baseline, the type of problem and the role of error correction. The next steps for a learner are hands-on programming with open-source tools and the study of linear algebra.

Exercises

25.1 Find a recent news article about a quantum computer and apply the questions of Section 25.4 to it. Which questions does the article answer, and which does it leave open?

25.2 Using what you learned in Chapters 16, 20 and 21, explain in a paragraph why a machine with 1,000 physical qubits cannot factor a 2048-bit RSA number.

25.3 Write a one-page explanation of entanglement for a friend who has never studied physics, using no equations. Then check it against Chapter 10 for any of the misunderstandings warned about there.

Glossary

This glossary collects the technical terms used in the book. The number in brackets after each entry gives the chapter where the term is introduced or explained most fully.

Amplitude (4, 5). A number, possibly negative or complex, attached to each possible outcome of a quantum state. The probability of an outcome is the square of the size of its amplitude. Amplitudes of indistinguishable ways of reaching the same outcome are added, which allows them to cancel.

Ancilla (21). An extra qubit used to help with a computation or measurement, for example to record the parity of other qubits during error correction.

Basis (6). A pair of perpendicular states (for one qubit) that serve as the two possible answers to a measurement. The computational basis is $|0\rangle$, $|1\rangle$; the diagonal or Hadamard basis is $|+\rangle$, $|-\rangle$.

BB84 (19). The first quantum key distribution protocol, invented by Bennett and Brassard in 1984, in which single photons are prepared and measured in randomly chosen bases.

Bell state (10). One of four maximally entangled states of two qubits, such as $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

Bell's theorem (12). The result, due to John Bell, that the correlations of entangled particles cannot be explained by instructions carried along with the particles (local realism). Confirmed by experiments.

Bit (2). The basic unit of classical information: a system with two states, 0 and 1.

Bloch sphere (7). A sphere on whose surface every pure state of a single qubit can be represented; $|0\rangle$ and $|1\rangle$ are at the poles.

BQP (24). The class of problems that quantum computers can solve efficiently (in polynomial time, with bounded error).

Circuit (8). A diagram, or the corresponding sequence of operations, showing which gates are applied to which qubits, in time order from left to right.

Classical (2). Not quantum; describing ordinary computers, bits and probabilities.

CNOT (controlled-NOT) (11). A two-qubit gate that flips the target qubit if the control qubit is 1. It is the most common entangling gate.

Coherence time (20). The time over which a qubit keeps its quantum state before decoherence destroys it; see T_1 and T_2.

Collapse (6). The change of a quantum state upon measurement to the basis state corresponding to the observed outcome.

Complex number (3). A number of the form a + bi, where i × i = −1; it can be pictured as an arrow in a plane, with a length and a direction (phase).

Decoherence (20). The loss of a qubit's quantum behaviour through unwanted interaction with its environment, which acts like an unintended measurement.

Dephasing (20). Decoherence in which the relative phase between amplitudes drifts randomly; characterised by the time T_2.

Deutsch–Jozsa algorithm (13). A quantum algorithm that decides with one query whether a function is constant or balanced.

Diffusion operator (17). The step of Grover's algorithm that reflects all amplitudes about their average.

Entanglement (10). A property of a joint state of several qubits that cannot be written as a product of separate states for each; it produces correlations stronger than any classical correlation.

Entropy (23). A measure of uncertainty or average information. Shannon entropy applies to classical probabilities; von Neumann entropy to quantum states.

Error correction (quantum) (21). Techniques that encode logical qubits in many physical qubits so that errors can be detected and corrected without measuring the encoded information.

Fault tolerance (21). The design of quantum computations, including error correction itself, so that individual faults cannot spread into uncorrectable errors.

Fourier transform (15). A mathematical operation that decomposes a signal into frequencies and turns a periodic pattern into evenly spaced peaks. The quantum Fourier transform (QFT) performs it on the amplitudes of a quantum register.

Gate (2, 8). A basic operation on bits (logic gate) or qubits (quantum gate). Quantum gates are reversible rotations of the state.

Global phase (5). A factor multiplying an entire state, such as −1; it has no observable effect.

Grover's algorithm (17). A quantum algorithm for searching N unstructured items with about √N queries.

Hadamard gate (H) (8). A single-qubit gate that turns $|0\rangle$ into $|+\rangle$ and $|1\rangle$ into $|-\rangle$, and vice versa.

Hamiltonian (18). The description of how a physical system's energy depends on its state; it determines how the state changes in time.

Interference (4). The adding and cancelling of waves, or of amplitudes. Constructive interference reinforces; destructive interference cancels.

Ket (5). The notation $|\ldots\rangle$ for a quantum state, introduced by Dirac.

Logical qubit (21). A qubit encoded in many physical qubits by an error-correcting code.

Measurement (6). The act of extracting a classical result from a quantum system. It gives a random outcome according to the squared amplitudes and changes the state.

Mixed state (7). A state with classical uncertainty, represented by a point inside the Bloch sphere.

NISQ (20). Noisy Intermediate-Scale Quantum: the current era of quantum computers with tens to thousands of qubits, without full error correction.

No-cloning theorem (12). The impossibility of copying an unknown quantum state.

NP, NP-complete (24). NP is the class of problems whose solutions can be checked efficiently; NP-complete problems are the hardest problems in NP.

Oracle (11, 13). A black-box operation that computes a function, $|x\rangle|y\rangle \to |x\rangle|y \oplus f(x)\rangle$; algorithms are often analysed by counting oracle calls.

P (24). The class of problems that classical computers can solve efficiently (in polynomial time).

Pauli gates (X, Y, Z) (8). The single-qubit gates that perform half-turns of the Bloch sphere about the x, y and z axes. X is a bit flip, Z a phase flip.

Phase (3, 5). The direction of a complex amplitude. The relative phase between amplitudes controls interference.

Phase estimation (15). A quantum algorithm that finds the phase an operation applies to one of its eigenstates.

Phase kickback (13). The effect by which an oracle acting on a target in the state $|-\rangle$ turns the function's value into a sign on the input register.

Photon (4). A particle of light; the smallest indivisible amount of light of a given colour.

Polynomial and exponential time (2). Growth of running time like a power of the input size (efficient) or like 2^n (inefficient).

Post-quantum cryptography (16). Classical public-key cryptography designed to resist attack by quantum computers.

Product state (9). A multi-qubit state that can be written as separate states of each qubit; not entangled.

Pure state (7). A quantum state known exactly, represented by a point on the surface of the Bloch sphere.

Quantum key distribution (QKD) (19). The creation of a shared secret key using quantum states, with security guaranteed by the laws of quantum mechanics.

Qubit (5). The basic unit of quantum information: a quantum system with two basis states $|0\rangle$ and $|1\rangle$, which can be in any superposition of them.

Relaxation (20). Decoherence in which a qubit loses energy and decays from $|1\rangle$ to $|0\rangle$; characterised by the time T_1.

Reversible (2). Describing a gate or computation that can be run backwards, so that no information is lost. All quantum gates are reversible.

Shor's algorithm (16). A quantum algorithm that factors large numbers efficiently by finding the period of a modular exponentiation.

Superposition (5). A quantum state that is a weighted combination, with amplitudes, of basis states.

Surface code (21). A quantum error-correcting code on a two-dimensional grid of qubits, with a high threshold and only nearest-neighbour interactions.

Syndrome (21). The results of the parity checks in error correction, which reveal where an error occurred without revealing the encoded state.

Teleportation (14). A protocol that transfers an unknown qubit state using a shared entangled pair and two classical bits.

Threshold theorem (21). The result that, if physical error rates are below a certain threshold, logical error rates can be made arbitrarily small by error correction.

Toffoli gate (11). A three-qubit gate that flips the target if both controls are 1; a reversible AND gate.

Unitary (8). The mathematical term for an operation that preserves the lengths of state arrows; every quantum gate is unitary.

XOR ($\oplus$) (2). The "exclusive or" of two bits: 1 if they differ, 0 if they are equal.

Hints and answers

Answers are given for most of the exercises that have a definite answer. Questions that ask for your own explanation or opinion are not answered here; compare your answer with the relevant section of the chapter.

Chapter 1

1.1 Half of 10^{12} books at 10^{9} per second takes 500 seconds, a little over eight minutes. With 10^{30} books it takes 5×10^{20} seconds, more than a thousand times the age of the universe.

Chapter 2

2.1 5 = 0101, 9 = 1001, 13 = 1101. The string 1111 represents 15.

2.2 $2^{40} = (2^{10})^4 \approx (10^3)^4 = 10^{12}$, about a trillion.

2.3 With both inputs equal to A, NAND gives 1 when A = 0 and 0 when A = 1, which is NOT A.

2.4 XOR alone is not reversible: output 1 could come from 01 or 10. With A also passed through, the outputs A and A XOR B determine B uniquely (B = A XOR output), so the gate becomes reversible. This is the CNOT of Chapter 11.

2.5 For n = 20, about one second (10^6 microseconds). For n = 40, about 10^{12} microseconds, roughly twelve days. For n = 80, about 10^{24} microseconds, some 3×10^{10} years, longer than the age of the universe.

Chapter 3

3.1 Lengths are 10, $\sqrt{2} \approx 1.414$ and 1. Only (0.6, −0.8) is a unit vector.

3.2 (0, 6) and $(2/\sqrt{2}, 0) = (\sqrt{2}, 0)$. The second (vertical) component cancels.

3.3 Swapping gives (0.8, 0.6). The Hadamard takes (0, 1) to $(1/\sqrt{2}, -1/\sqrt{2})$, and applying it again returns (0, 1).

3.4 Tails has probability 0.36. Two heads: $0.64 \times 0.64 \approx 0.41$.

3.5 $(-1/\sqrt{2})^2 + (1/\sqrt{2})^2 = 1/2 + 1/2 = 1$.

3.6 $(1 + i)(1 - i) = 1 - i + i - i^2 = 1 + 1 = 2$. The length of $1 + i$ is $\sqrt{2}$.

Chapter 4

4.1 With the lower route blocked, only the upper route remains, with amplitude $1/2$ to A and $1/2$ to B. Each detector clicks with probability $1/4$, and the photon is absorbed with probability $1/2$.

4.2 The lower route now starts with $-1/\sqrt{2}$ and the upper route's final step to A carries the minus sign. The amplitude for A is $(-1/2) + (-1/2) = -1$ and for B is $(-1/2) + 1/2 = 0$. Squaring gives the same predictions as before: A always clicks.

4.3 The upper route's contributions become $-1/2$ to A and $-1/2$ to B. The totals are $1/2 - 1/2 = 0$ for A and $-1/2 - 1/2 = -1$ for B, so B always clicks.

Chapter 5

5.1 (a) Valid: $0.36 + 0.64 = 1$. (b) Not valid: $0.25 + 0.25 = 0.5$. (c) Valid. (d) Valid: $3/4 + 1/4 = 1$.

5.3 $\beta^2 = 1 - 1/9 = 8/9$, so $\beta = \sqrt{8}/3 \approx 0.943$.

5.4 $(|+\rangle + |-\rangle)/\sqrt{2} = (1/2)(|0\rangle + |1\rangle + |0\rangle - |1\rangle) = |0\rangle$. Similarly $|1\rangle = (|+\rangle - |-\rangle)/\sqrt{2}$.

Chapter 6

6.1 Probability of 0 is $1/4$ and of 1 is $3/4$. After seeing 1 the state is $|1\rangle$, and a second measurement gives 1 with certainty.

6.2 $|1\rangle = (|+\rangle - |-\rangle)/\sqrt{2}$, so "+" and "−" each have probability $1/2$.

6.3 The fraction is $(\cos\theta)^2(\sin\theta)^2$. At 45° it is $1/2 \times 1/2 = 1/4$; at 0° and 90° it is 0.

6.4 Both states can give 0 when measured in the 0/1 basis, so a result of 0 does not settle the question. The same problem arises for every measurement, because $|0\rangle$ and $|+\rangle$ are not perpendicular (see Section 12.3).

6.5 For example $|+\rangle$, $|-\rangle$ and $|+i\rangle$, or the completely mixed state. Measuring copies in the diagonal basis distinguishes $|+\rangle$ (always "+") from $|-\rangle$ (always "−") from the others (random).

Chapter 7

7.1 The probability of 0 is 0.36 = (1 + z)/2, so z = −0.28: slightly south of the equator.

7.2 $|-\mathrm{i}\rangle = (|0\rangle - \mathrm{i}|1\rangle)/\sqrt{2}$.

7.3 Probability of 0 is 1/2. The x coordinate is cos 22.5° ≈ 0.924, so "+" has probability about (1 + 0.924)/2 ≈ 0.96.

7.4 For example a quarter-turn about the y axis, or the Hadamard's half-turn. A quarter-turn about y takes $|1\rangle$ to $|-\rangle$.

Chapter 8

8.1 X gives $0.8|0\rangle + 0.6|1\rangle$; Z then gives $0.8|0\rangle - 0.6|1\rangle$.

8.2 $\mathrm{H}|-\rangle = (1/\sqrt{2})(\mathrm{H}|0\rangle - \mathrm{H}|1\rangle) = (1/2)(|0\rangle + |1\rangle - |0\rangle + |1\rangle) = |1\rangle$. The $|0\rangle$ contributions cancel.

8.3 $|0\rangle \to |+\rangle \to |+\rangle$ (X leaves $|+\rangle$ unchanged) $\to |0\rangle$; and $|1\rangle \to |-\rangle \to -|-\rangle \to -|1\rangle$. That is the action of Z.

8.4 S twice is a half-turn about z, which is Z. T four times is also Z.

8.5 X then H gives $|-\rangle$. H then S gives $|+\mathrm{i}\rangle$.

8.6 H, Z, H is X, so the qubit ends in $|1\rangle$ and the measurement always gives 1.

Chapter 9

9.1 $(|00\rangle - |01\rangle + |10\rangle - |11\rangle)/2$. The probability of 11 is 1/4.

9.2 Sixteen terms, each with amplitude 1/4.

9.3 About 70 qubits.

9.4 The second qubit is 1 with probability 1/4 + 1/2 = 3/4 and 0 with probability 1/4. After seeing 0 the state is $|00\rangle$. After seeing 1 the remaining terms $(1/2)|01\rangle + (1/\sqrt{2})|11\rangle$, rescaled, give $(1/\sqrt{3})|01\rangle + \sqrt{(2/3)}|11\rangle$.

9.5 No. A product would need αγ, αδ, βγ all positive and βδ negative, but (αδ)(βγ) = (αγ)(βδ) is then positive on one side and negative on the other. The state is entangled.

Chapter 10

10.3 In the 0/1 basis their results always disagree. In the diagonal basis they also always disagree: $|\Psi^-\rangle = (|+-\rangle - |-+\rangle)/\sqrt{2}$, up to a sign.

10.4 Z on the first qubit gives $|\Phi^-\rangle$. X on the first qubit of that gives $(|10\rangle - |01\rangle)/\sqrt{2} = -|\Psi^-\rangle$.

Chapter 11

11.1 $|10\rangle$ gives $|\Phi^-\rangle$, $|01\rangle$ gives $|\Psi^+\rangle$, and $|11\rangle$ gives $(|01\rangle - |10\rangle)/\sqrt{2} = |\Psi^-\rangle$.

11.2 Neither result is entangled. $|+\rangle|+\rangle$ is unchanged. $|+\rangle|-\rangle$ becomes $|-\rangle|-\rangle$: the CNOT flips the target only in the $|1\rangle$ branch, and since $X|-\rangle = -|-\rangle$ this puts a minus sign on that branch, changing the control from $|+\rangle$ to $|-\rangle$. This is phase kickback (Chapter 13).

11.3 $|01\rangle \to |01\rangle \to |11\rangle \to |10\rangle$. $|10\rangle \to |11\rangle \to |01\rangle \to |01\rangle$. The states have swapped.

11.4 Apply X to a and b, compute their AND into a fresh $|0\rangle$ with a Toffoli, apply X to the result, then restore a and b with X gates.

11.5 Width 20, depth 10, 200 gates.

Chapter 12

12.1 By linearity it produces $(|+\rangle|+\rangle + |-\rangle|-\rangle)/\sqrt{2}$, which equals $(|00\rangle + |11\rangle)/\sqrt{2}$, an entangled state, not $|0\rangle|0\rangle$.

12.3 Agreement probabilities are 1/2, 1/2, 1 and (for the case needing disagreement) 1. The average is 3/4, exactly the classical limit.

Chapter 13

13.1 For f_2 the qubit is $-|+\rangle$ and the measurement gives 0 (constant). For f_4 it is $-|-\rangle$ and the measurement gives 1 (balanced).

13.2 The oracle leaves $|+\rangle$ unchanged whether f(x) is 0 or 1, so no sign is kicked back and the algorithm learns nothing.

13.4 0, 1 and 0. (For s = 110 the box returns $x_1 \oplus x_2$.)

13.5 About $(1/2)^9 \approx 0.2\%$.

Chapter 14

14.2 Bob's qubit is $|1\rangle$, $|0\rangle$, $-|1\rangle$ or $-|0\rangle$ for Alice's results 00, 01, 10, 11. Each occurs with probability 1/4, so before correction Bob's qubit is equally likely to be $|0\rangle$ or $|1\rangle$ and tells him nothing.

14.3 $00 \to |\Phi^+\rangle$, $01 \to |\Psi^+\rangle$, $10 \to |\Phi^-\rangle$, $11 \to |\Psi^-\rangle$ (up to a sign), where the first bit decides Z and the second X.

14.4 100 entangled pairs and 200 classical bits.

Chapter 15

15.1 The first four arrows are evenly spread and sum to zero. The second set gives two arrows pointing right and two pointing left, also summing to zero.

15.2 At 0, 4, 8 and 12.

15.3 The amplitudes are $1/2$, $i/2$, $-1/2$ and $-i/2$.

15.4 For 4 qubits: 4 Hadamards and 6 rotations, 10 gates. For 10 qubits: 10 + 45 = 55 gates.

Chapter 16

16.1 1, 2, 4, 8, 1, 2, 4: period 4. Then $2^2 = 4$, and the greatest common divisors of 3 and 5 with 15 give 3 and 5.

16.2 1, 14, 1, 14, 1: period 2. Then $a^{r/2} = 14$, and 14 + 1 = 15 is a multiple of 15, so the divisors found are trivial (1 and 15). Another a must be tried.

16.3 221 mod 91 = 39; 91 mod 39 = 13; 39 mod 13 = 0. The answer is 13, and indeed 221 = 13 × 17.

16.4 x = 3, 7, 11, 15. The peaks are in the same places (0, 4, 8, 12); only their phases differ.

16.5 2^x mod 21: 1, 2, 4, 8, 16, 11, 1, so r = 6. Then $2^3 = 8$, and the greatest common divisors of 7 and 9 with 21 are 7 and 3.

Chapter 17

17.1 Initial amplitudes 1/2. After marking, the mean is 1/4; unmarked amplitudes become 0 and the marked one becomes 1.

17.2 The success probability falls to about 0.33. The state has rotated past the marked item.

17.3 About 785 rounds for 10^6 items and about 785,000 for 10^{12}.

17.4 With more marked items the starting angle is larger, so fewer rounds are needed: about $(\pi/4)\sqrt{(N/M)}$.

Chapter 19

19.1 The message is 1010. Reusing a key is fatal because the XOR of two ciphertexts equals the XOR of the two messages, with the key cancelled out.

19.2 Five of eight. On average one half survive.

19.3 $(3/4)^4 \approx 0.32$ for 4 bits. For 100 bits, $(3/4)^{100} \approx 3 \times 10^{-13}$.

19.4 About 2.5%.

Chapter 20

20.1 About 2,000 gates.

20.2 $e^{-1} \approx 0.37$.

20.3 $|+\rangle$ ends at the centre after dephasing and at the north pole ($|0\rangle$) after relaxation. $|1\rangle$ is unaffected by dephasing.

Chapter 21

21.1 For p = 0.1: 3 × 0.01 × 0.9 + 0.001 = 0.028, better than 0.1. For p = 0.6 the vote is wrong with probability about 0.65, worse than not encoding.

21.2 $\alpha|001\rangle + \beta|110\rangle$. Qubits 1 and 2 agree; qubits 2 and 3 disagree.

21.4 Both give $\alpha|000\rangle - \beta|111\rangle$, a valid code state that the parity checks cannot distinguish from an undamaged one.

21.5 About 98 qubits for distance 7 and about 1,250 for distance 25.

Chapter 22

22.1 About −273.14 °C, roughly 270 times colder than deep space on the kelvin scale.

22.2 2×10^7 and 5,000 gates per second. 10^9 gates take 50 seconds and about 2.3 days respectively.

Chapter 23

23.1 Three bits; about 2.58 bits.

23.2 $(1/4) \times 2 + (3/4) \times 0.415 \approx 0.81$ bits.

23.3 Zero for $|+\rangle$ (a pure state); one bit at the centre.

Chapter 24

24.2 About $2^{30} \approx 10^9$ calls, around 20 minutes at one microsecond per call, ignoring error-correction overheads, which in practice make each call far slower.

Further reading

This book is a beginning. Here are some suggestions for where to go next, grouped by what you want to do and roughly in order of difficulty within each group. All of them are by other authors and are listed simply as good next steps. Resources marked *free* can be read or used at no cost; editions and web addresses change, so a quick search by title is the most reliable way to find them. The original research papers behind the results in this book are listed separately in the References that follow.

Free resources to start with

Andy Matuschak and Michael Nielsen, *Quantum Country* (quantum.country). *Free.* An online introduction to quantum computing with built-in review questions that reappear over the following weeks so that the ideas stick. It goes a little further into the mathematics than this book and is an ideal next step.

John Watrous, *Understanding Quantum Information and Computation* (IBM Quantum Learning, 2024). *Free.* A complete online course with videos and written lessons, covering the basics through quantum algorithms and error correction, with exercises you can run on real hardware.

IBM Quantum Learning and the Qiskit documentation. *Free.* Tutorials that take you from a single qubit to the algorithms of Part III in code.

Ronald de Wolf, *Quantum Computing: Lecture Notes* (arXiv:1907.09415, updated regularly). *Free.* Clear, compact university lecture notes, strong on algorithms, assuming basic linear algebra.

Scott Aaronson, *Introduction to Quantum Information Science*, lecture notes (University of Texas at Austin). *Free.* An undergraduate course in Aaronson's characteristically clear and witty style.

Andrew Childs, *Lecture Notes on Quantum Algorithms* (University of Maryland). *Free.* A more advanced survey of quantum algorithms, for when you want to see what lies beyond Shor and Grover.

John Preskill, *Lecture Notes for Physics 219: Quantum Computation* (Caltech). *Free.* Superb, detailed notes on quantum information, entanglement and error correction, at graduate level.

R. P. Feynman, R. B. Leighton and M. Sands, *The Feynman Lectures on Physics*, Volume III (feynmanlectures.caltech.edu). *Free online.* Feynman's own introduction to quantum mechanics, which begins, like Chapter 4 of this book, with interference experiments.

Mathematics first

The single most useful preparation for going further is *linear algebra*: vectors, matrices, complex numbers, eigenvalues and eigenvectors. **Gilbert Strang's linear algebra lectures (MIT OpenCourseWare 18.06)** are *free* and have introduced the subject to millions of students. **3Blue1Brown, *Essence of Linear Algebra*** is a *free* series of short animated videos that builds geometric intuition in the spirit of the arrows used in this book.

Gentle books

Chris Bernhardt, *Quantum Computing for Everyone* (MIT Press, 2019). A short, friendly book using only high-school mathematics, going a little further into the algebra than this one.

Leonard Susskind and Art Friedman, *Quantum Mechanics: The Theoretical Minimum* (Basic Books, 2014). An introduction to quantum mechanics itself, for readers willing to work through some mathematics, written by a leading physicist for non-specialists.

Scott Aaronson, *Quantum Computing Since Democritus* (Cambridge University Press, 2013). A lively, opinionated tour of quantum computing, complexity theory and the foundations of physics. Some chapters are demanding, but the writing is a pleasure.

N. David Mermin, *Quantum Computer Science: An Introduction* (Cambridge University Press, 2007). A compact, elegant introduction for computer scientists, assuming some comfort with linear algebra.

Learning by programming

Eric R. Johnston, Nic Harrigan and Mercedes Gimeno-Segovia, *Programming Quantum Computers* (O'Reilly, 2019). Teaches quantum computing through code and visual "circle notation", with examples that run in a web browser.

Robert S. Sutor, *Dancing with Qubits* (Packt, 2nd edition 2024). Builds up the necessary mathematics from scratch and connects it to programming real quantum computers.

Open-source frameworks such as **Qiskit**, **Cirq** and others are *free*, come with simulators that run on an ordinary laptop, and give access to small real devices over the internet. The circuits in this book make a good set of first exercises: create a Bell pair, check its correlations in two bases, teleport a qubit, and run Grover's algorithm on two qubits.

The standard textbooks

Michael A. Nielsen and Isaac L. Chuang, *Quantum Computation and Quantum Information* (Cambridge University Press, 10th anniversary edition 2010). The classic graduate textbook, often called "Mike and Ike". Comprehensive and authoritative, covering everything in this book and much more in full mathematical detail. Best approached after linear algebra and a gentler introduction.

Phillip Kaye, Raymond Laflamme and Michele Mosca, *An Introduction to Quantum Computing* (Oxford University Press, 2007). A careful textbook focused on algorithms, suitable for advanced undergraduates.

Mark M. Wilde, *Quantum Information Theory* (Cambridge University Press, 2nd edition 2017). The standard reference for the material of Chapter 23 and much more; a *free* preprint version is available on arXiv.

History and context

Jim Baggott, *The Quantum Story* (Oxford University Press, 2011) tells the history of quantum mechanics from Planck to the present. **Louisa Gilder, *The Age of Entanglement* (Knopf, 2008)** tells the story of entanglement, from Einstein's

objections to Bell's theorem and the experiments, partly through reconstructed conversations.

References

These are the original sources for the main results, experiments and historical facts described in the book, grouped by chapter. Most of the research papers can be found free of charge on the arXiv preprint server (arxiv.org) or on the authors' web pages. Some works appear only under the chapter where they are first discussed.

Chapter 1: Why build a quantum computer?

G. E. Moore, "Cramming more components onto integrated circuits", *Electronics* **38**(8), 114–117 (1965).

M. Planck, "Ueber das Gesetz der Energieverteilung im Normalspectrum", *Annalen der Physik* **4**, 553–563 (1901).

P. Benioff, "The computer as a physical system: a microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines", *Journal of Statistical Physics* **22**, 563–591 (1980).

Yu. I. Manin, *Computable and Uncomputable* (Sovetskoye Radio, Moscow, 1980), in Russian.

R. P. Feynman, "Simulating physics with computers", *International Journal of Theoretical Physics* **21**, 467–488 (1982).

Chapter 2: How an ordinary computer thinks

R. Landauer, "Irreversibility and heat generation in the computing process", *IBM Journal of Research and Development* **5**, 183–191 (1961).

C. H. Bennett, "Logical reversibility of computation", *IBM Journal of Research and Development* **17**, 525–532 (1973).

Chapter 4: Light, waves and a puzzle

T. Young, "The Bakerian Lecture: On the theory of light and colours", *Philosophical Transactions of the Royal Society of London* **92**, 12–48 (1802).

A. Einstein, "Über einen die Erzeugung und Verwandlung des Lichtes betreffenden heuristischen Gesichtspunkt", *Annalen der Physik* **17**, 132–148 (1905).

G. I. Taylor, "Interference fringes with feeble light", *Proceedings of the Cambridge Philosophical Society* **15**, 114–115 (1909).

M. Born, "Zur Quantenmechanik der Stoßvorgänge", *Zeitschrift für Physik* **37**, 863–867 (1926).

P. Grangier, G. Roger and A. Aspect, "Experimental evidence for a photon anticorrelation effect on a beam splitter: a new light on single-photon interferences", *Europhysics Letters* **1**, 173–179 (1986).

Chapter 5: The qubit

P. A. M. Dirac, "A new notation for quantum mechanics", *Mathematical Proceedings of the Cambridge Philosophical Society* **35**, 416–418 (1939).

E. Schrödinger, "Die gegenwärtige Situation in der Quantenmechanik", *Die Naturwissenschaften* **23**, 807–812, 823–828 and 844–849 (1935).

A. S. Holevo, "Bounds for the quantity of information transmitted by a quantum communication channel", *Problems of Information Transmission* **9**, 177–183 (1973).

Chapters 6 and 7: Measurement and the Bloch sphere

W. Heisenberg, "Über den anschaulichen Inhalt der quantentheoretischen Kinematik und Mechanik", *Zeitschrift für Physik* **43**, 172–198 (1927).

F. Bloch, "Nuclear induction", *Physical Review* **70**, 460–474 (1946).

Chapter 8: Quantum gates

W. Pauli, "Zur Quantenmechanik des magnetischen Elektrons", *Zeitschrift für Physik* **43**, 601–623 (1927).

Chapter 10: Entanglement

A. Einstein, B. Podolsky and N. Rosen, "Can quantum-mechanical description of physical reality be considered complete?", *Physical Review* **47**, 777–780 (1935).

E. Schrödinger, "Discussion of probability relations between separated systems", *Mathematical Proceedings of the Cambridge Philosophical Society* **31**, 555–563 (1935).

D. M. Greenberger, M. A. Horne and A. Zeilinger, "Going beyond Bell's theorem", in M. Kafatos (ed.), *Bell's Theorem, Quantum Theory and Conceptions of the Universe* (Kluwer, Dordrecht, 1989), 69–72.

J. Yin et al., "Satellite-based entanglement distribution over 1200 kilometers", *Science* **356**, 1140–1144 (2017).

Chapter 11: Two-qubit gates and quantum circuits

T. Toffoli, "Reversible computing", in *Automata, Languages and Programming*, Lecture Notes in Computer Science **85** (Springer, 1980), 632–644.

A. Barenco et al., "Elementary gates for quantum computation", *Physical Review A* **52**, 3457–3467 (1995).

C. M. Dawson and M. A. Nielsen, "The Solovay–Kitaev algorithm", *Quantum Information and Computation* **6**, 81–95 (2006).

Chapter 12: What the quantum world forbids

W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned", *Nature* **299**, 802–803 (1982).

D. Dieks, "Communication by EPR devices", *Physics Letters A* **92**, 271–272 (1982).

J. S. Bell, "On the Einstein Podolsky Rosen paradox", *Physics* **1**, 195–200 (1964).

J. F. Clauser, M. A. Horne, A. Shimony and R. A. Holt, "Proposed experiment to test local hidden-variable theories", *Physical Review Letters* **23**, 880–884 (1969).

B. S. Cirel'son (Tsirelson), "Quantum generalizations of Bell's inequality", *Letters in Mathematical Physics* **4**, 93–100 (1980).

S. J. Freedman and J. F. Clauser, "Experimental test of local hidden-variable theories", *Physical Review Letters* **28**, 938–941 (1972).

A. Aspect, J. Dalibard and G. Roger, "Experimental test of Bell's inequalities using time-varying analyzers", *Physical Review Letters* **49**, 1804–1807 (1982).

B. Hensen et al., "Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres", *Nature* **526**, 682–686 (2015).

M. Giustina et al., "Significant-loophole-free test of Bell's theorem with entangled photons", *Physical Review Letters* **115**, 250401 (2015).

L. K. Shalm et al., "Strong loophole-free test of local realism", *Physical Review Letters* **115**, 250402 (2015).

Chapter 13: Interference as a way of computing

D. Deutsch, "Quantum theory, the Church–Turing principle and the universal quantum computer", *Proceedings of the Royal Society of London A* **400**, 97–117 (1985).

D. Deutsch and R. Jozsa, "Rapid solution of problems by quantum computation", *Proceedings of the Royal Society of London A* **439**, 553–558 (1992).

R. Cleve, A. Ekert, C. Macchiavello and M. Mosca, "Quantum algorithms revisited", *Proceedings of the Royal Society of London A* **454**, 339–354 (1998).

E. Bernstein and U. Vazirani, "Quantum complexity theory", *SIAM Journal on Computing* **26**, 1411–1473 (1997); conference version in *Proceedings of the 25th ACM Symposium on Theory of Computing* (1993).

Chapter 14: Teleportation and superdense coding

C. H. Bennett and S. J. Wiesner, "Communication via one- and two-particle operators on Einstein–Podolsky–Rosen states", *Physical Review Letters* **69**, 2881–2884 (1992).

C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein–Podolsky–Rosen channels", *Physical Review Letters* **70**, 1895–1899 (1993).

D. Bouwmeester et al., "Experimental quantum teleportation", *Nature* **390**, 575–579 (1997).

D. Boschi et al., "Experimental realization of teleporting an unknown pure quantum state via dual classical and Einstein–Podolsky–Rosen channels", *Physical Review Letters* **80**, 1121–1125 (1998).

X.-S. Ma et al., “Quantum teleportation over 143 kilometres using active feed-forward”, *Nature* **489**, 269–273 (2012).

J.-G. Ren et al., “Ground-to-satellite quantum teleportation”, *Nature* **549**, 70–73 (2017).

Chapter 15: The quantum Fourier transform

J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series”, *Mathematics of Computation* **19**, 297–301 (1965).

D. Coppersmith, “An approximate Fourier transform useful in quantum factoring”, IBM Research Report RC 19642 (1994).

A. Yu. Kitaev, “Quantum measurements and the Abelian stabilizer problem”, arXiv:quant-ph/9511026 (1995).

Chapter 16: Shor’s algorithm

R. L. Rivest, A. Shamir and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems”, *Communications of the ACM* **21**, 120–126 (1978).

P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring”, *Proceedings of the 35th Annual Symposium on Foundations of Computer Science* (IEEE, 1994), 124–134. Expanded version: “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer”, *SIAM Journal on Computing* **26**, 1484–1509 (1997).

L. M. K. Vandersypen et al., “Experimental realization of Shor’s quantum factoring algorithm using nuclear magnetic resonance”, *Nature* **414**, 883–887 (2001).

C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”, *Quantum* **5**, 433 (2021); preprint 2019.

C. Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits”, arXiv:2505.15917 (2025).

National Institute of Standards and Technology, Federal Information Processing Standards FIPS 203, 204 and 205 (post-quantum cryptography standards), August 2024.

Chapter 17: Grover’s algorithm

L. K. Grover, “A fast quantum mechanical algorithm for database search”, *Proceedings of the 28th ACM Symposium on Theory of Computing* (1996), 212–219.

L. K. Grover, “Quantum mechanics helps in searching for a needle in a haystack”, *Physical Review Letters* **79**, 325–328 (1997).

C. H. Bennett, E. Bernstein, G. Brassard and U. Vazirani, “Strengths and weaknesses of quantum computing”, *SIAM Journal on Computing* **26**, 1510–1523 (1997).

G. Brassard, P. Høyer, M. Mosca and A. Tapp, “Quantum amplitude amplification and estimation”, *Contemporary Mathematics* **305**, 53–74 (2002).

Chapter 18: Simulating nature

P. Jordan and E. Wigner, "Über das Paulische Äquivalenzverbot", *Zeitschrift für Physik* **47**, 631–651 (1928).

H. F. Trotter, "On the product of semi-groups of operators", *Proceedings of the American Mathematical Society* **10**, 545–551 (1959).

W. Kohn and L. J. Sham, "Self-consistent equations including exchange and correlation effects", *Physical Review* **140**, A1133–A1138 (1965).

S. Lloyd, "Universal quantum simulators", *Science* **273**, 1073–1078 (1996).

A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor", *Nature Communications* **5**, 4213 (2014).

E. Farhi, J. Goldstone and S. Gutmann, "A quantum approximate optimization algorithm", arXiv:1411.4028 (2014).

M. Reiher, N. Wiebe, K. M. Svore, D. Wecker and M. Troyer, "Elucidating reaction mechanisms on quantum computers", *Proceedings of the National Academy of Sciences* **114**, 7555–7560 (2017).

E. Tang, "A quantum-inspired classical algorithm for recommendation systems", *Proceedings of the 51st ACM Symposium on Theory of Computing* (2019), 217–228; preprint 2018.

Chapter 19: Quantum cryptography

C. E. Shannon, "Communication theory of secrecy systems", *Bell System Technical Journal* **28**, 656–715 (1949).

S. Wiesner, "Conjugate coding", *ACM SIGACT News* **15**, 78–88 (1983).

C. H. Bennett and G. Brassard, "Quantum cryptography: public key distribution and coin tossing", *Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore (1984), 175–179.

A. K. Ekert, "Quantum cryptography based on Bell's theorem", *Physical Review Letters* **67**, 661–663 (1991).

H.-K. Lo and H. F. Chau, "Unconditional security of quantum key distribution over arbitrarily long distances", *Science* **283**, 2050–2056 (1999).

P. W. Shor and J. Preskill, "Simple proof of security of the BB84 quantum key distribution protocol", *Physical Review Letters* **85**, 441–444 (2000).

D. Mayers, "Unconditional security in quantum cryptography", *Journal of the ACM* **48**, 351–406 (2001).

S.-K. Liao et al., "Satellite-relayed intercontinental quantum network", *Physical Review Letters* **120**, 030501 (2018).

D. P. Nadlinger et al., "Experimental quantum key distribution certified by Bell's theorem", *Nature* **607**, 682–686 (2022).

Chapter 20: Noise

W. H. Zurek, "Decoherence, einselection, and the quantum origins of the classical", *Reviews of Modern Physics* **75**, 715–775 (2003).

K. Temme, S. Bravyi and J. M. Gambetta, "Error mitigation for short-depth quantum circuits", *Physical Review Letters* **119**, 180509 (2017).

J. Preskill, "Quantum computing in the NISQ era and beyond", *Quantum* **2**, 79 (2018).

F. Arute et al., "Quantum supremacy using a programmable superconducting processor", *Nature* **574**, 505–510 (2019).

Chapter 21: Protecting quantum information

P. W. Shor, "Scheme for reducing decoherence in quantum computer memory", *Physical Review A* **52**, R2493–R2496 (1995).

A. M. Steane, "Error correcting codes in quantum theory", *Physical Review Letters* **77**, 793–797 (1996).

D. Aharonov and M. Ben-Or, "Fault-tolerant quantum computation with constant error", *Proceedings of the 29th ACM Symposium on Theory of Computing* (1997), 176–188.

E. Knill, R. Laflamme and W. H. Zurek, "Resilient quantum computation", *Science* **279**, 342–345 (1998).

D. Gottesman, "The Heisenberg representation of quantum computers", arXiv:quant-ph/9807006 (1998).

A. Yu. Kitaev, "Fault-tolerant quantum computation by anyons", *Annals of Physics* **303**, 2–30 (2003); preprint 1997.

S. Bravyi and A. Kitaev, "Universal quantum computation with ideal Clifford gates and noisy ancillas", *Physical Review A* **71**, 022316 (2005).

A. G. Fowler, M. Mariantoni, J. M. Martinis and A. N. Cleland, "Surface codes: towards practical large-scale quantum computation", *Physical Review A* **86**, 032324 (2012).

D. Bluvstein et al., "Logical quantum processor based on reconfigurable atom arrays", *Nature* **626**, 58–65 (2024).

Google Quantum AI and collaborators, "Quantum error correction below the surface code threshold", *Nature* **638**, 920–926 (2025).

Chapter 22: Building a quantum computer

J. I. Cirac and P. Zoller, "Quantum computations with cold trapped ions", *Physical Review Letters* **74**, 4091–4094 (1995).

C. Monroe, D. M. Meekhof, B. E. King, W. M. Itano and D. J. Wineland, "Demonstration of a fundamental quantum logic gate", *Physical Review Letters* **75**, 4714–4717 (1995).

D. Loss and D. P. DiVincenzo, "Quantum computation with quantum dots", *Physical Review A* **57**, 120–126 (1998).

Y. Nakamura, Yu. A. Pashkin and J. S. Tsai, "Coherent control of macroscopic quantum states in a single-Cooper-pair box", *Nature* **398**, 786–788 (1999).

D. P. DiVincenzo, "The physical implementation of quantum computation", *Fortschritte der Physik* **48**, 771–783 (2000).

D. Jaksch et al., "Fast quantum gates for neutral atoms", *Physical Review Letters* **85**, 2208–2211 (2000).

E. Knill, R. Laflamme and G. J. Milburn, "A scheme for efficient quantum computation with linear optics", *Nature* **409**, 46–52 (2001).

R. Raussendorf and H. J. Briegel, "A one-way quantum computer", *Physical Review Letters* **86**, 5188–5191 (2001).

J. Koch et al., "Charge-insensitive qubit design derived from the Cooper pair box", *Physical Review A* **76**, 042319 (2007).

Chapter 23: Measuring information

J. von Neumann, *Mathematische Grundlagen der Quantenmechanik* (Springer, Berlin, 1932).

C. E. Shannon, "A mathematical theory of communication", *Bell System Technical Journal* **27**, 379–423 and 623–656 (1948).

B. Schumacher, "Quantum coding", *Physical Review A* **51**, 2738–2747 (1995).

Chapter 24: What quantum computers can and cannot do

A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem", *Proceedings of the London Mathematical Society* **s2-42**, 230–265 (1936).

S. A. Cook, "The complexity of theorem-proving procedures", *Proceedings of the 3rd ACM Symposium on Theory of Computing* (1971), 151–158.

M. Agrawal, N. Kayal and N. Saxena, "PRIMES is in P", *Annals of Mathematics* **160**, 781–793 (2004).

S. Aaronson and A. Arkhipov, "The computational complexity of linear optics", *Theory of Computing* **9**, 143–252 (2013); conference version 2011.